

Sequence Models II: Transformer Networks

Advanced Machine Learning and Computer Vision

9 October 2026

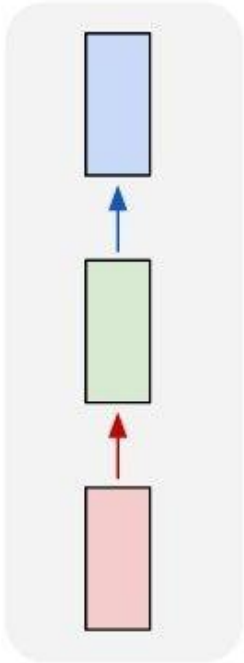
Prof. Fabio Galasso



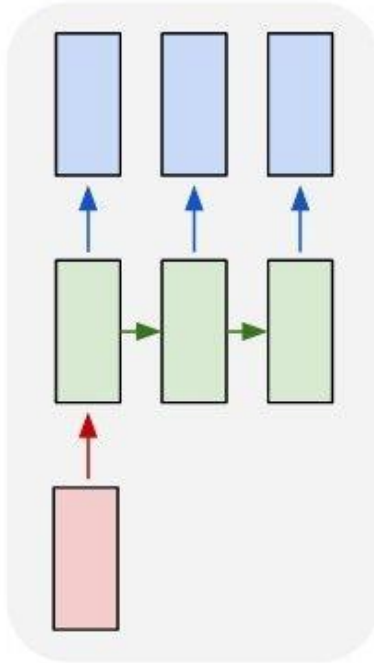
SAPIENZA
UNIVERSITÀ DI ROMA

Process Sequences

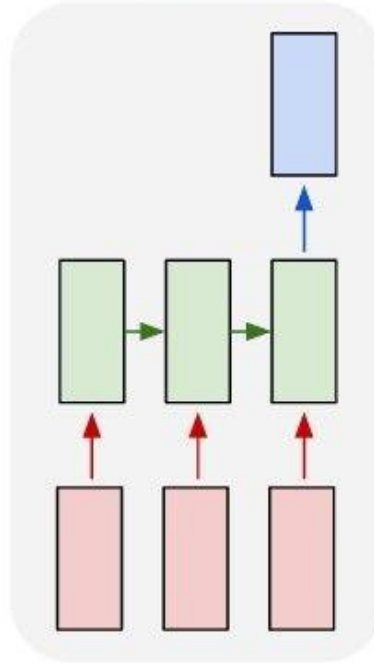
one to one



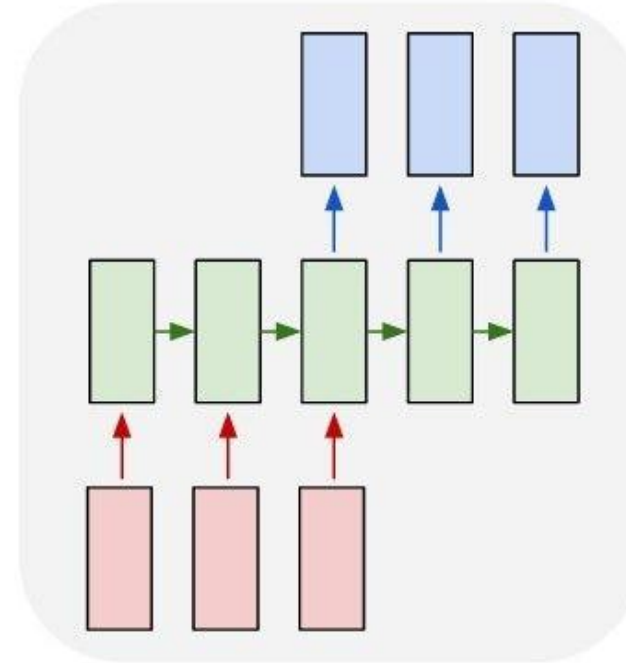
one to many



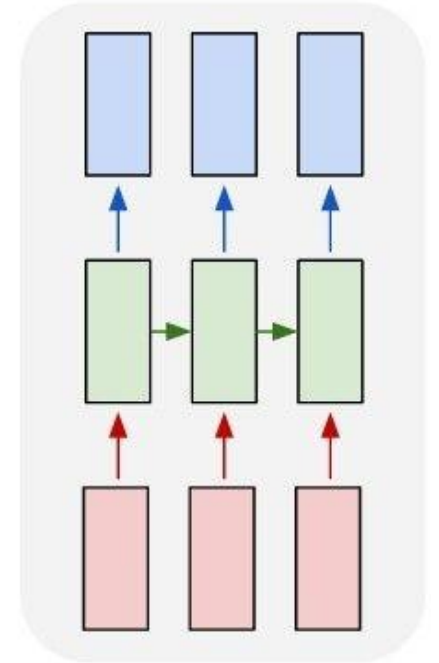
many to one



many to many



many to many



Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- Transformer Networks
- BERT

Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- Transformer Networks
- BERT

Towards Transformers: Attention and self-attention



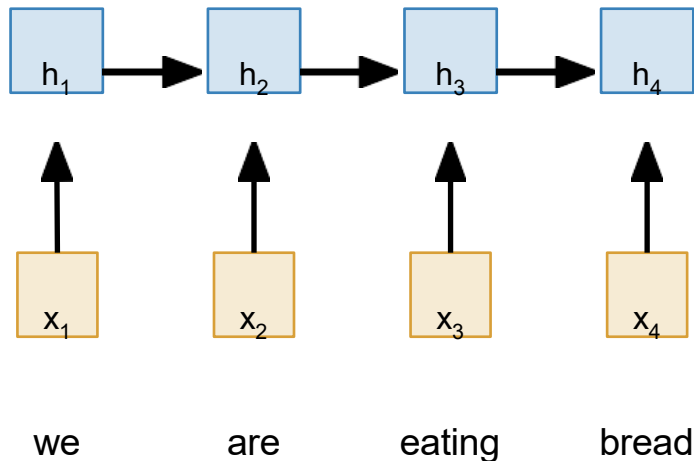
Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

A motivating example: machine translation
English $\rightarrow$ Spanish

Encoder: $h_t = f_W(x_t, h_{t-1})$



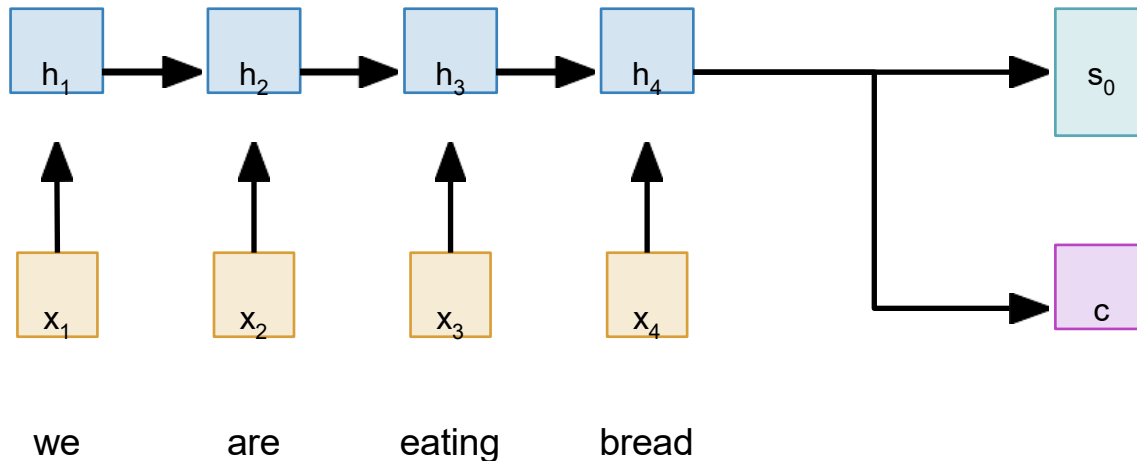
Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

From final hidden state predict:

Encoder: $h_t = f_W(x_t, h_{t-1})$ **Initial decoder state** s_0
Context vector c (often $c=h_T$)



Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

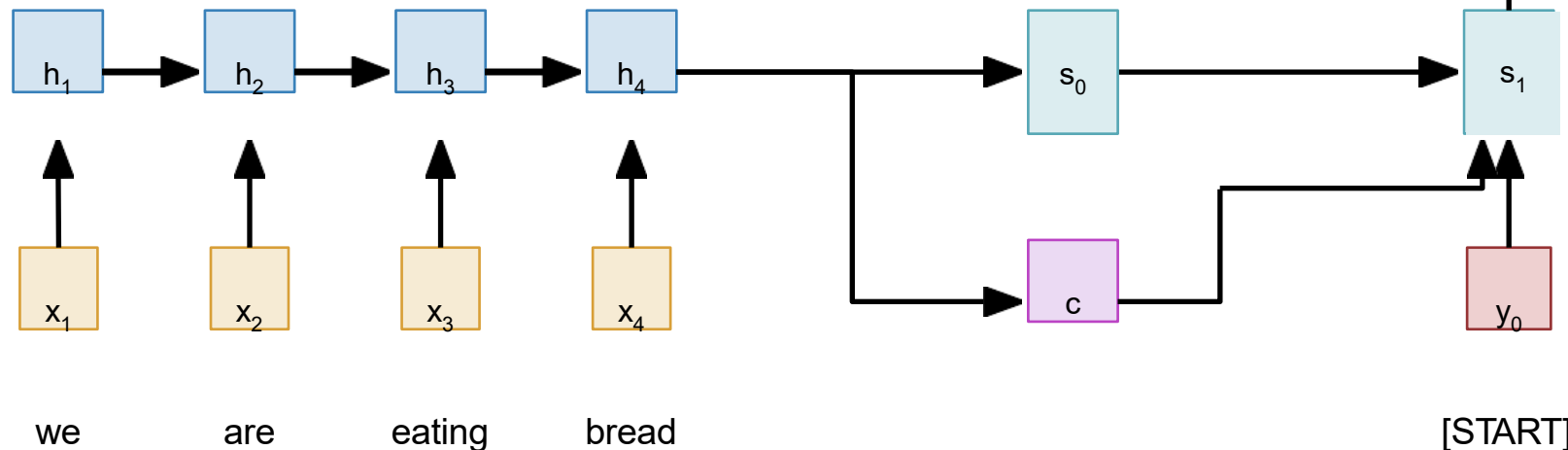
estamos

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

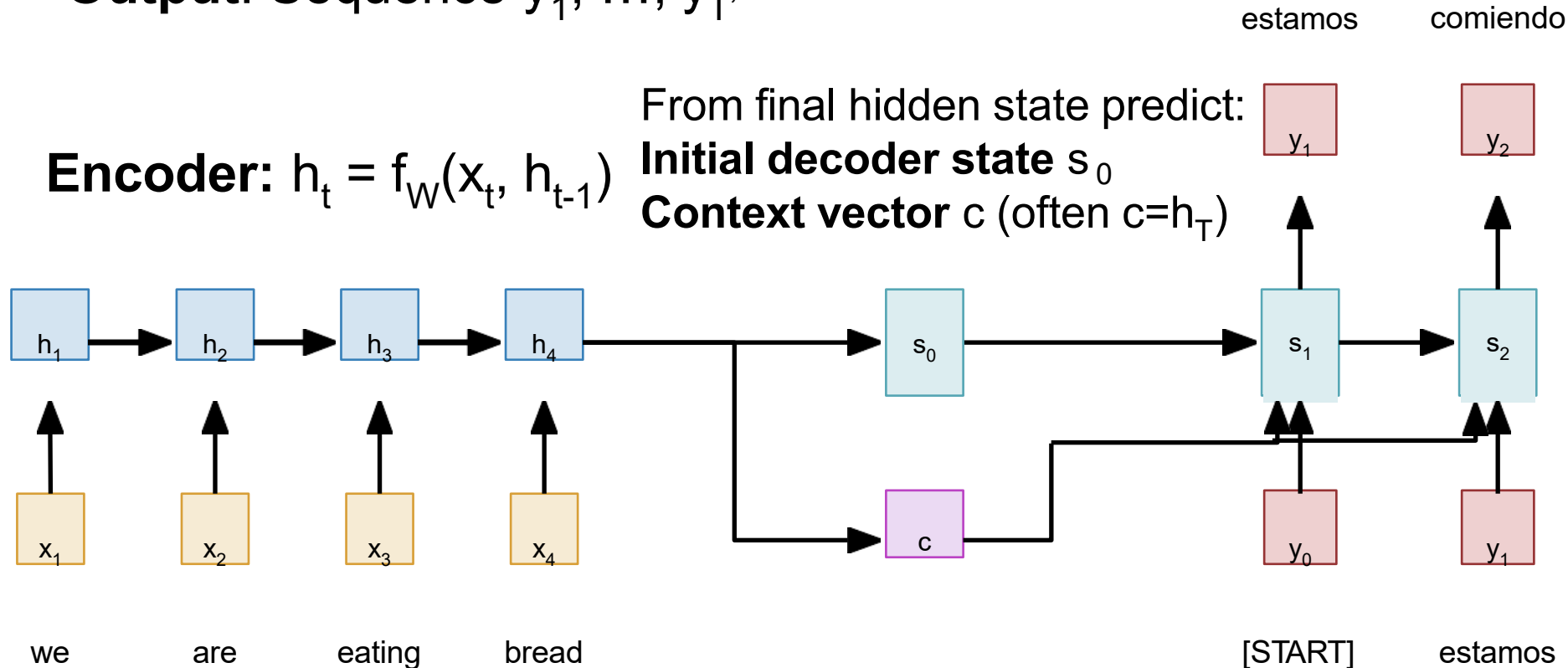
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)



Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

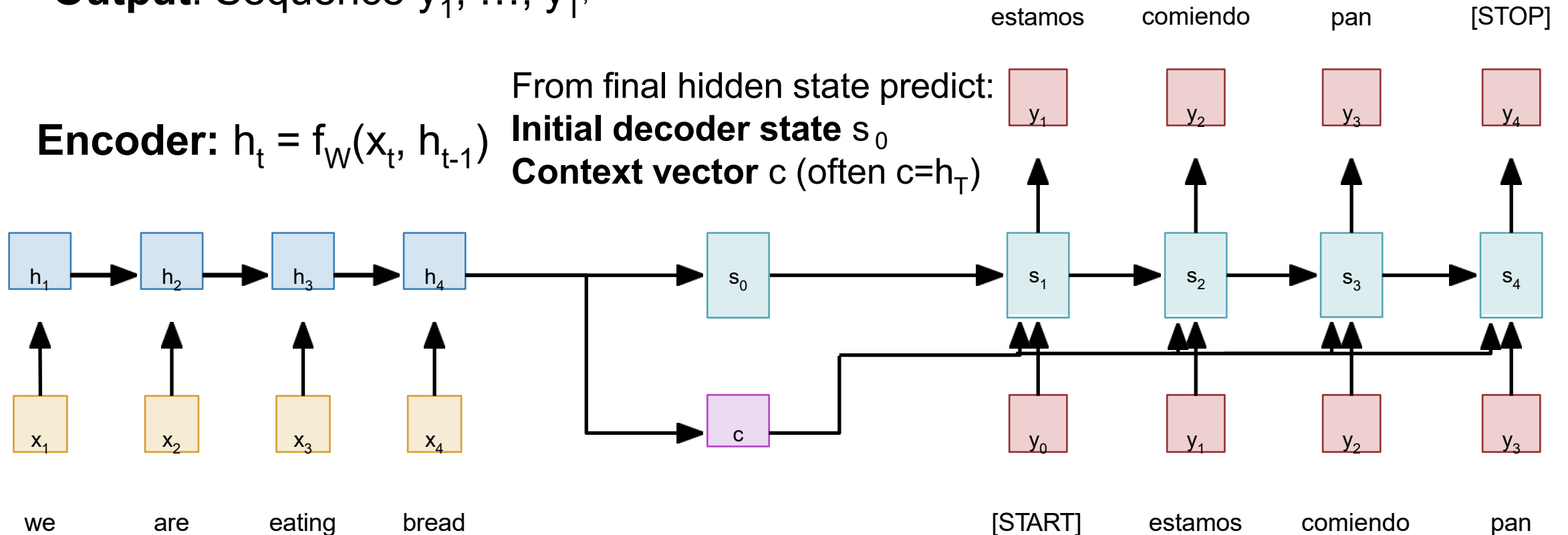
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)

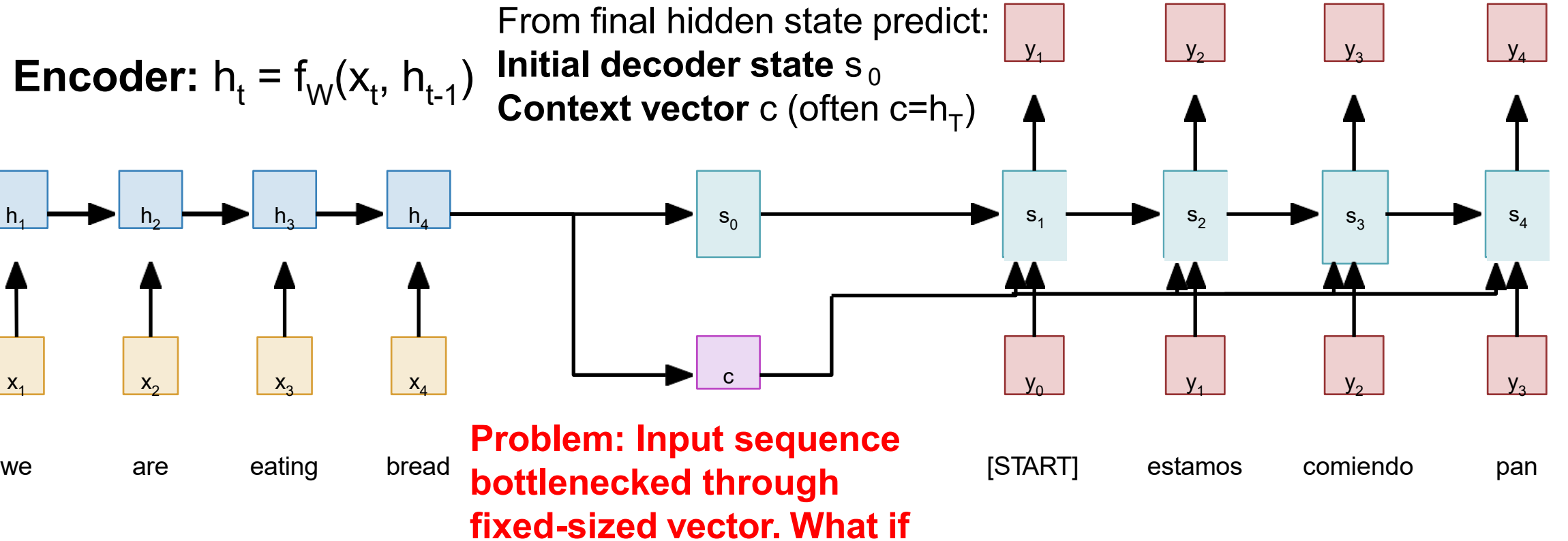


Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

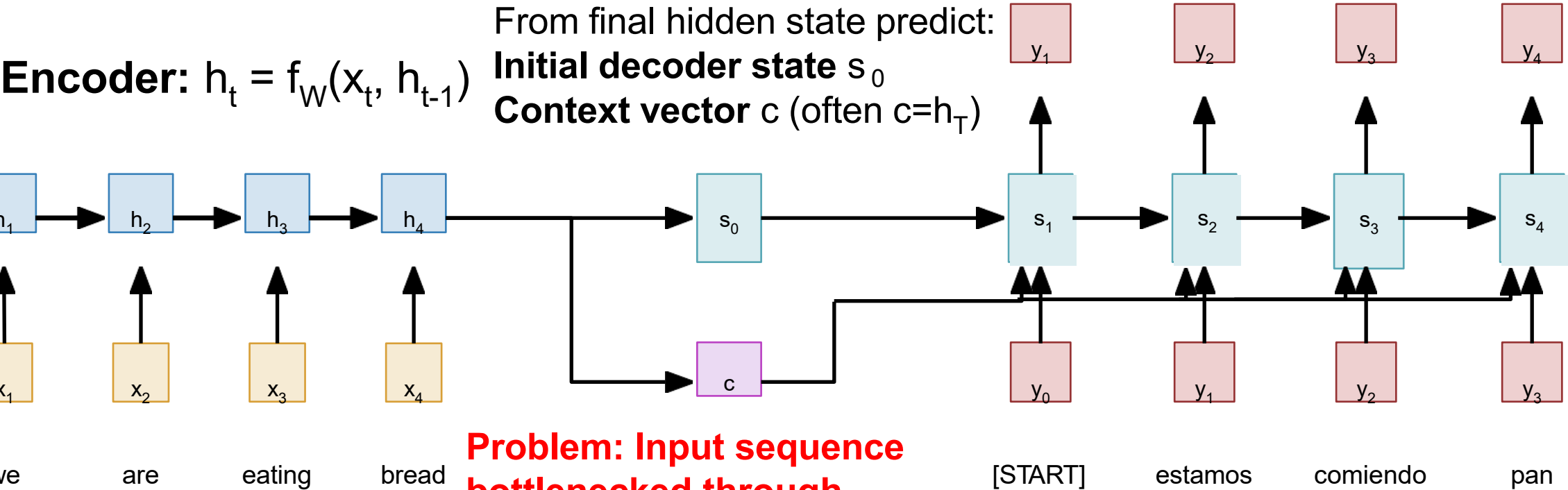
Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$



Sequence to Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$
Output: Sequence $y_1, \dots, y_T$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$



Problem: Input sequence bottlenecked through fixed-sized vector. What if $T=1000$?

Idea: refer to the inputs while we decode

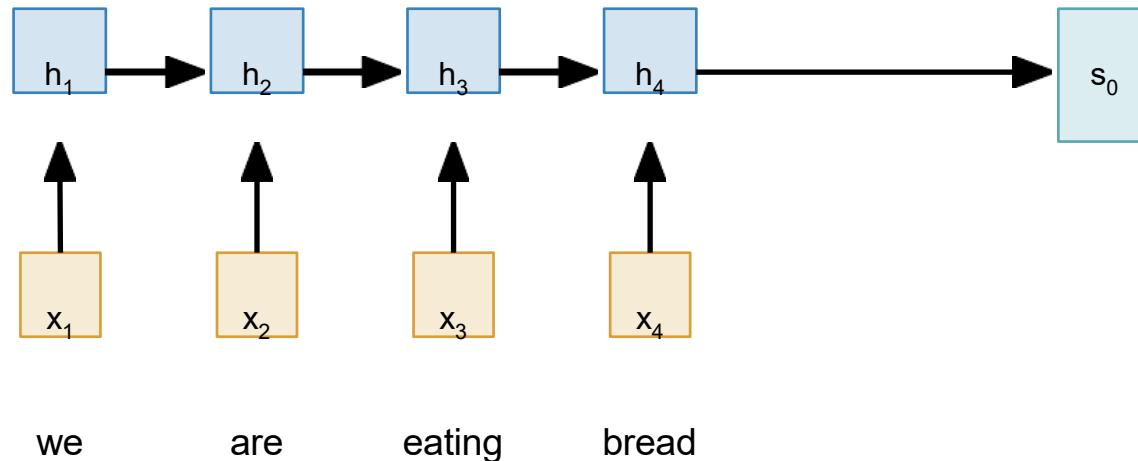
Sutskever et al. "Sequence to sequence learning with neural networks". NeurIPS 2014

Sequence to Sequence with RNNs and Attention

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_T$

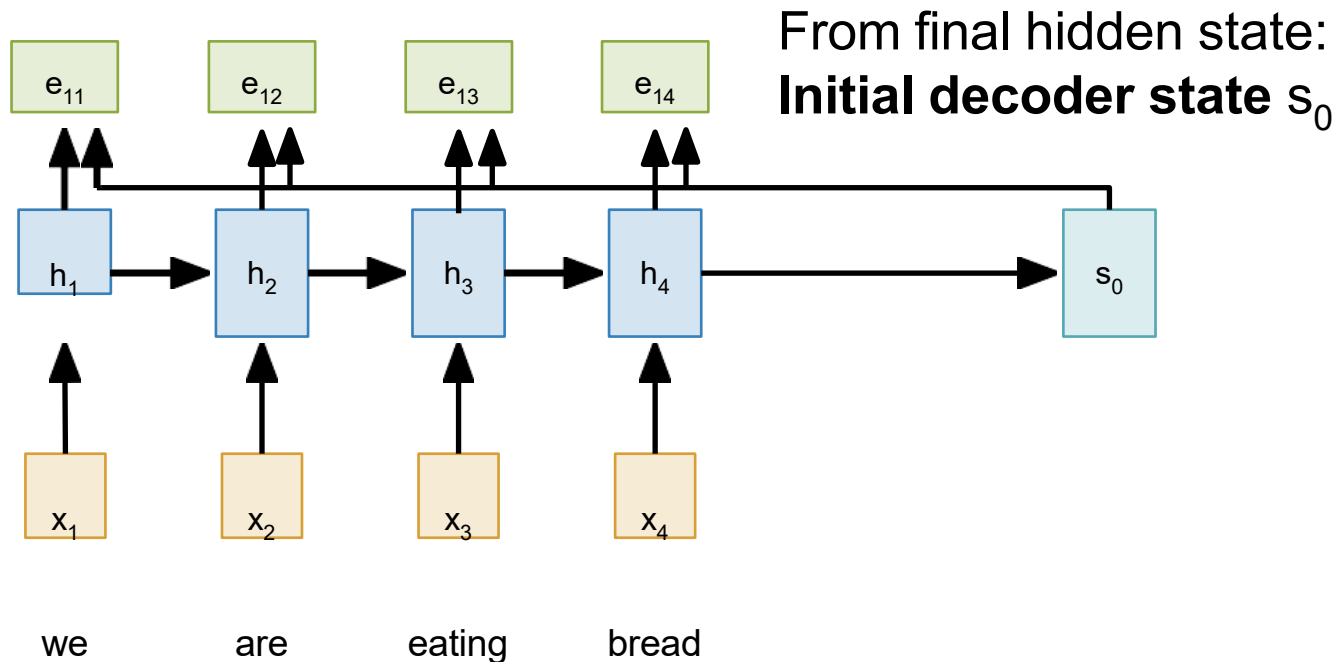
Encoder: $h_t = f_W(x_t, h_{t-1})$ From final hidden state:
Initial decoder state s_0



Sequence to Sequence with RNNs and **Attention**

Compute (scalar) **alignment scores**

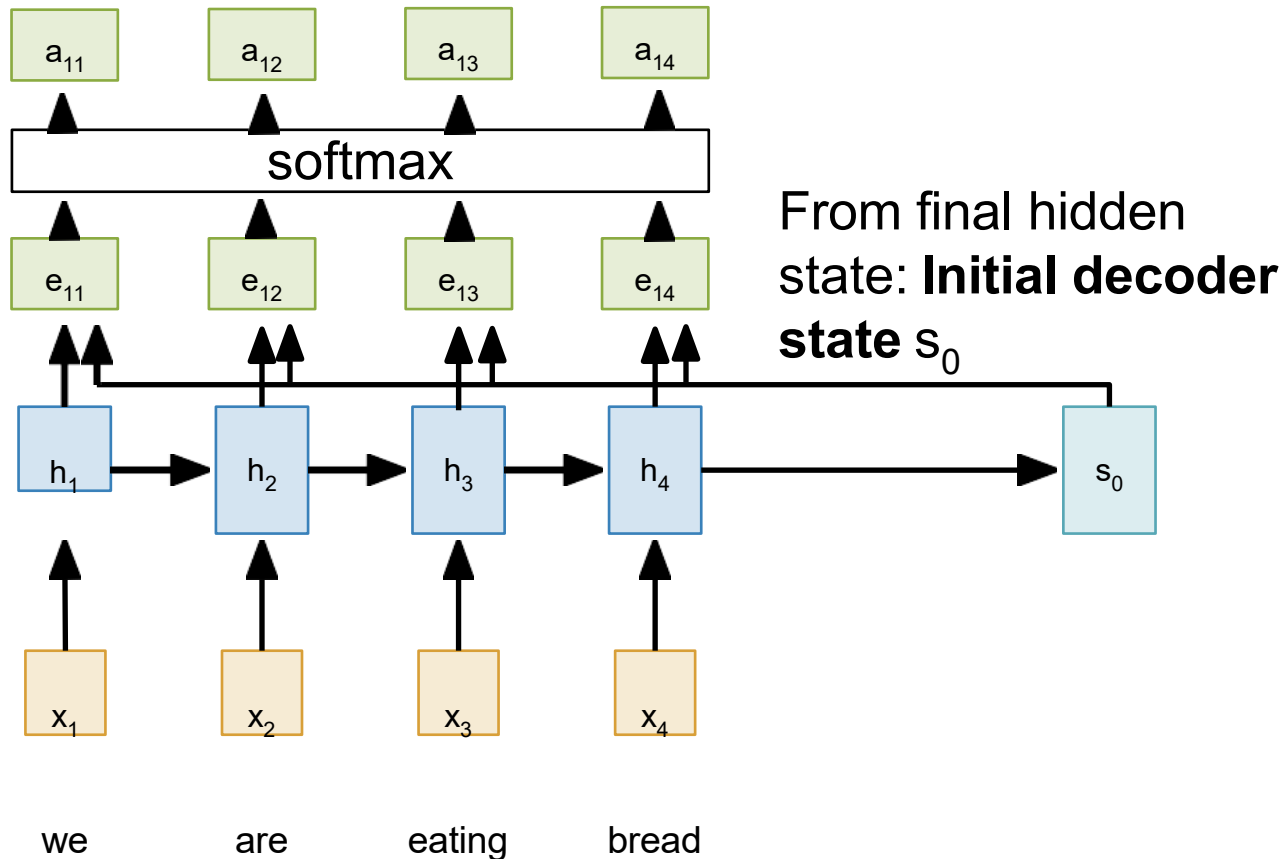
$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i) \quad (f_{\text{att}} \text{ is an MLP})$$



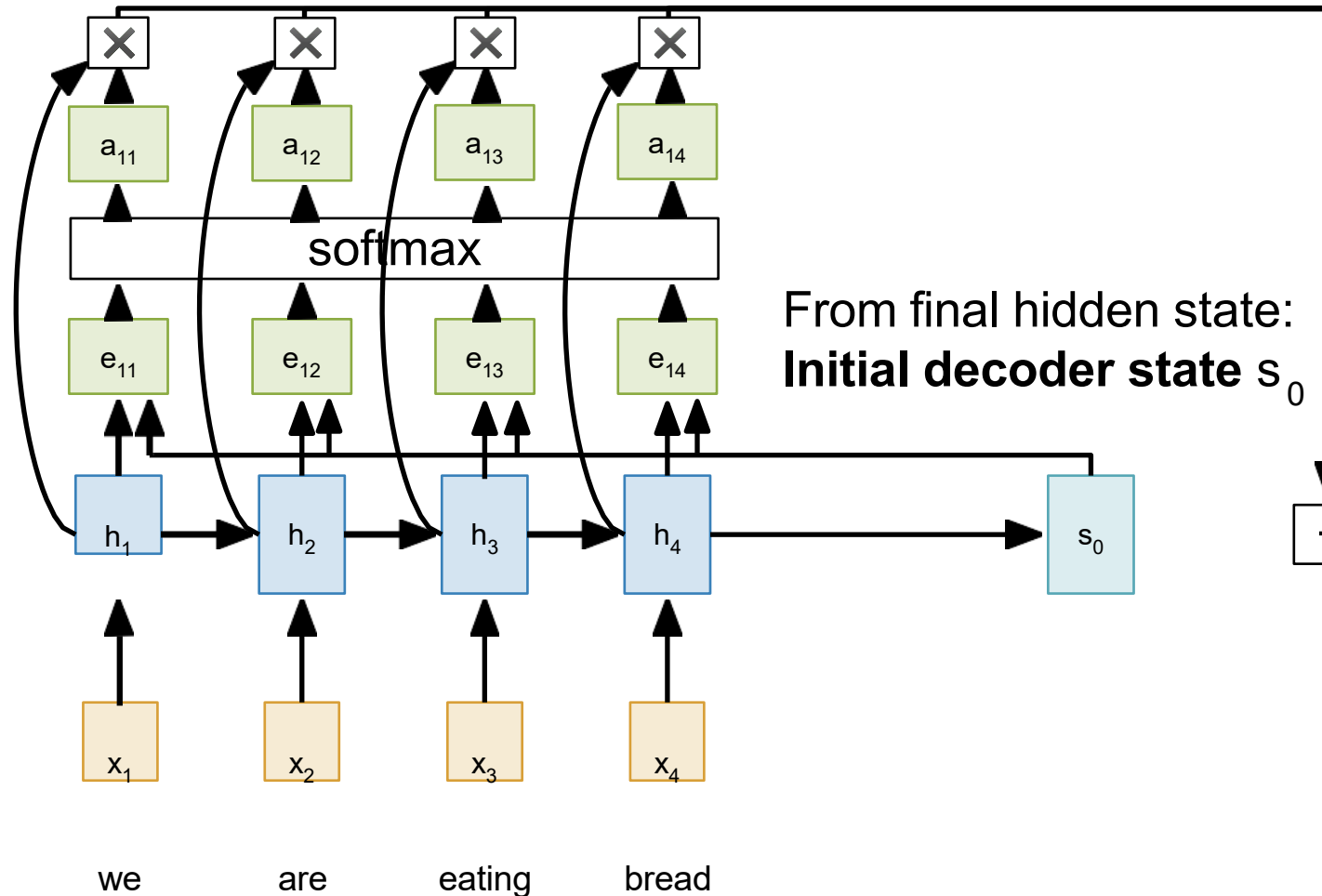
Sequence to Sequence with RNNs and Attention

Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores
to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$



Sequence to Sequence with RNNs and Attention



Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**

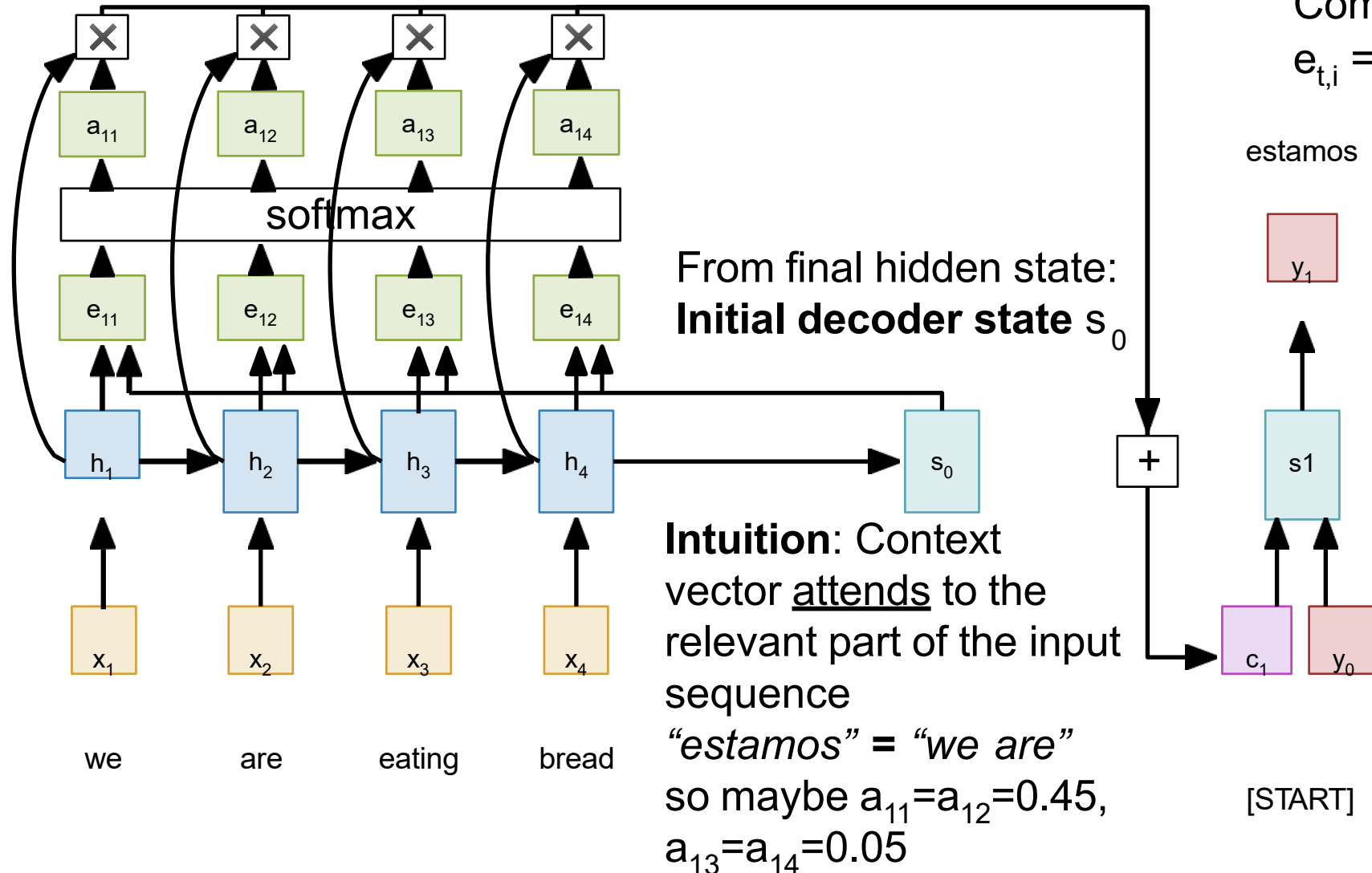
$$0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$$

Compute context vector as **weighted sum of hidden states**

$$c_t = \sum_i a_{t,i} h_i$$

[START]

Sequence to Sequence with RNNs and Attention



Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

Compute context vector as weighted sum of hidden states

$$c_t = \sum_i a_{t,i} h_i$$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

estamos

y_1

$\uparrow$

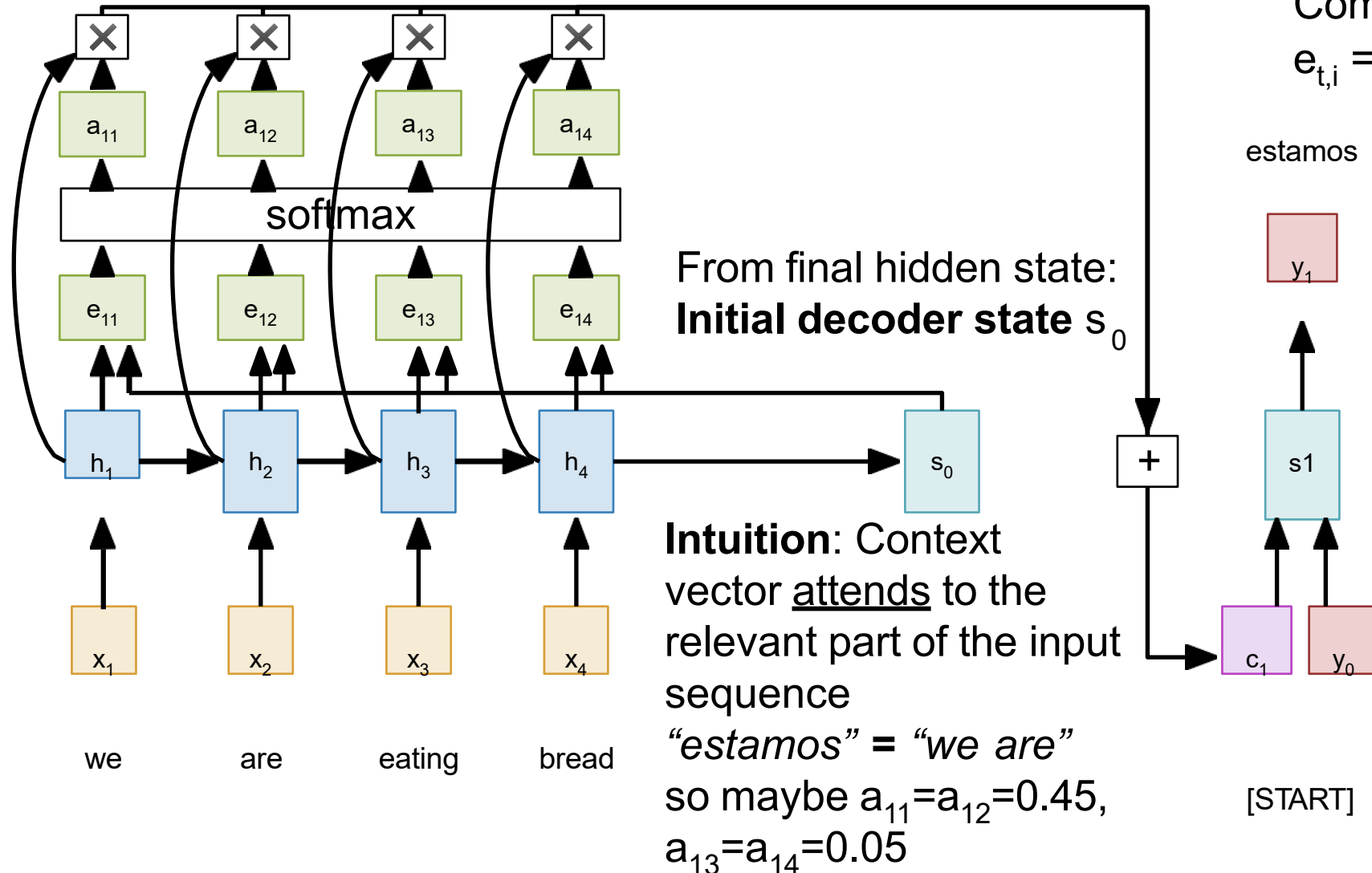
s_1

$\uparrow$

c_1 y_0

[START]

Sequence to Sequence with RNNs and Attention



Compute (scalar) **alignment scores**
 $e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$ (f_{att} is an MLP)

Normalize alignment scores to get **attention weights**
 $0 < a_{t,i} < 1 \quad \sum_i a_{t,i} = 1$

Compute context vector as weighted sum of hidden states

$$c_t = \sum_i a_{t,i} h_i$$

Use context vector in decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c_t)$

This is all differentiable! No supervision on attention weights – backprop through everything

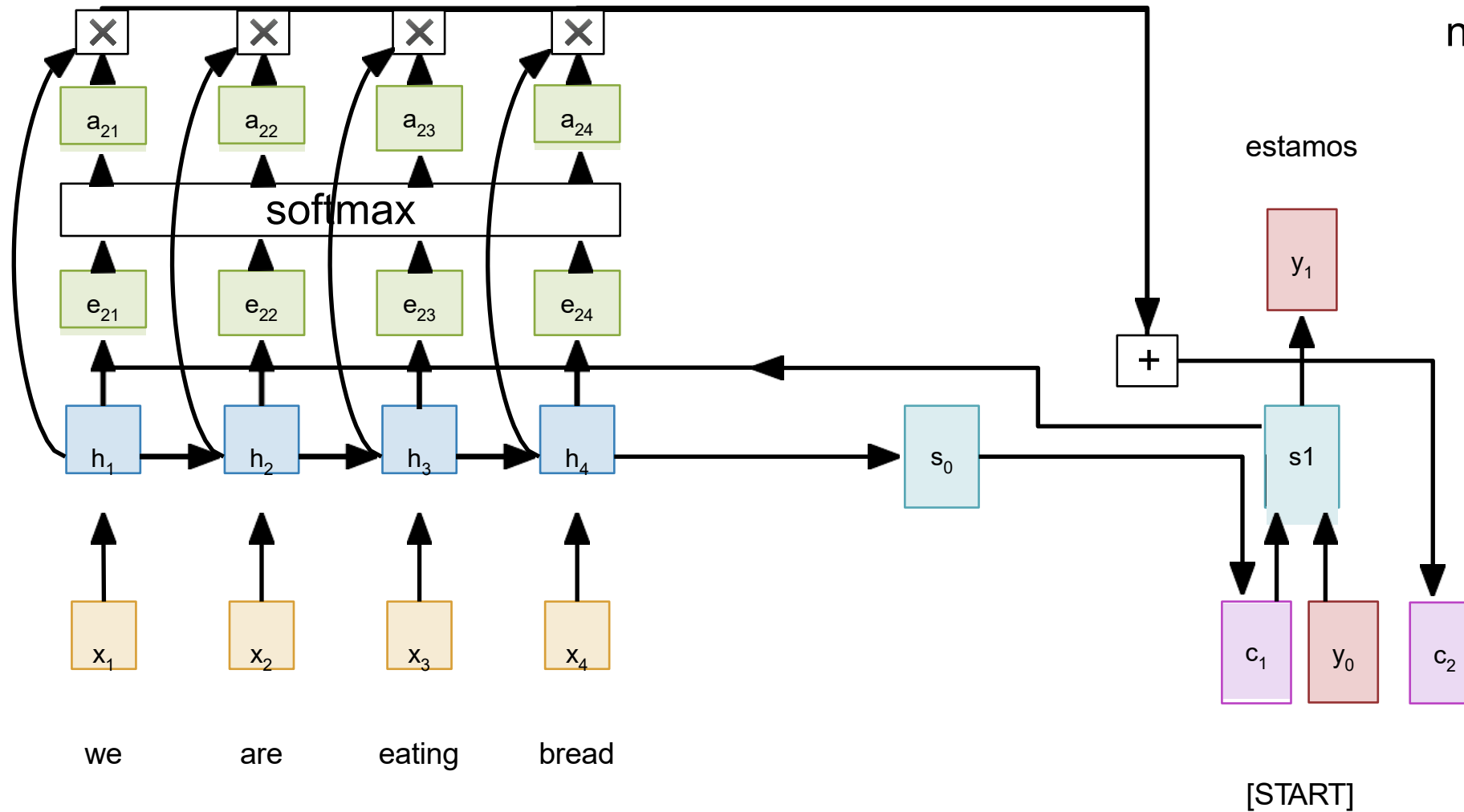
Sequence to Sequence with RNNs and Attention

Repeat: Use s_1 to compute new context vector c_2

Compute (scalar) **alignment scores**:

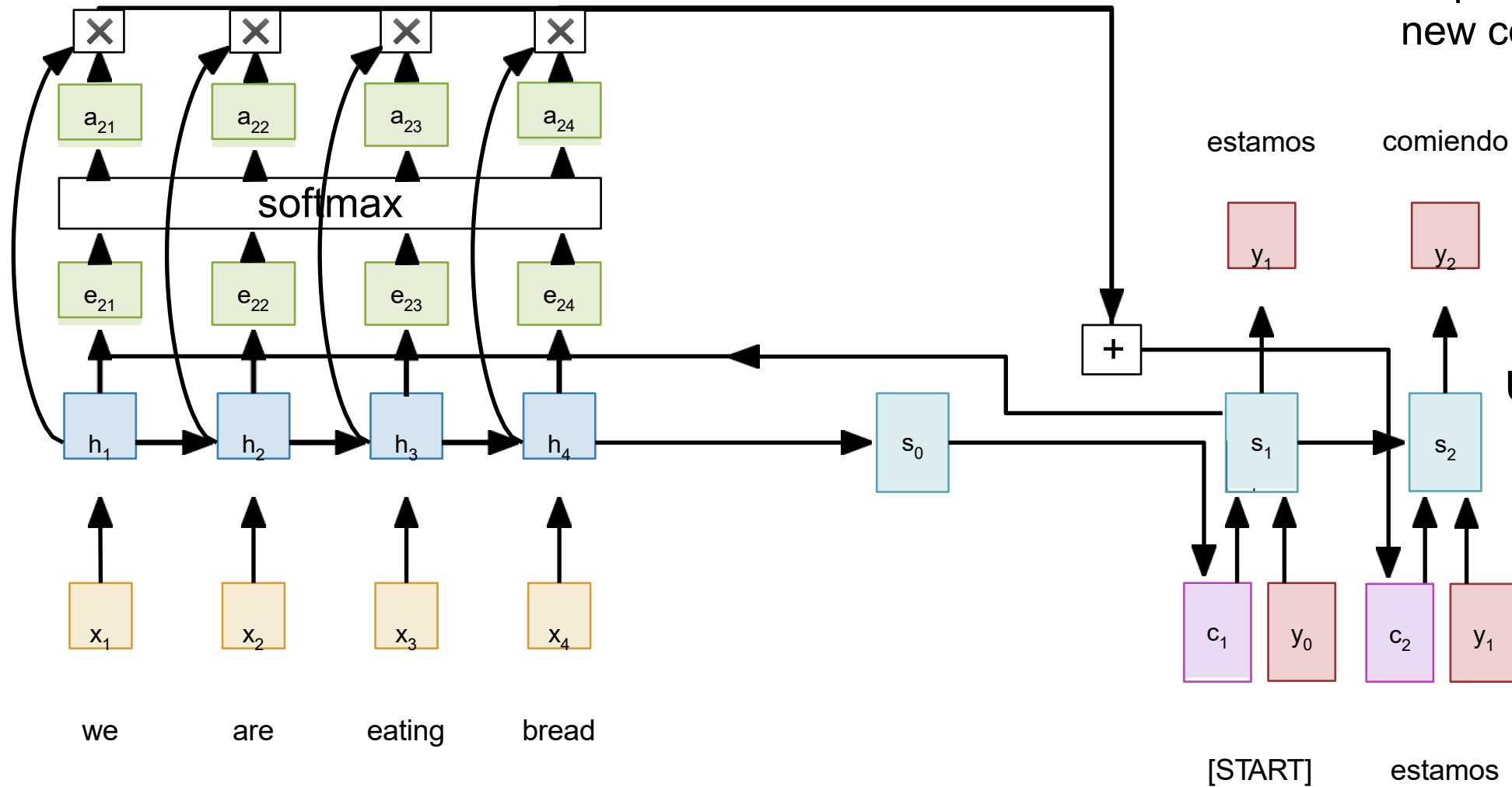
$$e_{t,i} = f_{\text{att}}(s_{t-1}, h_i)$$

(f_{att} is an MLP)



Sequence to Sequence with RNNs and Attention

Repeat: Use s_1 to compute new context vector c_2



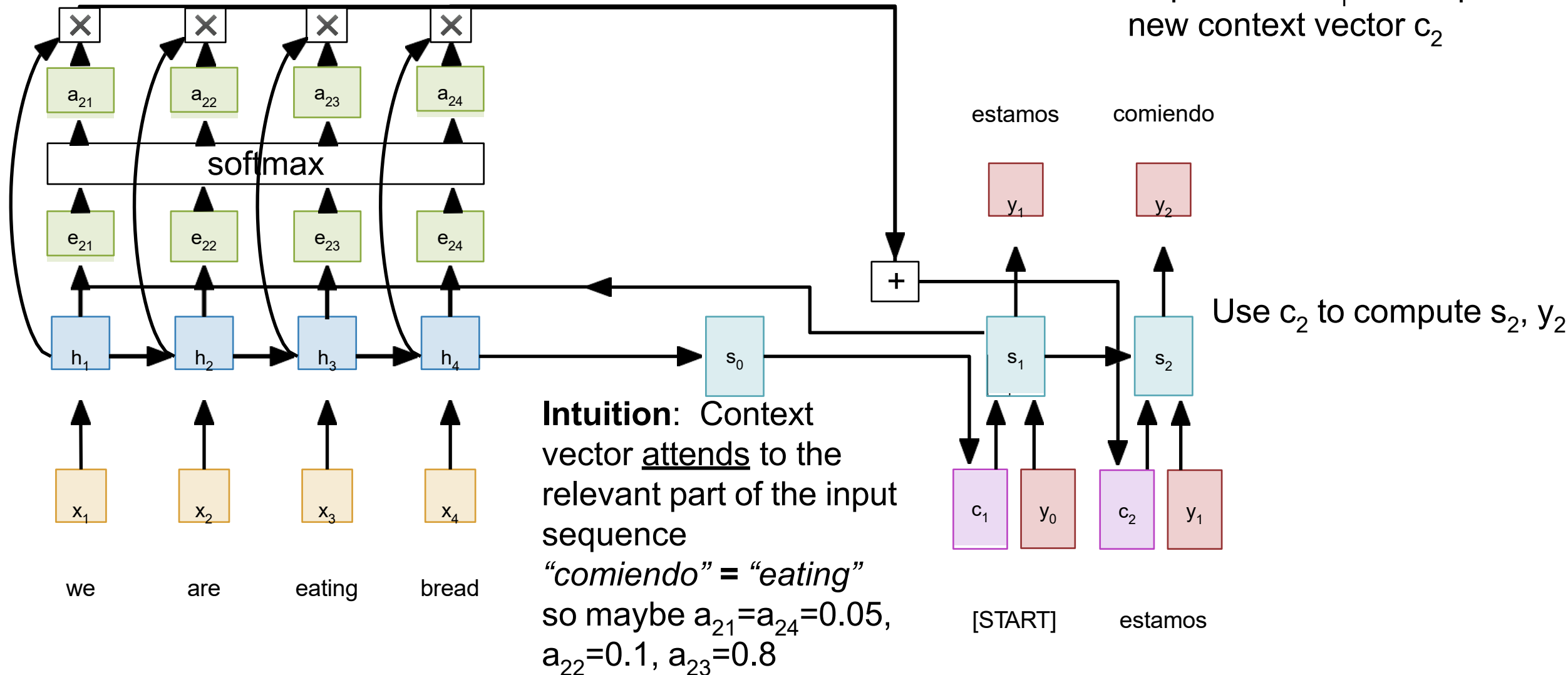
Use context vector in decoder:

$$s_t = g_U(y_{t-1}, s_{t-1}, c_t)$$

Use c_2 to compute s_2, y_2

Sequence to Sequence with RNNs and Attention

Repeat: Use s_1 to compute new context vector c_2



Use c_2 to compute s_2, y_2

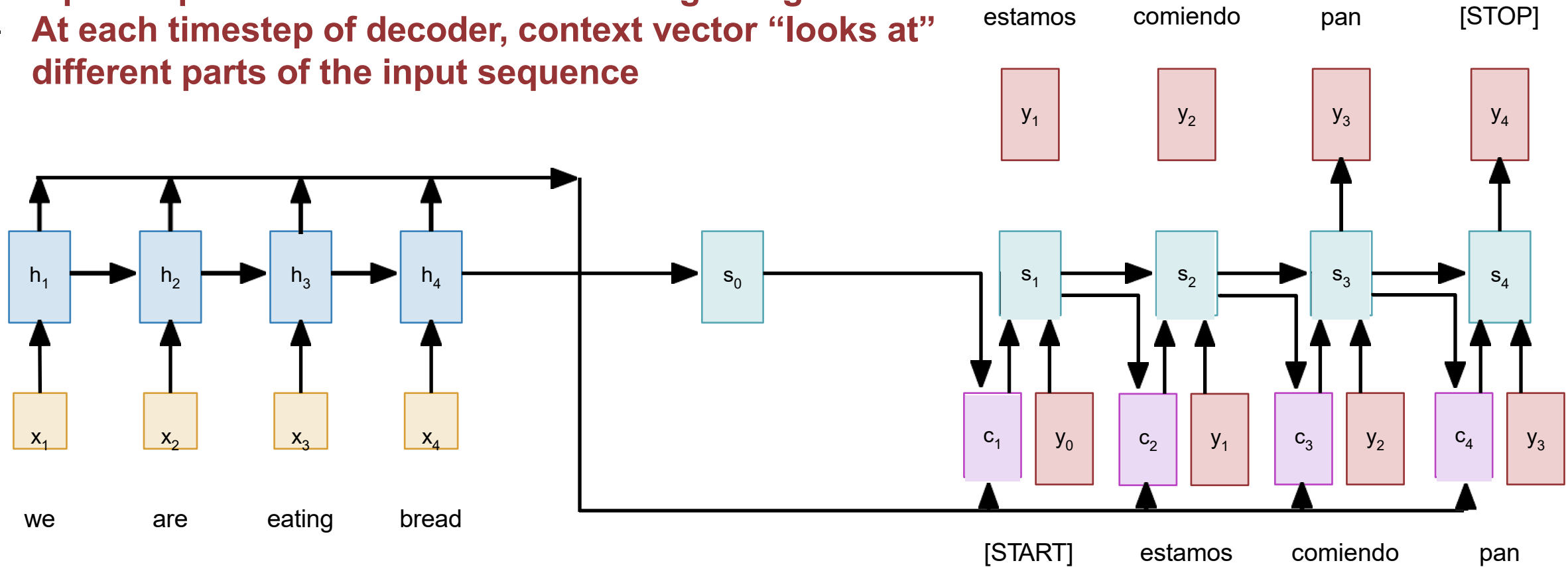
Intuition: Context vector attends to the relevant part of the input sequence

"comiendo" = "eating"
so maybe $a_{21}=a_{24}=0.05$,
 $a_{22}=0.1$, $a_{23}=0.8$

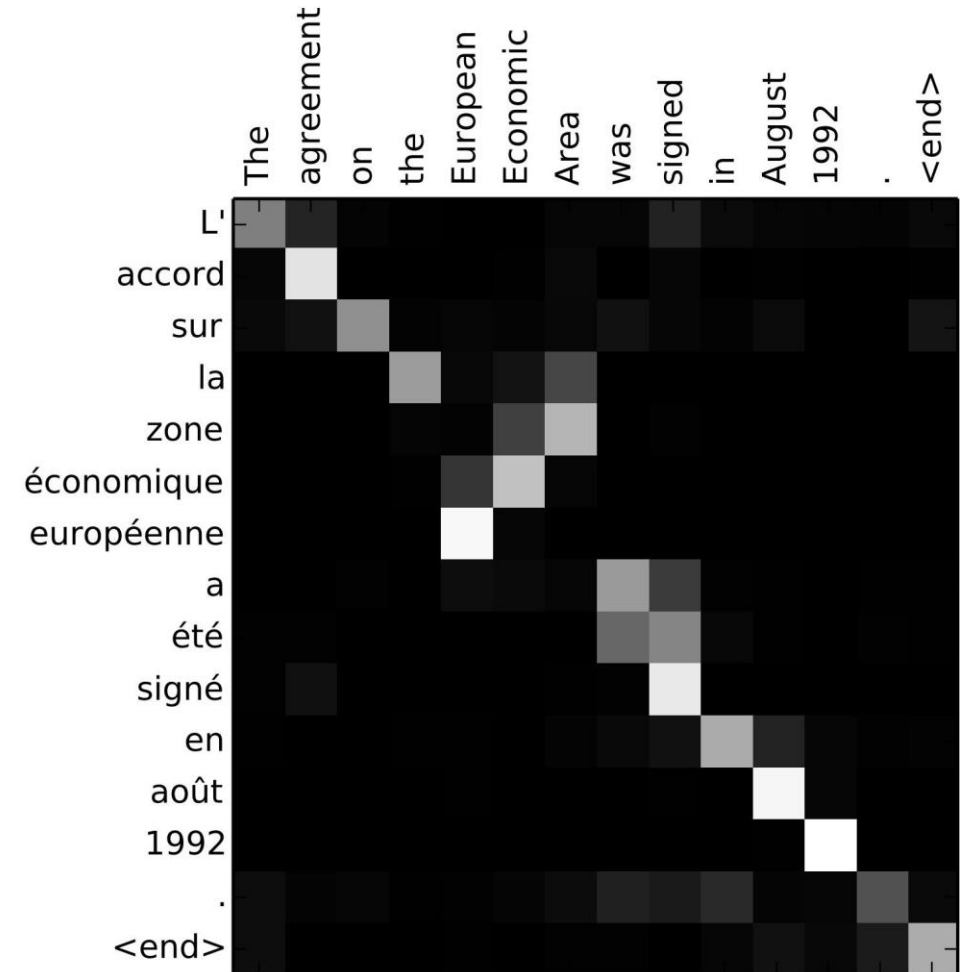
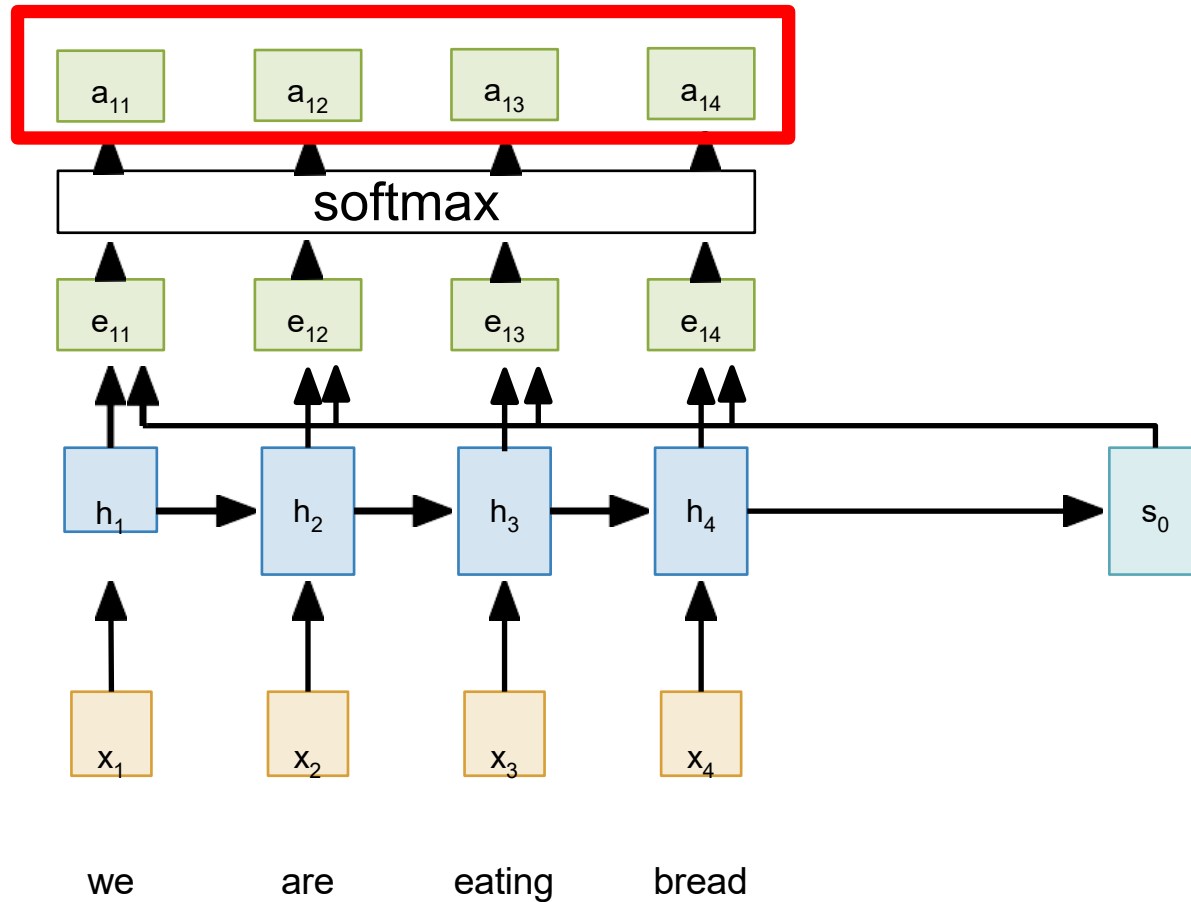
Sequence to Sequence with RNNs and Attention

Use a different context vector in each timestep of decoder

- Input sequence not bottlenecked through single vector
- At each timestep of decoder, context vector “looks at” different parts of the input sequence



Sequence to Sequence with RNNs and Attention



Bahdanau et al. "Neural machine translation by jointly learning to align and translate". ICLR 2015

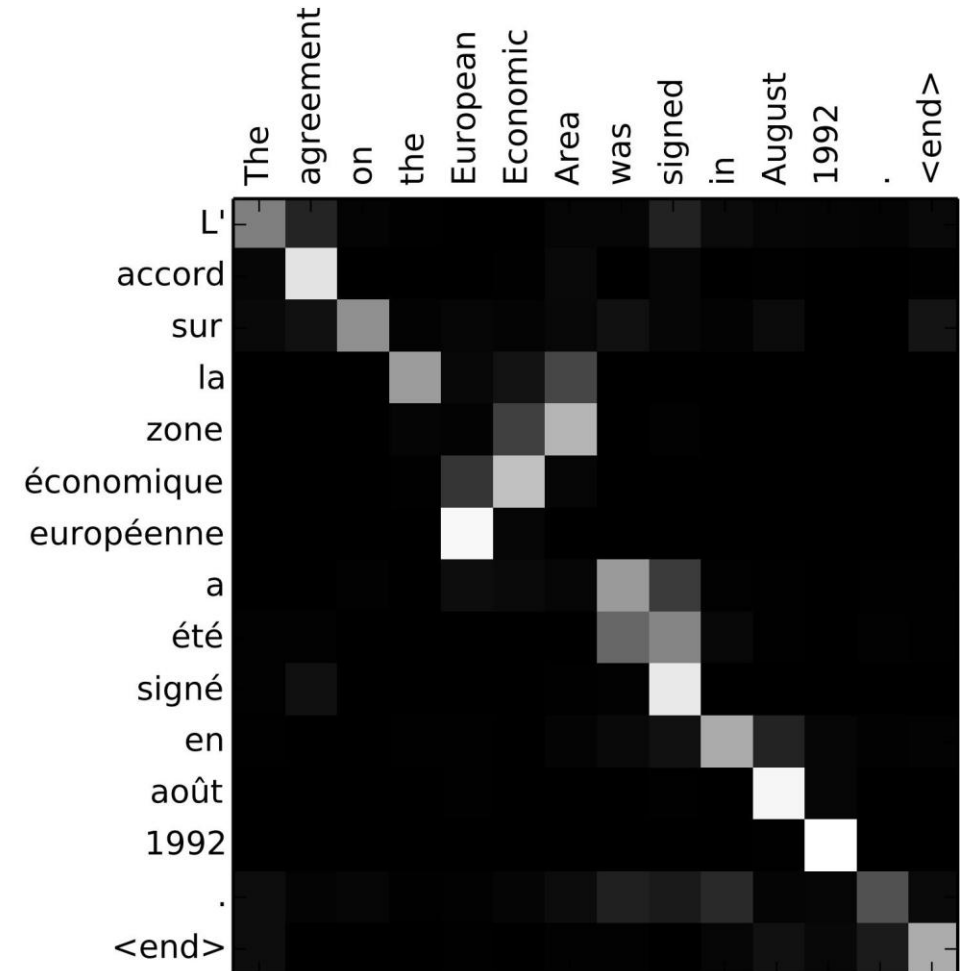
Sequence to Sequence with RNNs and Attention

Example: English to French translation

Input: “The agreement on the European Economic Area was signed in August 1992.”

Output: “L’accord sur la zone économique européenne a été signé en août 1992.”

Visualize attention weights $a_{t,i}$



Sequence to Sequence with RNNs and Attention

Example: English to French translation

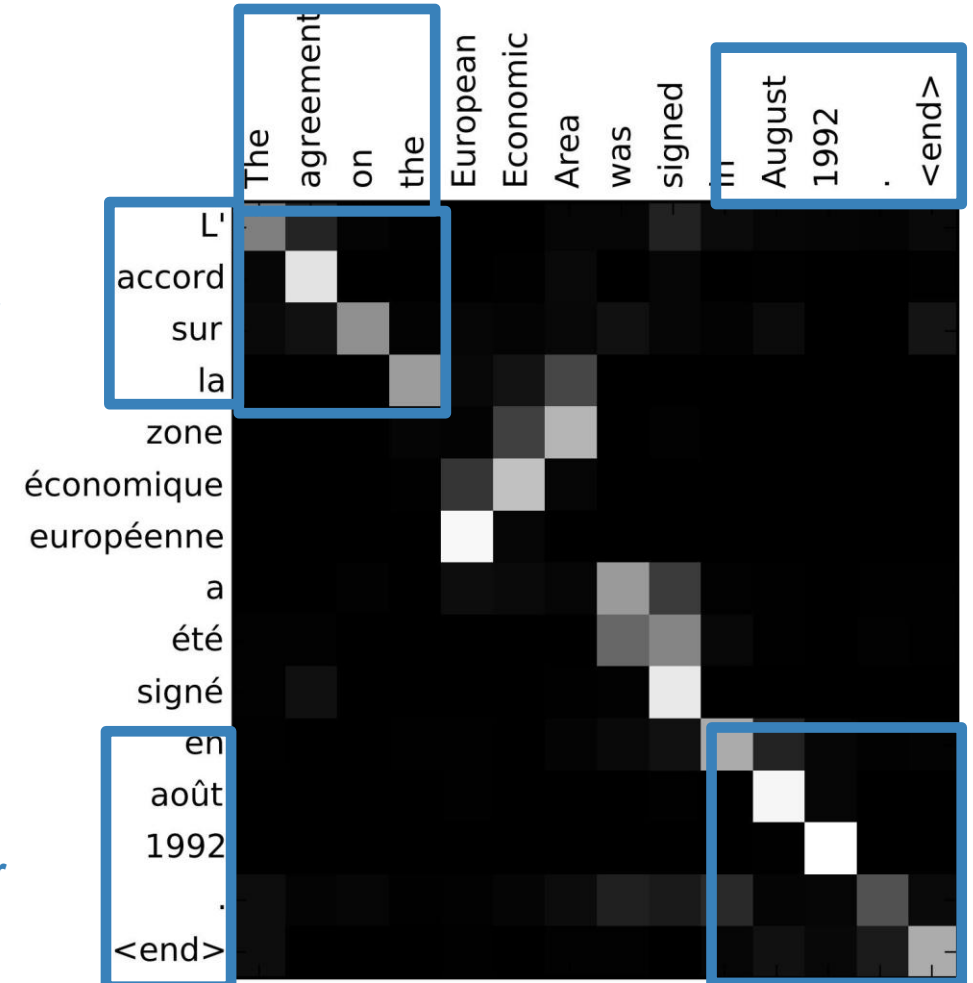
Input: “**The agreement on the** European Economic Area was signed **in August 1992.**”

Output: “**L'accord sur la** zone économique européenne a été signé **en août 1992.**”

Diagonal attention means words correspond in order

Diagonal attention means words correspond in order

Visualize attention weights $a_{t,i}$



Sequence to Sequence with RNNs and Attention

Example: English to French translation

Input: “**The agreement on the European Economic Area** was signed **in August 1992**.”

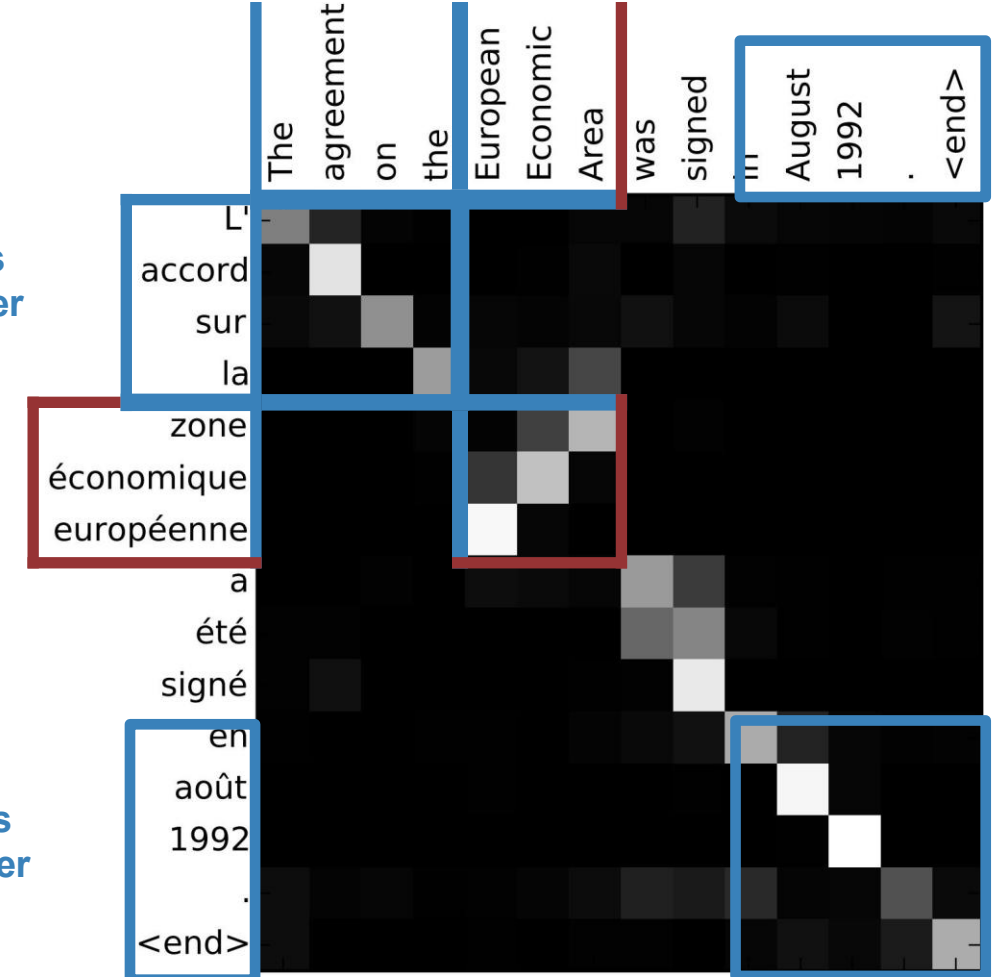
Output: “**L’accord sur la zone économique européenne** a été signé **en août 1992**.”

Visualize attention weights $a_{t,i}$

Diagonal attention means words correspond in order

Attention figures out different word orders

Diagonal attention means words correspond in order



Sequence to Sequence with RNNs and Attention

The decoder doesn't use the fact that h_i form an ordered sequence – it just treats them as an unordered set $\{h_i\}$

Can use similar architecture given any set of input hidden vectors $\{h_i\}$! (calculate attention weights, sum them)

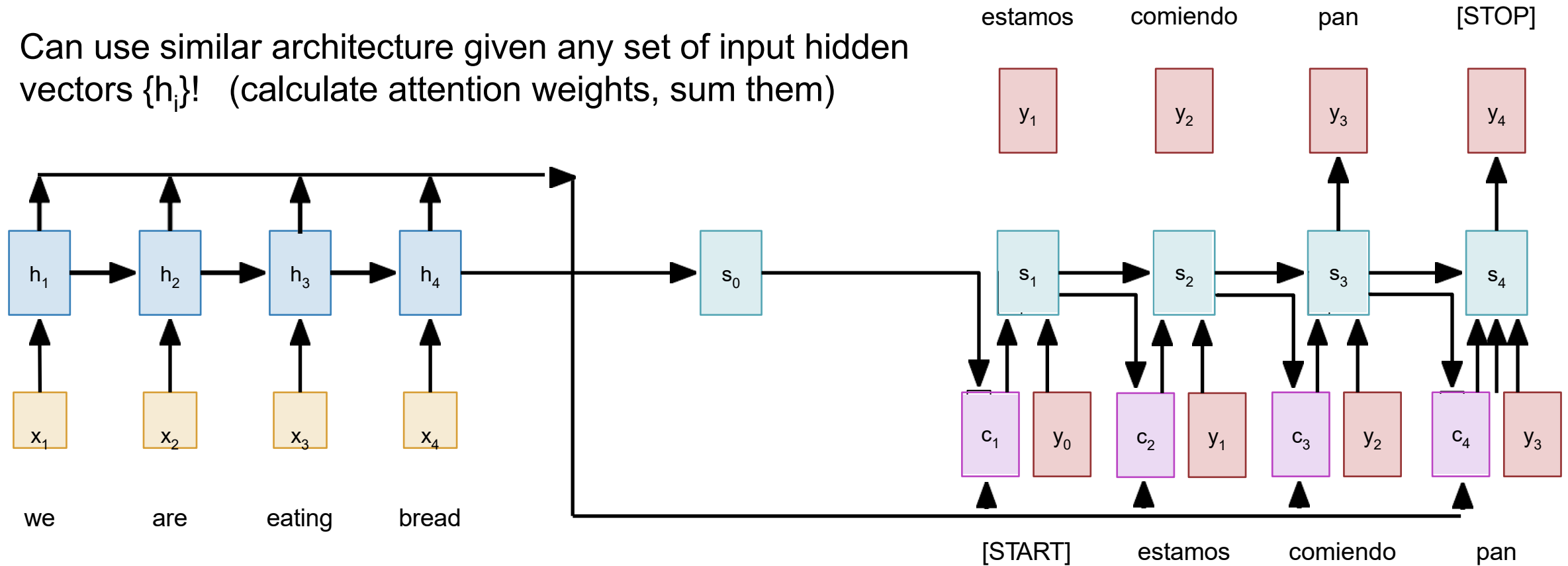
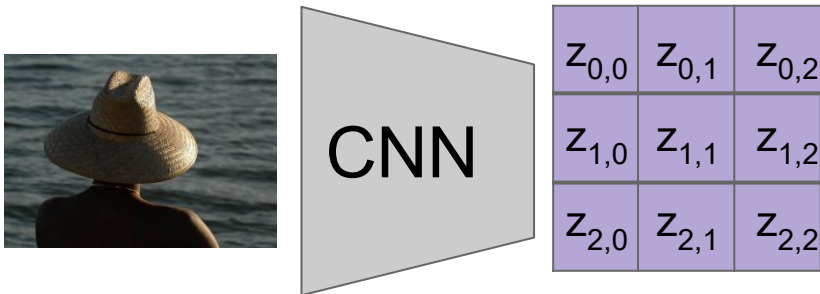


Image Captioning with RNNs and Attention

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

An example network for image captioning



Extract spatial
features from a
pretrained CNN

Features:
 $H \times W \times D$

Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention

Compute alignments
scores (scalars):

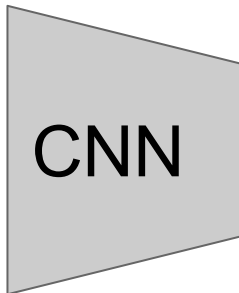
$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(\cdot)$ is an MLP

Alignment scores:

H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$



Extract spatial
features from a
pretrained CNN

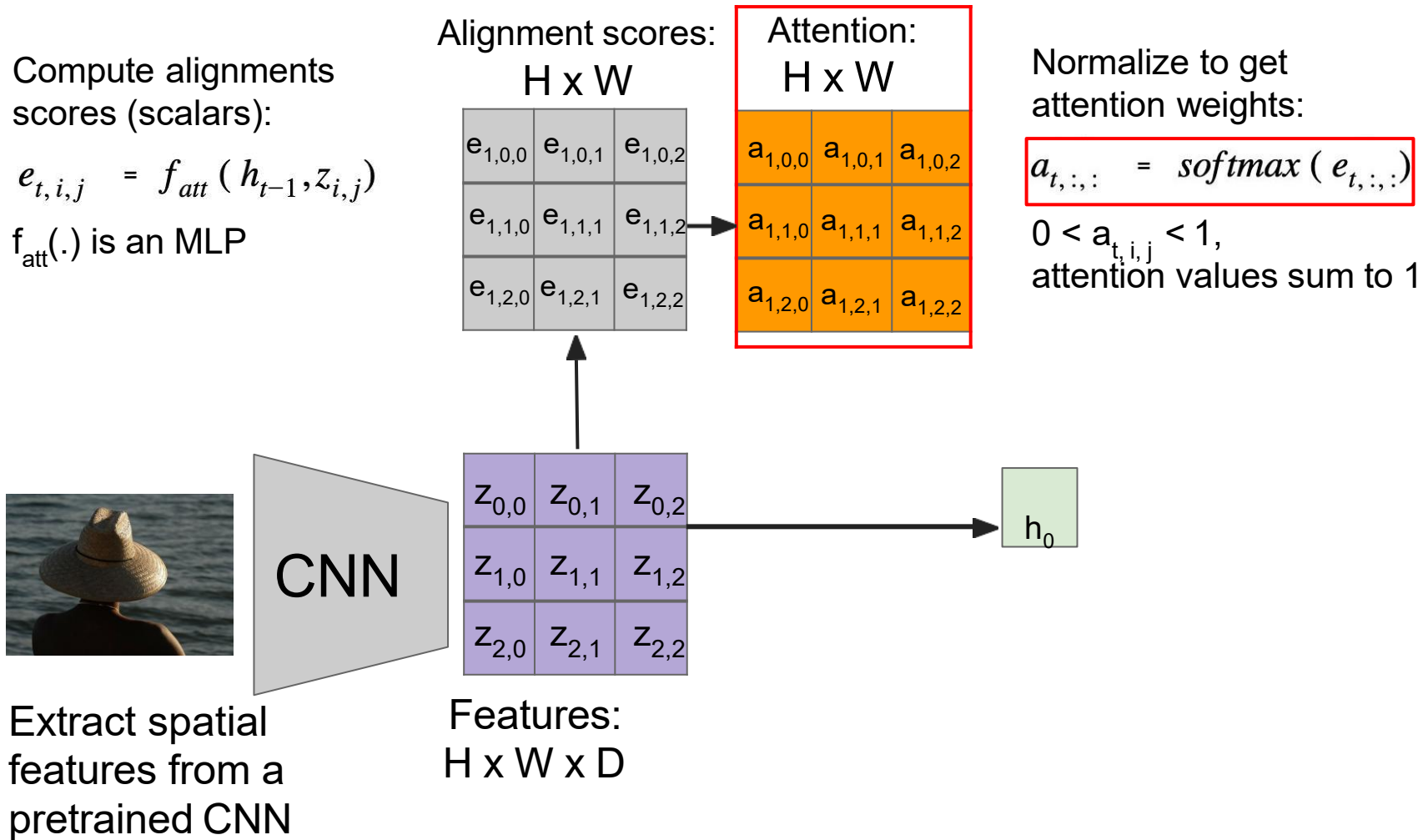
$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D



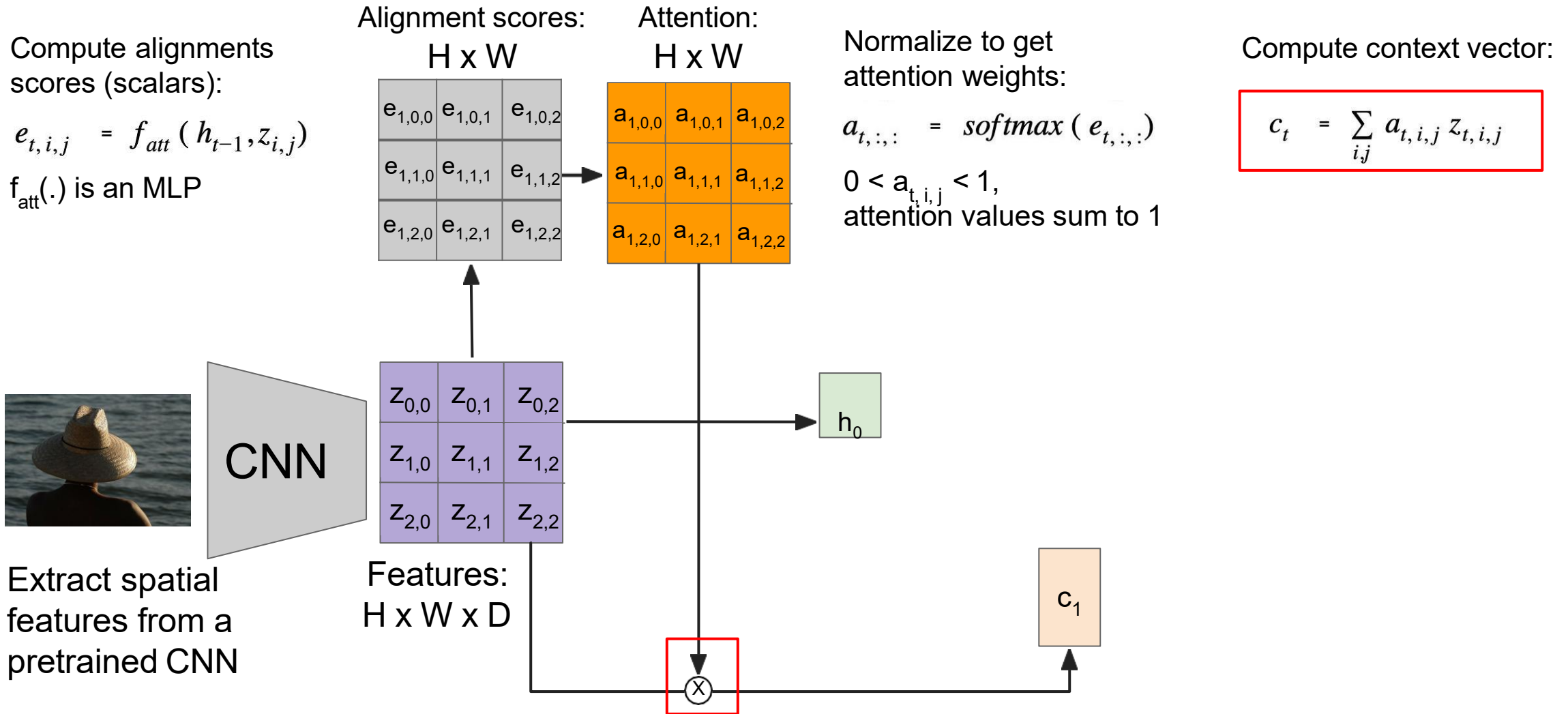
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention



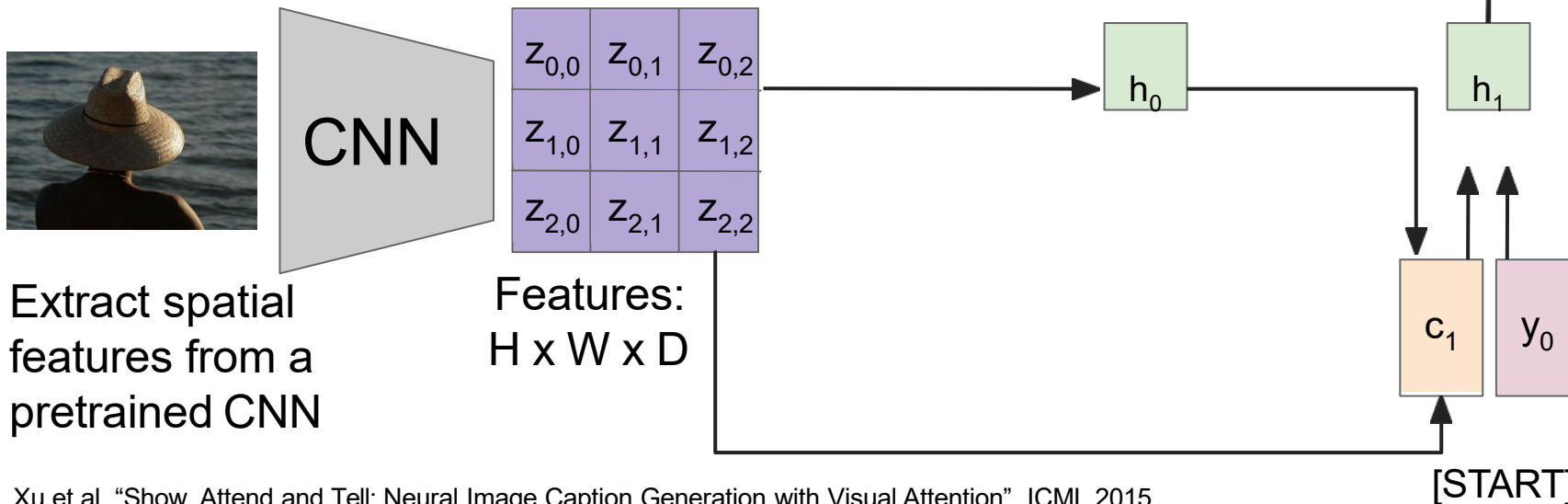
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

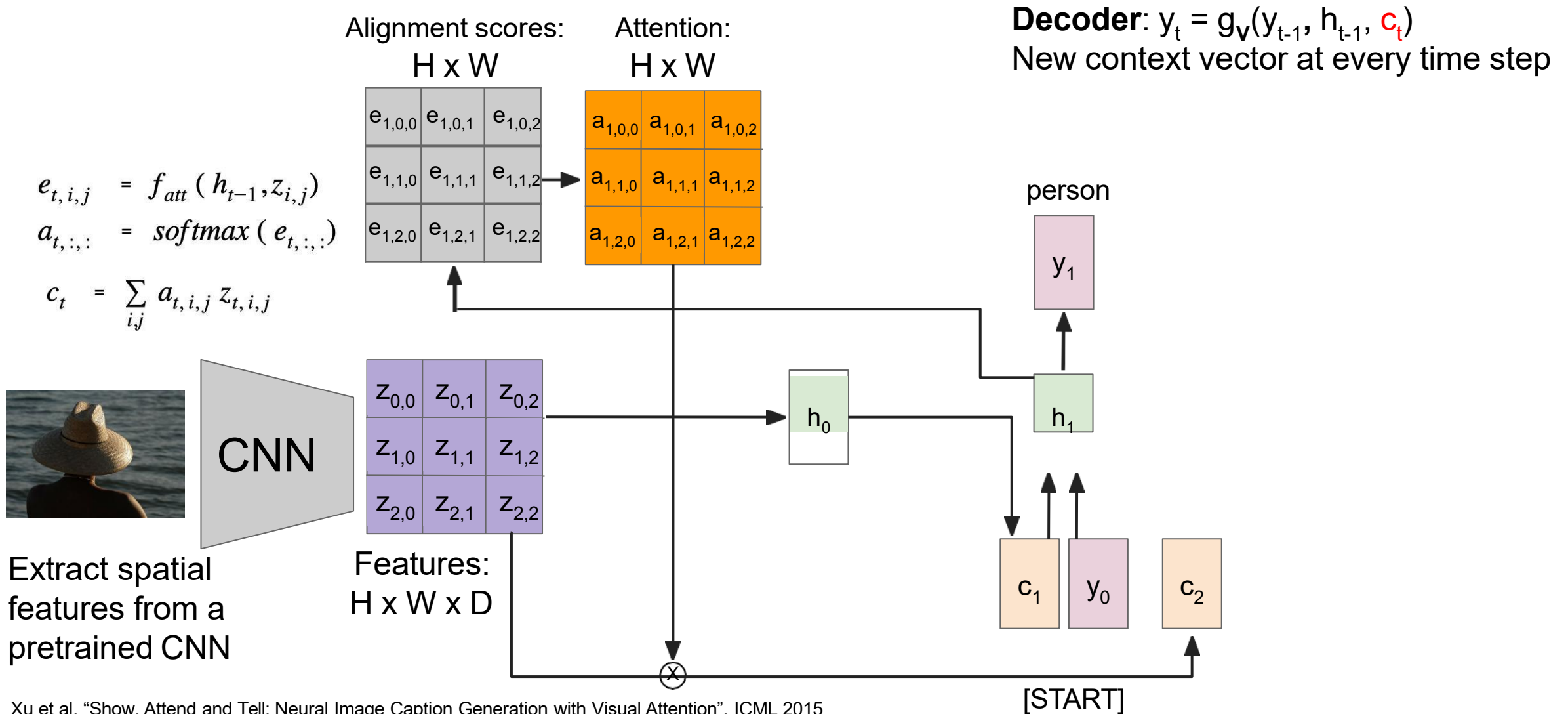
Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:) = softmax(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention



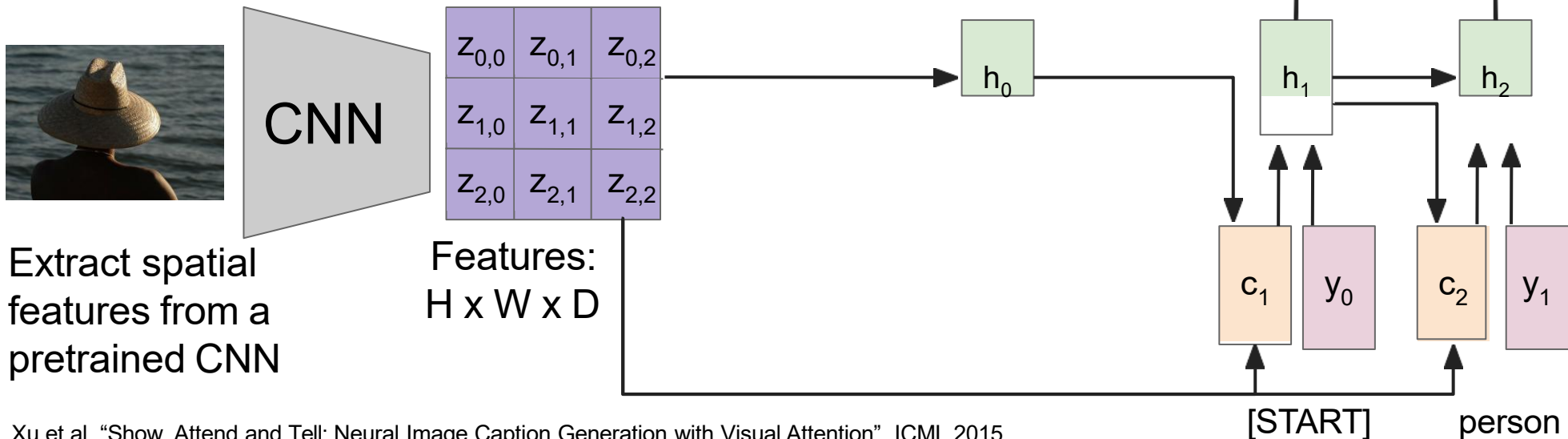
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



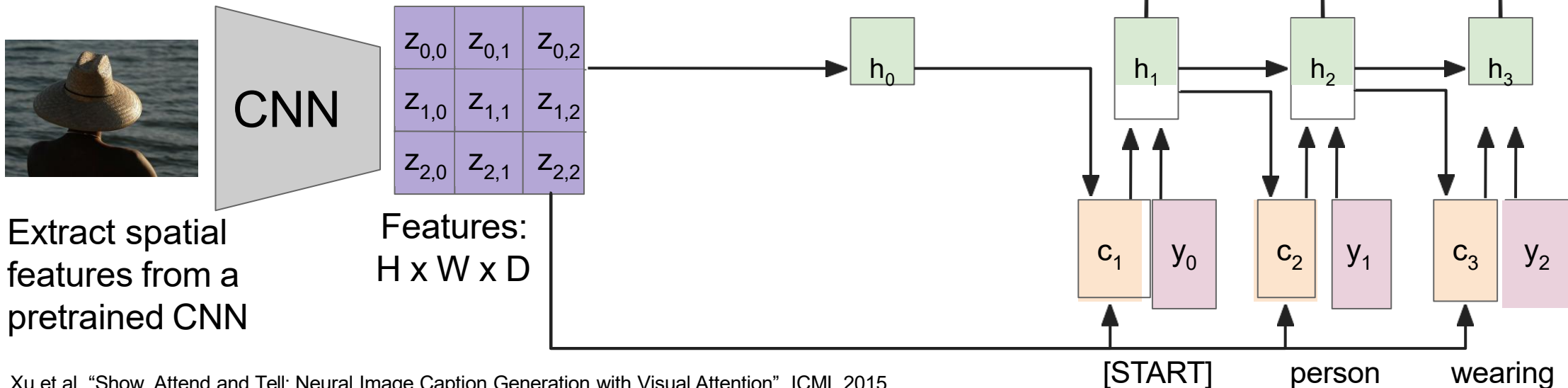
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



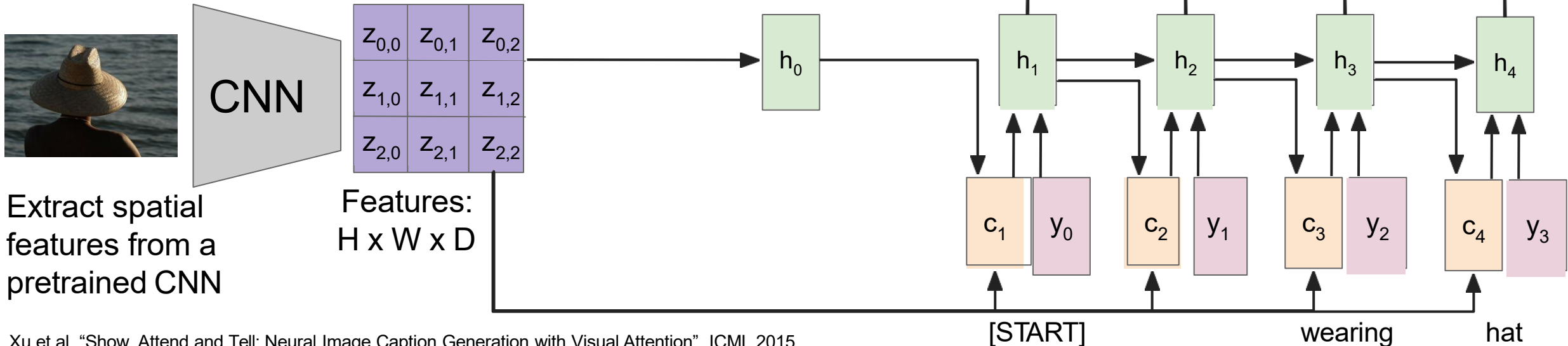
Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention

Each timestep of decoder uses a different context vector that looks at different parts of the input image

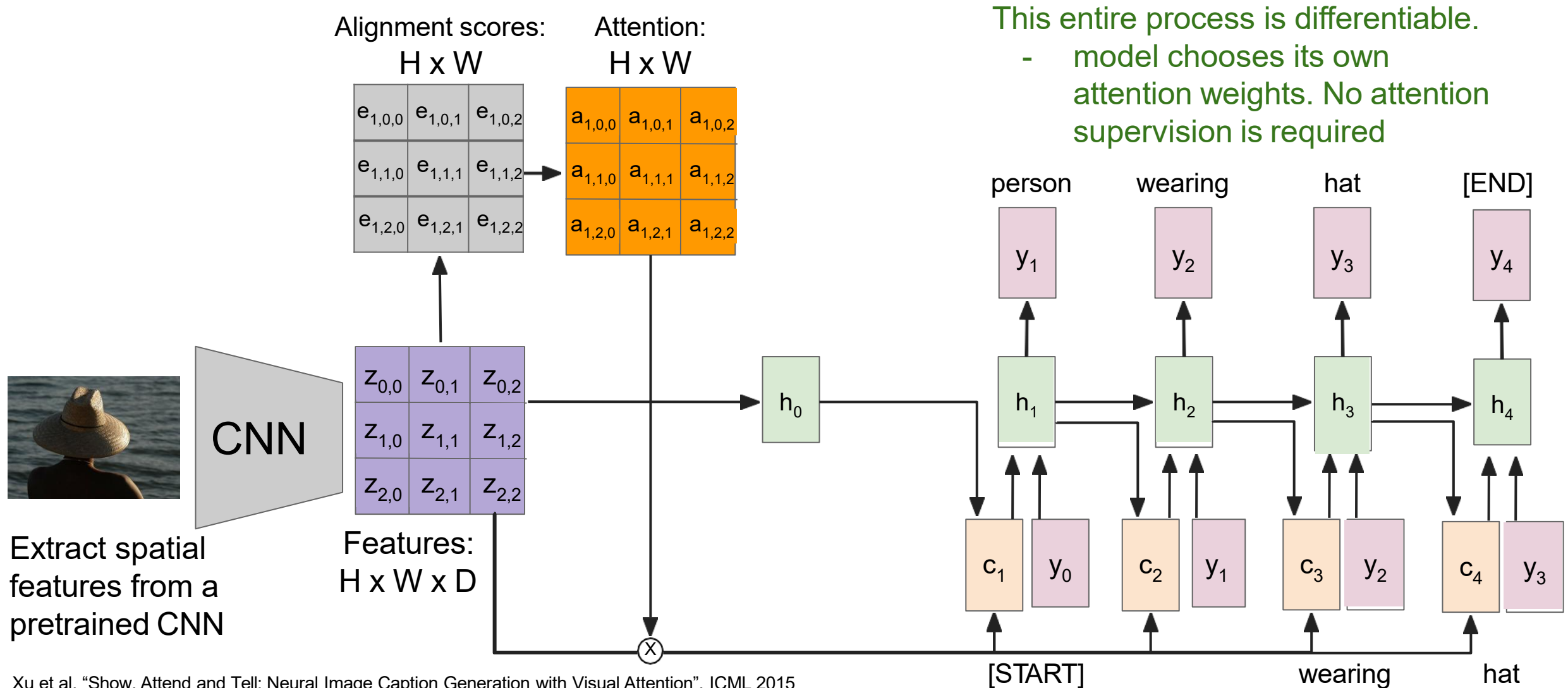
Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c_t)$
New context vector at every time step

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$
$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} z_{i,j}$$



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with RNNs and Attention



Xu et al, "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Image Captioning with Attention



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.

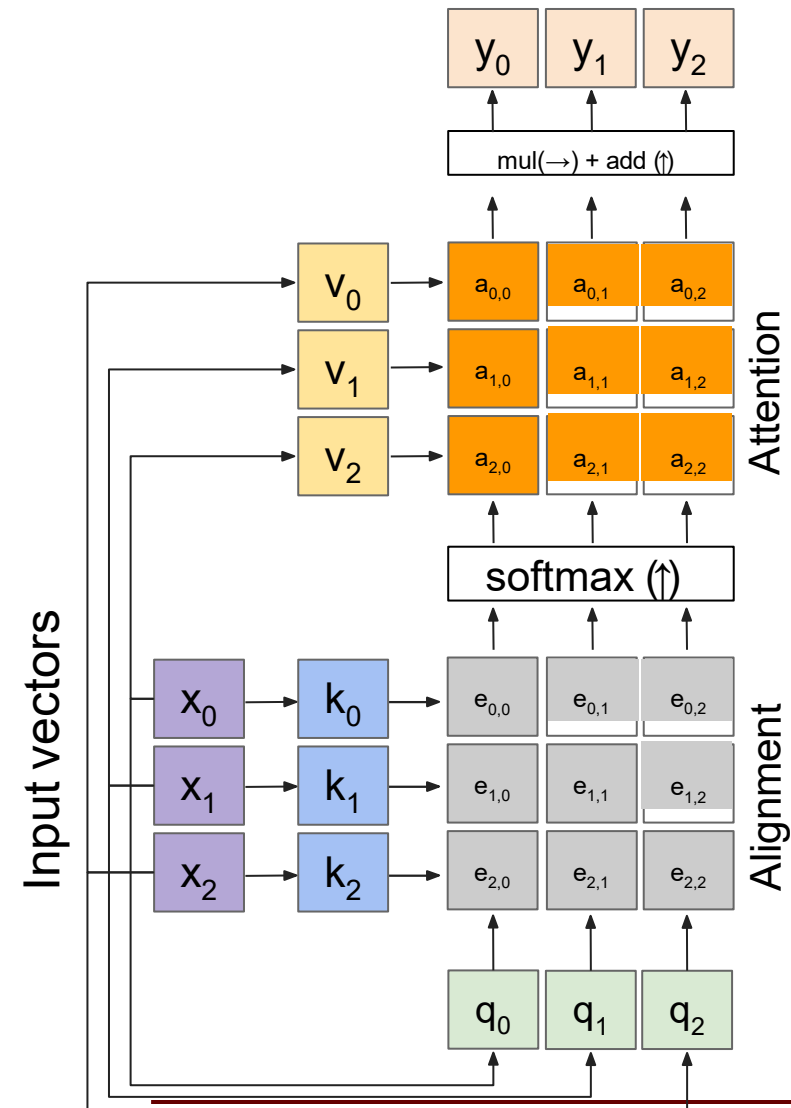


A giraffe standing in a forest with trees in the background.

Xu et al, "Show, Attend, and Tell: Neural Image Caption Generation with Visual Attention", ICML 2015

Figure copyright Kelvin Xu, Jimmy Lei Ba, Jamie Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard S. Zemel, and Yoshua Bengio, 2015. Reproduced with permission.

Self attention layer

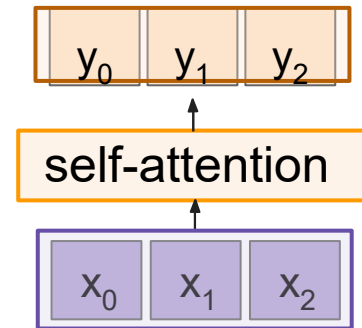
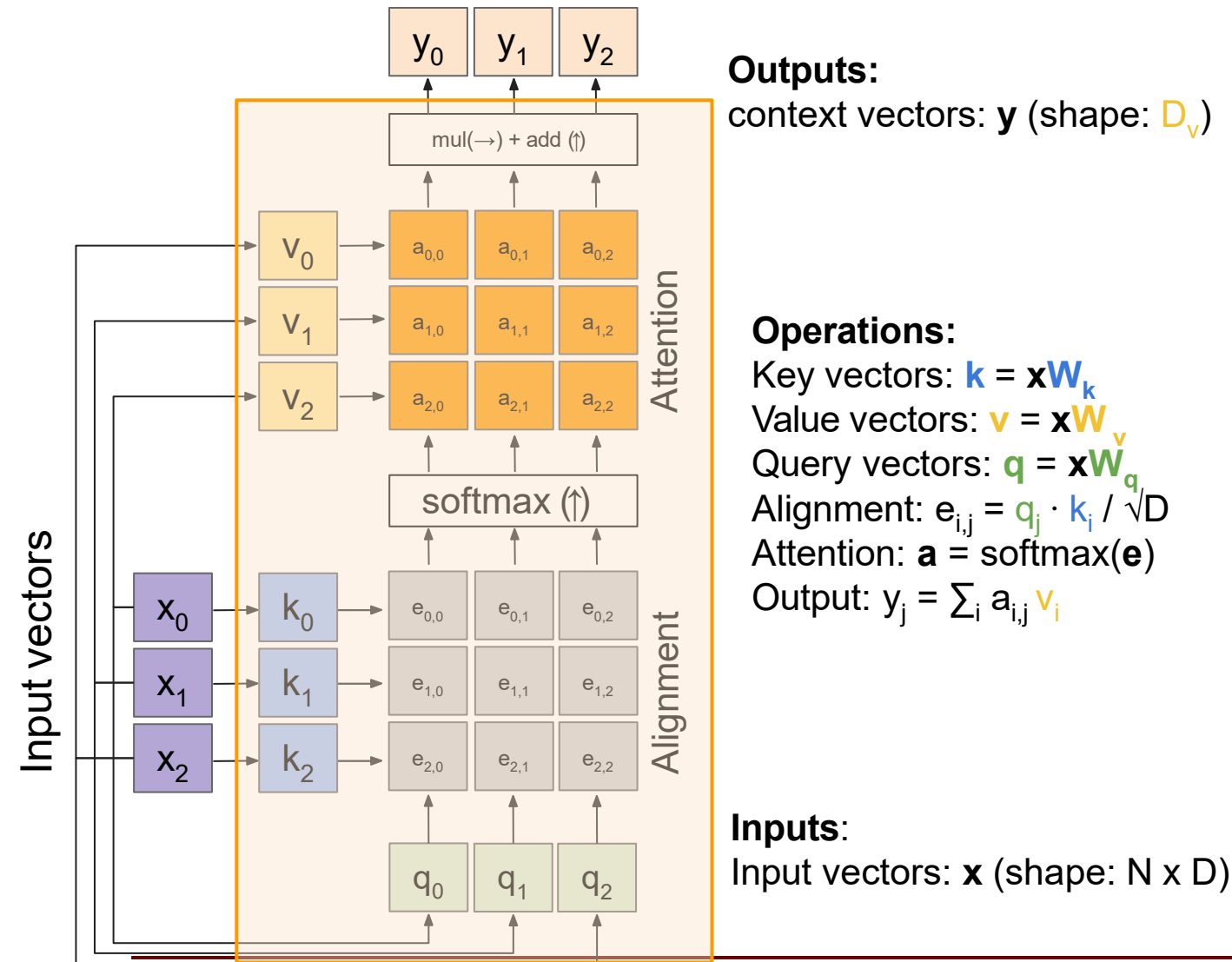


Outputs:
context vectors: $\mathbf{y}$ (shape: D_v)

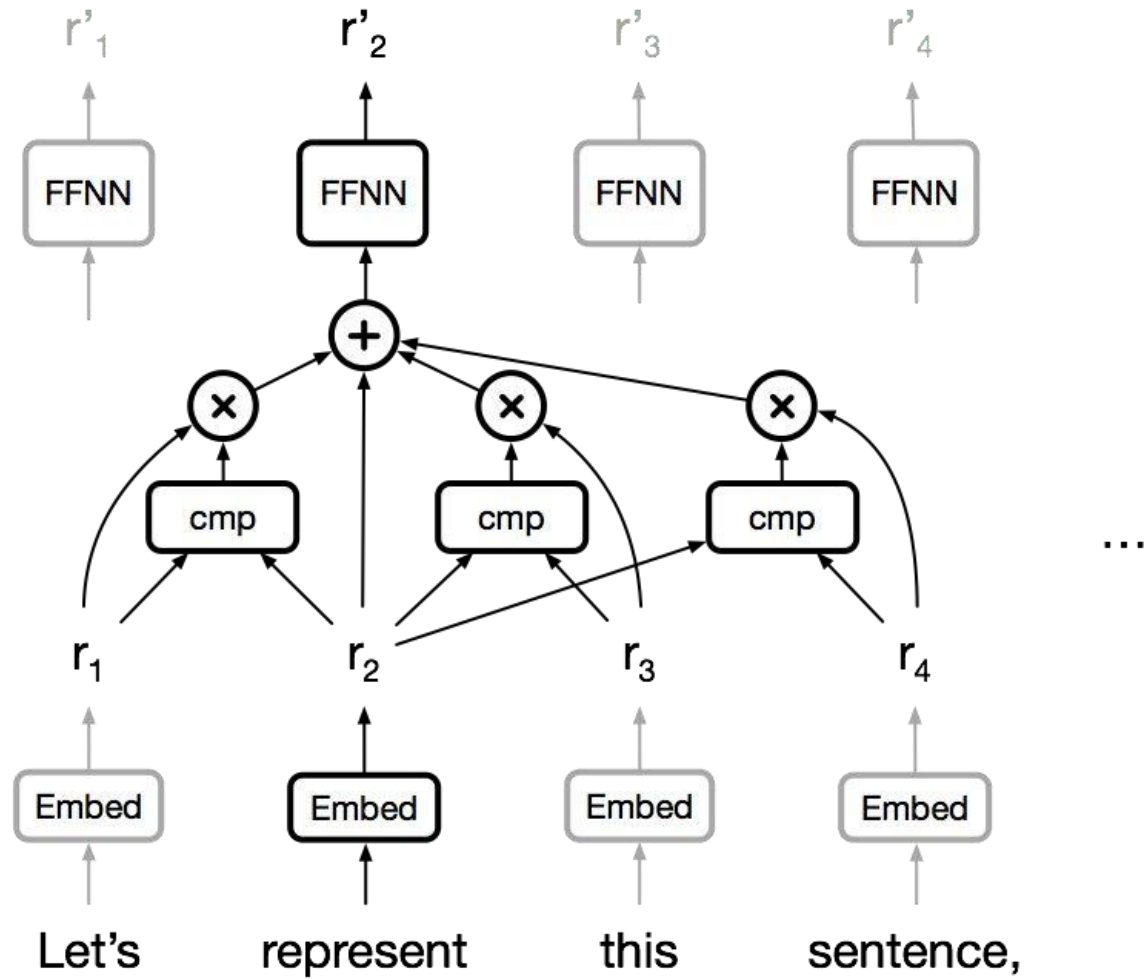
Operations:
Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$
Query vectors: $\mathbf{q} = \mathbf{x}\mathbf{W}_q$
Alignment: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D}$
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Self attention layer - attends over sets of inputs

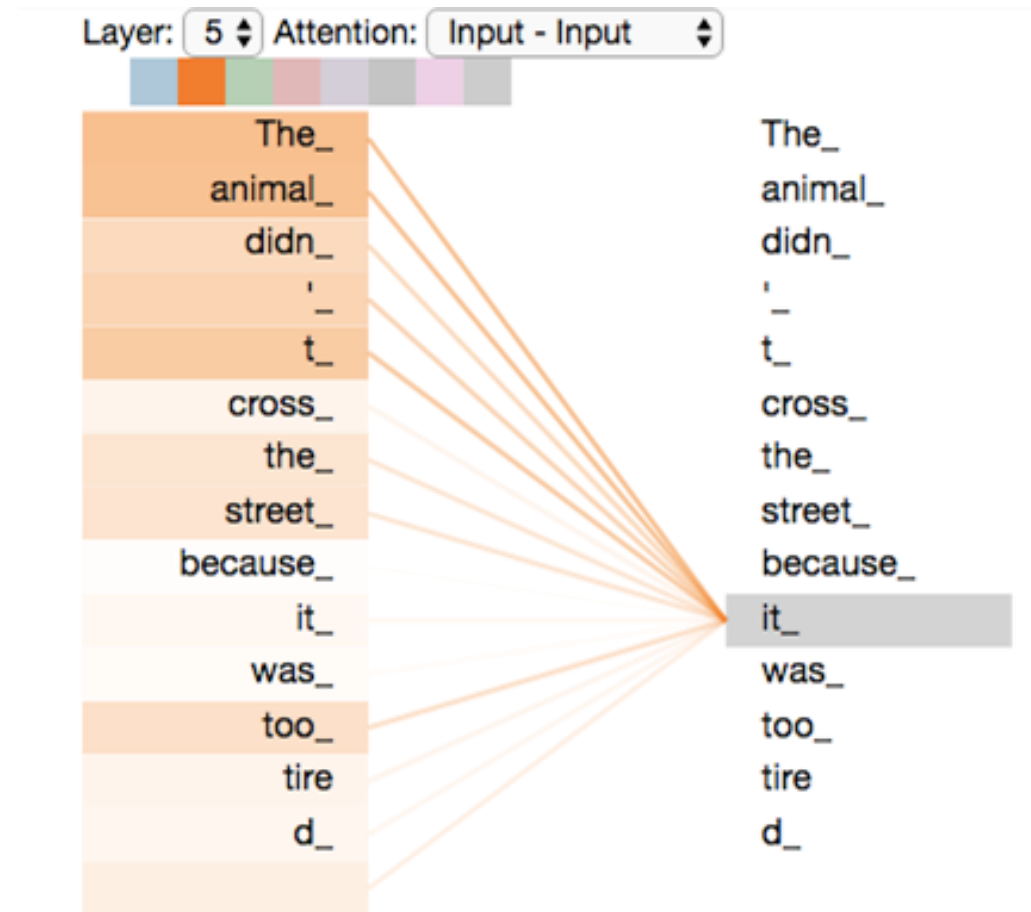


Self-Attention

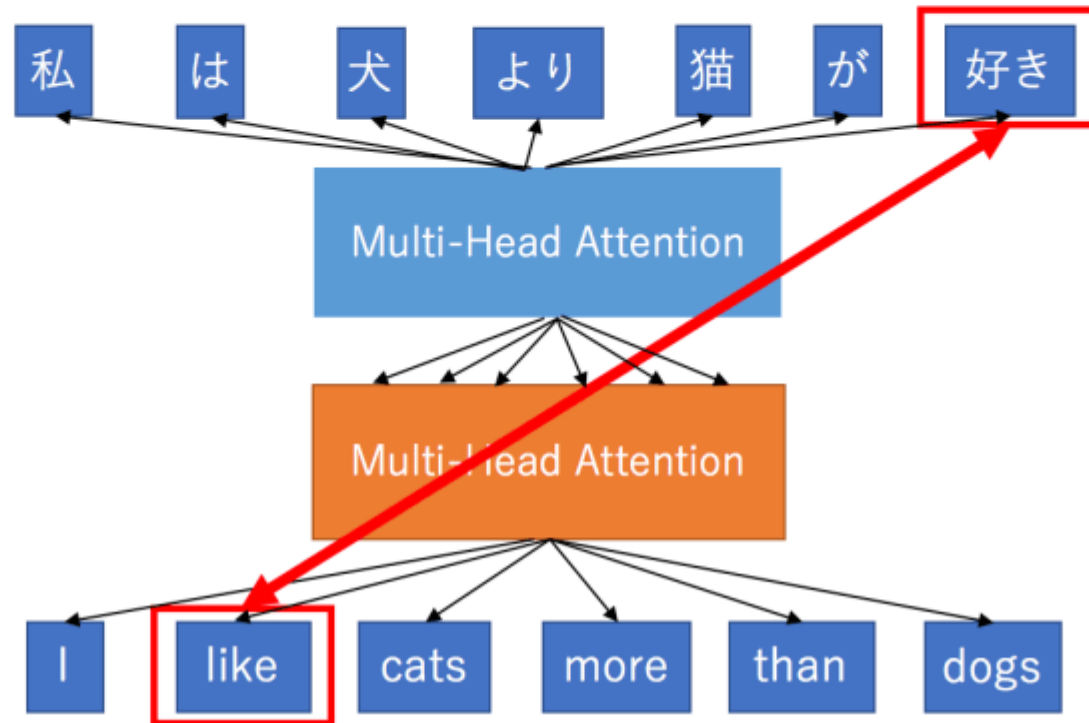
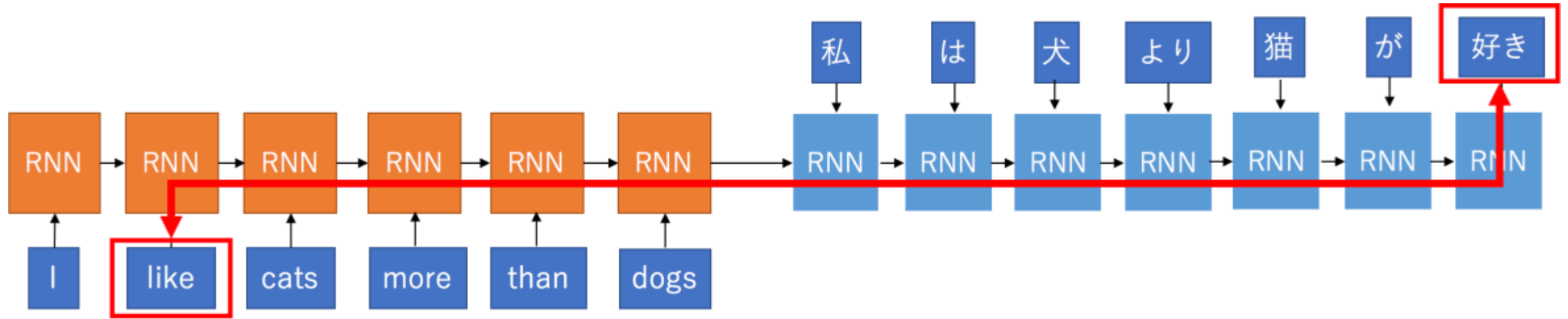


Self-Attention

- Self-attention: weight according to relevance



Maximum Path Length in RNN and Self-attention



Acknowledge to Figure from
<http://mlexplained.com/2017/12/29/attention-is-all-you-need-explained/>

Self-Attention

Constant 'path length' between any two positions

Gating/multiplicative interactions

Trivial to parallelize (per layer)

Why self-attention

Key aspects

- Complexity per layer
- Amount of computation which can be parallelized
- Path length between long-range dependencies in the network

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n is hundreds, $d=512$ or 1024 , k conv kernel size, r neighborhood reduced attention

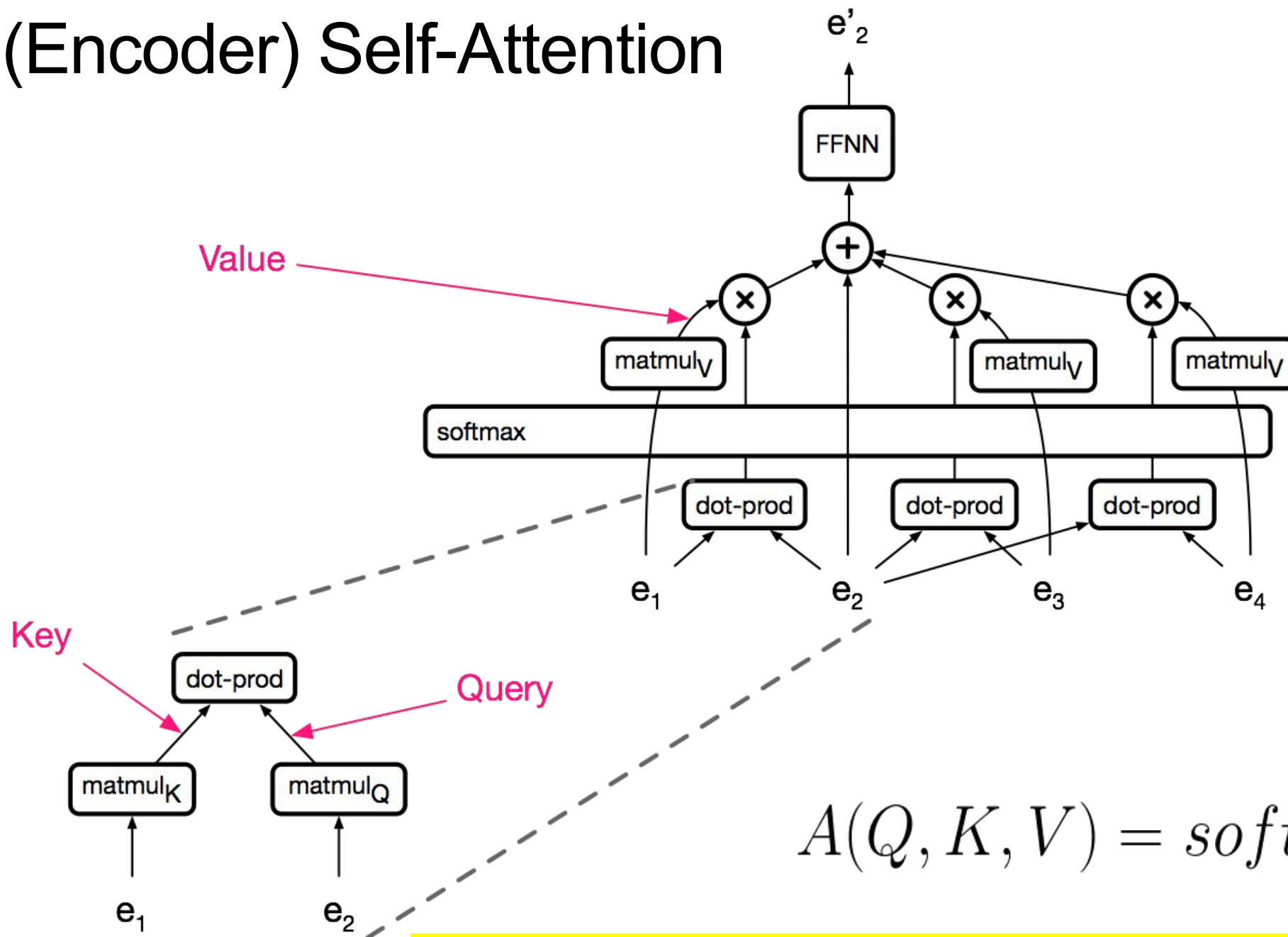
Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- Transformer Networks
- BERT

Positional encoding and masked self-attention

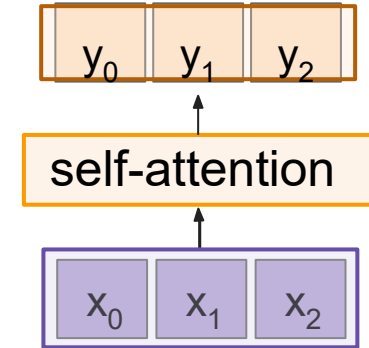
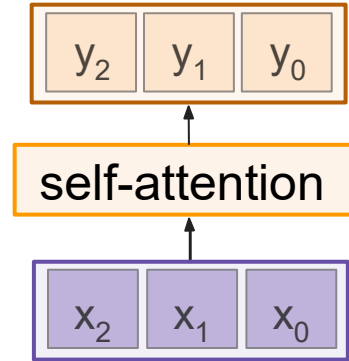
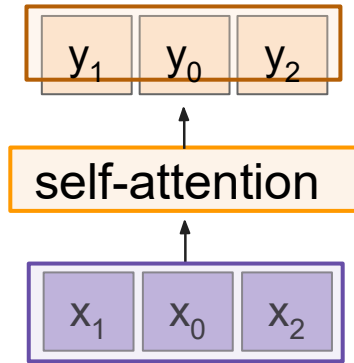


(Encoder) Self-Attention



Note: for each query, this is permutation invariant to word order

Self attention layer - attends over sets of inputs

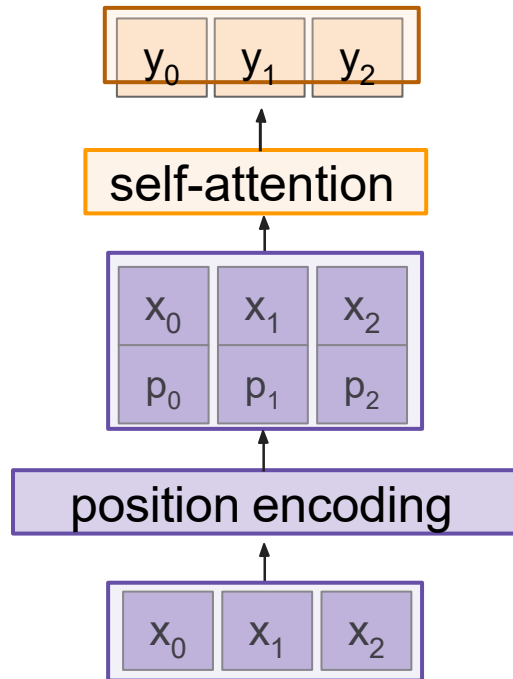


Permutation equivariant

Self-attention layer doesn't care about the orders of the inputs!

Problem: How can we encode ordered sequences like language or spatially ordered image features?

Positional encoding



Concatenate/add special positional encoding p_j to each input vector x_j

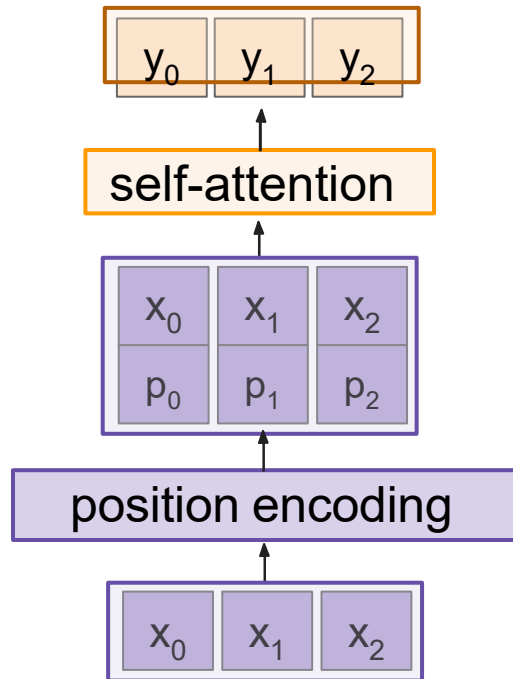
We use a function $pos: \mathbb{N} \rightarrow \mathbb{R}^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = pos(j)$

Desiderata of $pos(.)$:

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

Positional encoding



Concatenate/add special positional encoding p_j to each input vector x_j

We use a function $pos: \mathbb{N} \rightarrow \mathbb{R}^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = pos(j)$

Options for $pos(\cdot)$

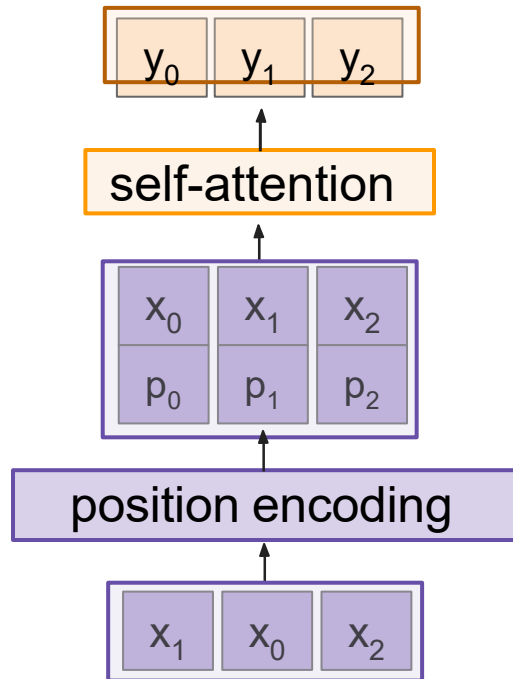
1. Learn a lookup table:
 - Learn parameters to use for $pos(t)$ for $t \in [0, T)$
 - Lookup table contains $T \times d$ parameters.

Desiderata of $pos(\cdot)$:

1. It should output a **unique** encoding for each time-step (word's position in a sentence)
2. **Distance** between any two time-steps should be consistent across sentences with different lengths.
3. Our model should generalize to **longer** sentences without any efforts. Its values should be bounded.
4. It must be **deterministic**.

Vaswani et al, "Attention is all you need", NeurIPS 2017

Positional encoding



Concatenate/add special positional encoding p_j to each input vector x_j

We use a function $pos: \mathbb{N} \rightarrow \mathbb{R}^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = pos(j)$

Options for $pos(\cdot)$

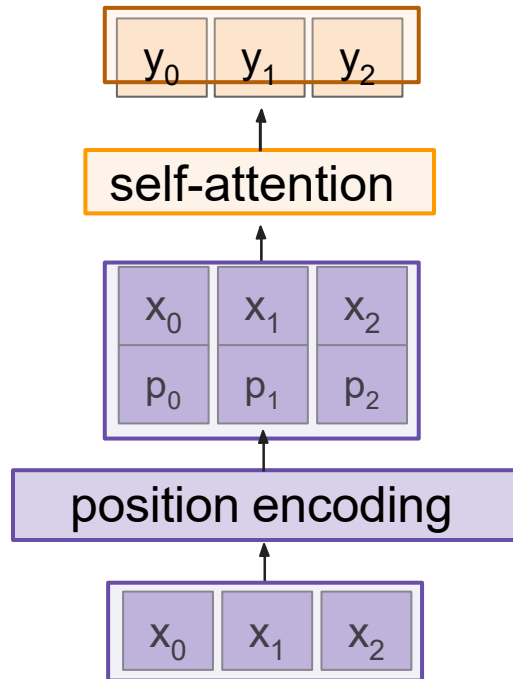
1. Learn a lookup table:
 - Learn parameters to use for $pos(t)$ for $t \in [0, T)$
 - Lookup table contains $T \times d$ parameters.
2. Design a fixed function with the desiderata

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

$$\text{where } \omega_k = \frac{1}{10000^{2k/d}}$$

Vaswani et al, "Attention is all you need", NeurIPS 2017

Positional encoding



Concatenate/add special positional encoding p_j to each input vector x_j

We use a function $pos: \mathbb{N} \rightarrow \mathbb{R}^d$ to process the position j of the vector into a d -dimensional vector

So, $p_j = pos(j)$

Options for $pos(\cdot)$

1. Learn a lookup table:
 - Learn parameters to use for $pos(t)$ for $t \in [0, T)$
 - Lookup table contains $T \times d$ parameters.
2. Design a fixed function with the desiderata

$$p(t) = \begin{bmatrix} \sin(\omega_1 \cdot t) \\ \cos(\omega_1 \cdot t) \\ \\ \sin(\omega_2 \cdot t) \\ \cos(\omega_2 \cdot t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} \cdot t) \\ \cos(\omega_{d/2} \cdot t) \end{bmatrix}_d$$

Intuition:

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

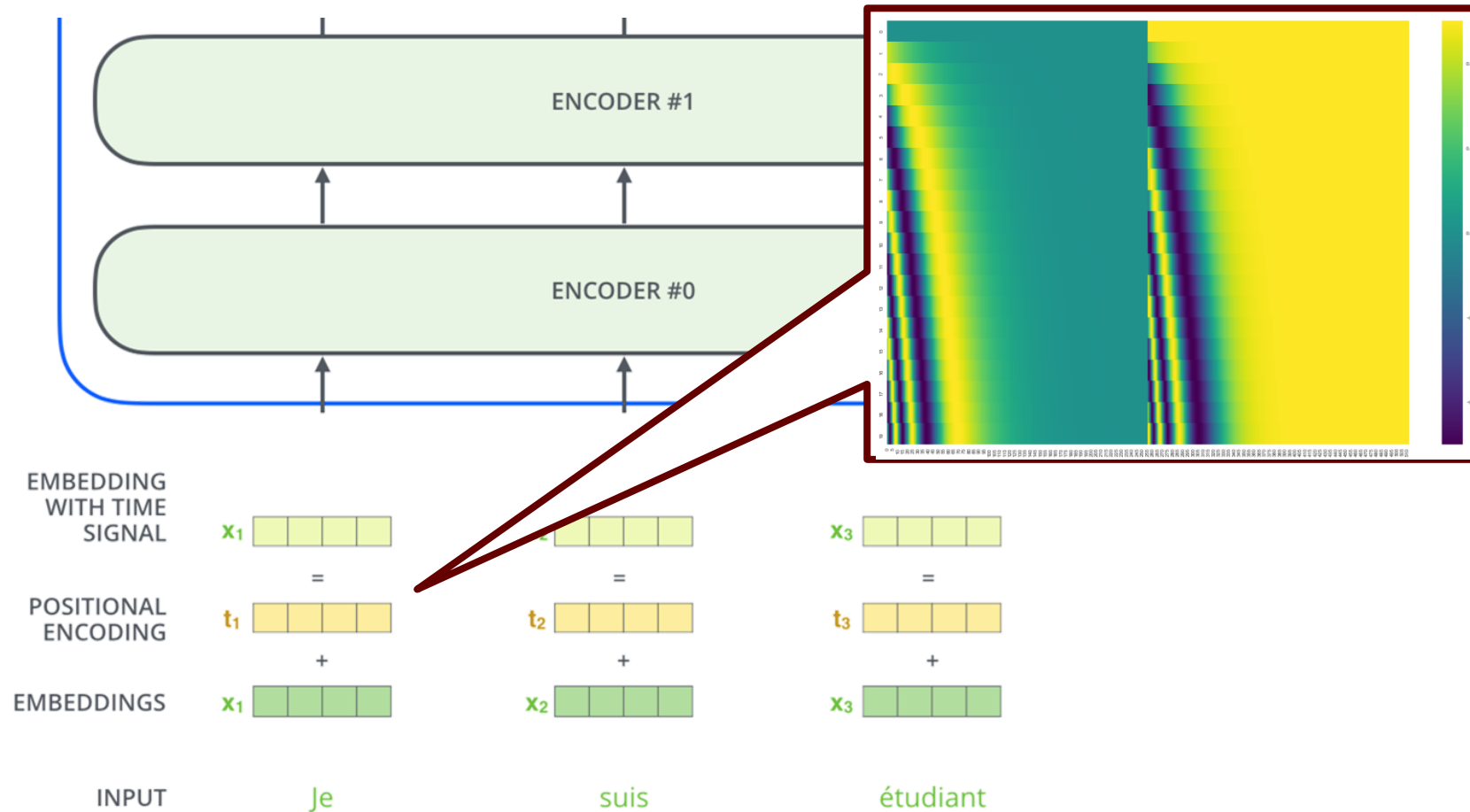
where $\omega_k = \frac{1}{10000^{2k/d}}$

[image source](#)

Vaswani et al, "Attention is all you need", NeurIPS 2017

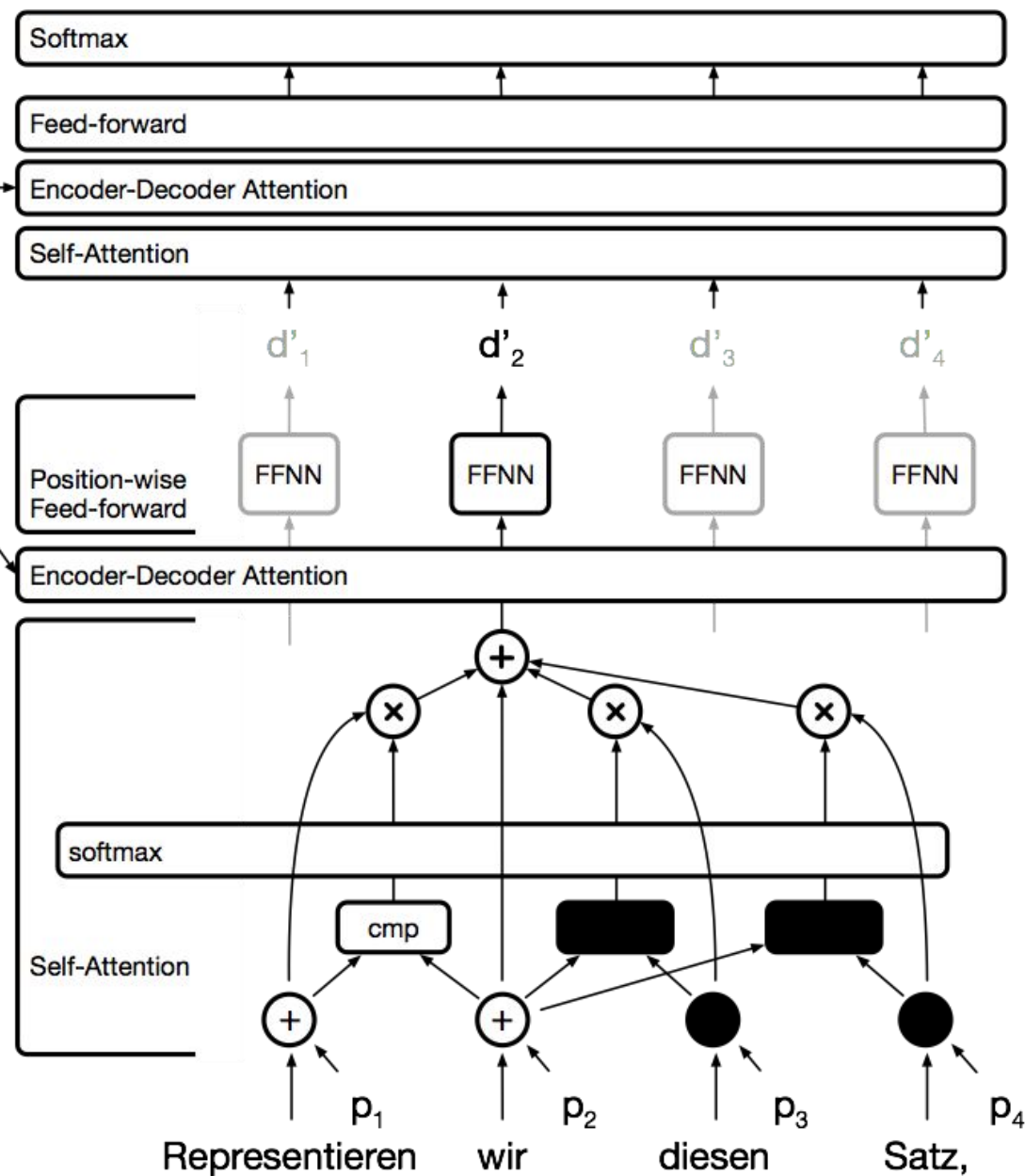
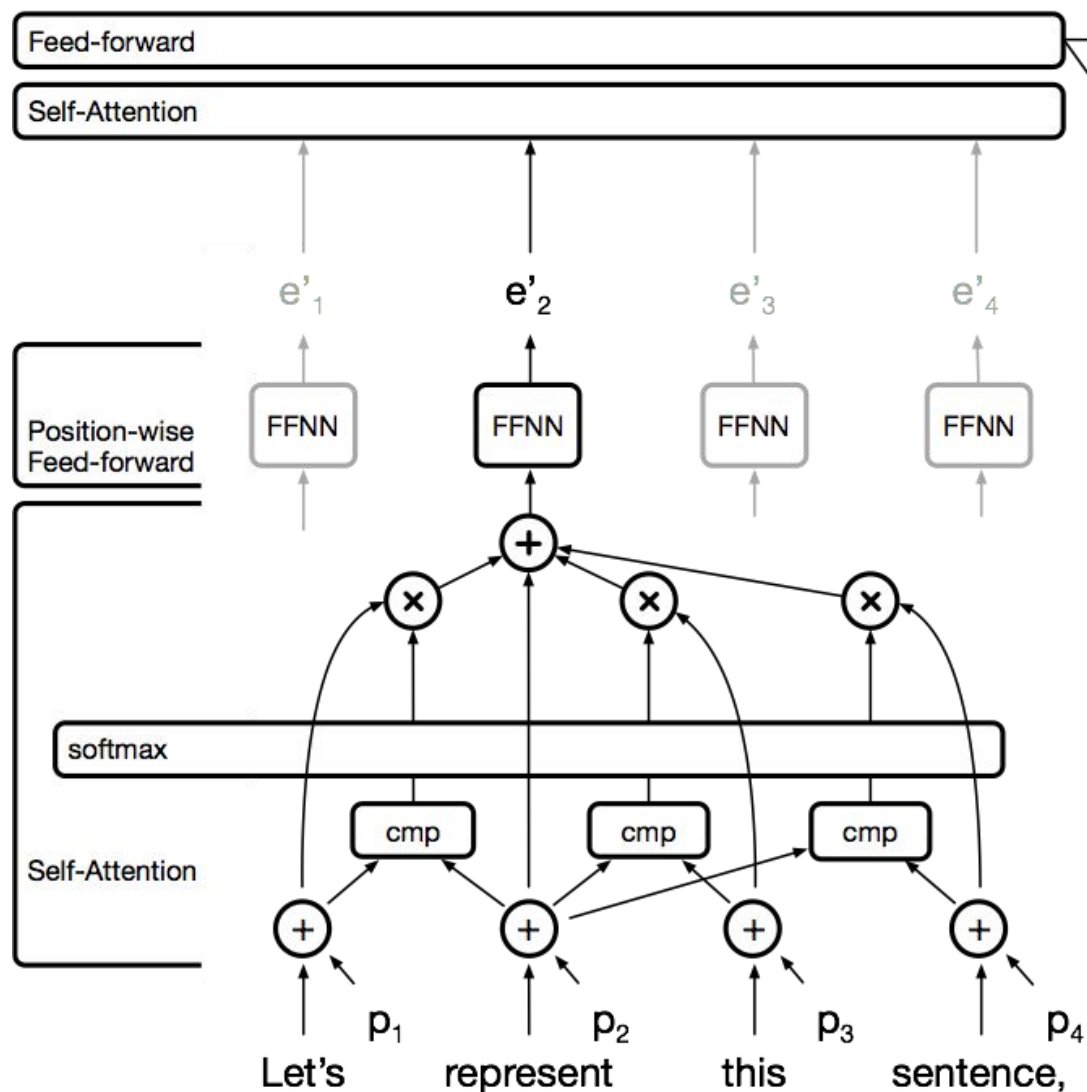
Positional encoding

- Encode each word position relative position

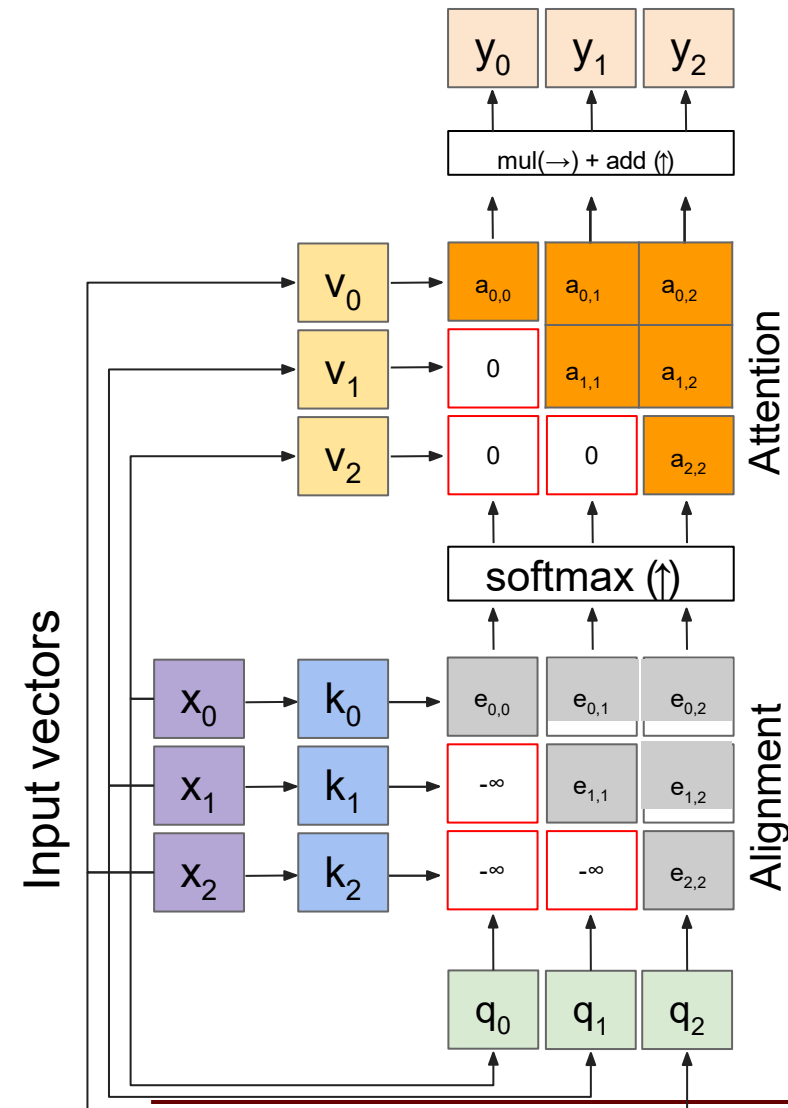


The Transformer

“Attention is All you Need”



Masked self-attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D_v)

Operations:

Key vectors: $\mathbf{k} = \mathbf{x} \mathbf{W}_k$

Value vectors: $\mathbf{v} = \mathbf{x} \mathbf{W}_v$

Query vectors: $\mathbf{q} = \mathbf{x} \mathbf{W}_q$

Alignment: $e_{i,j} = \mathbf{q}_j \cdot \mathbf{k}_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

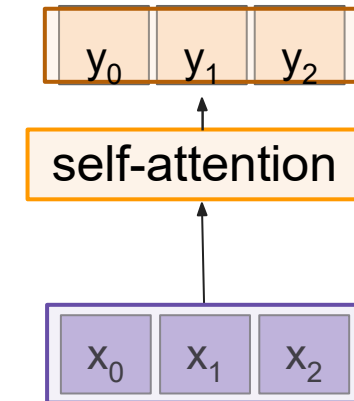
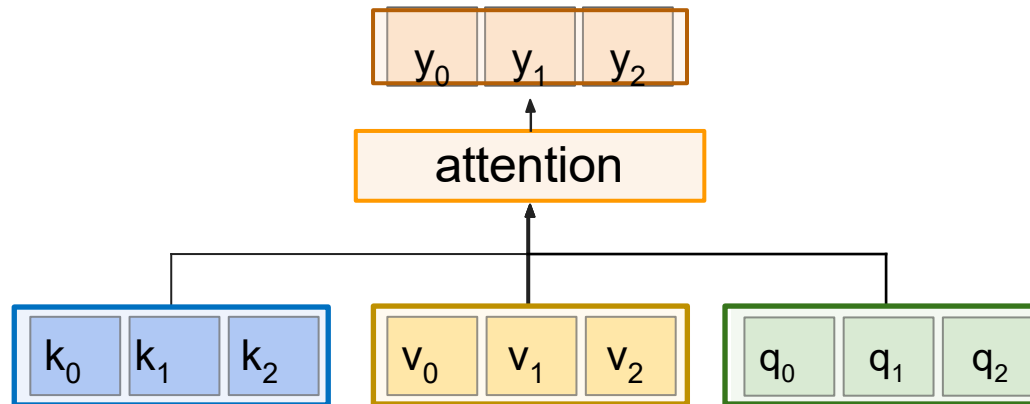
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

- Prevent vectors from looking at future vectors.
- Manually set alignment scores to -infinity (-nan)
- Allows us to parallelize attention across time
- Don't need to calculate the context vectors from the previous timestep first!

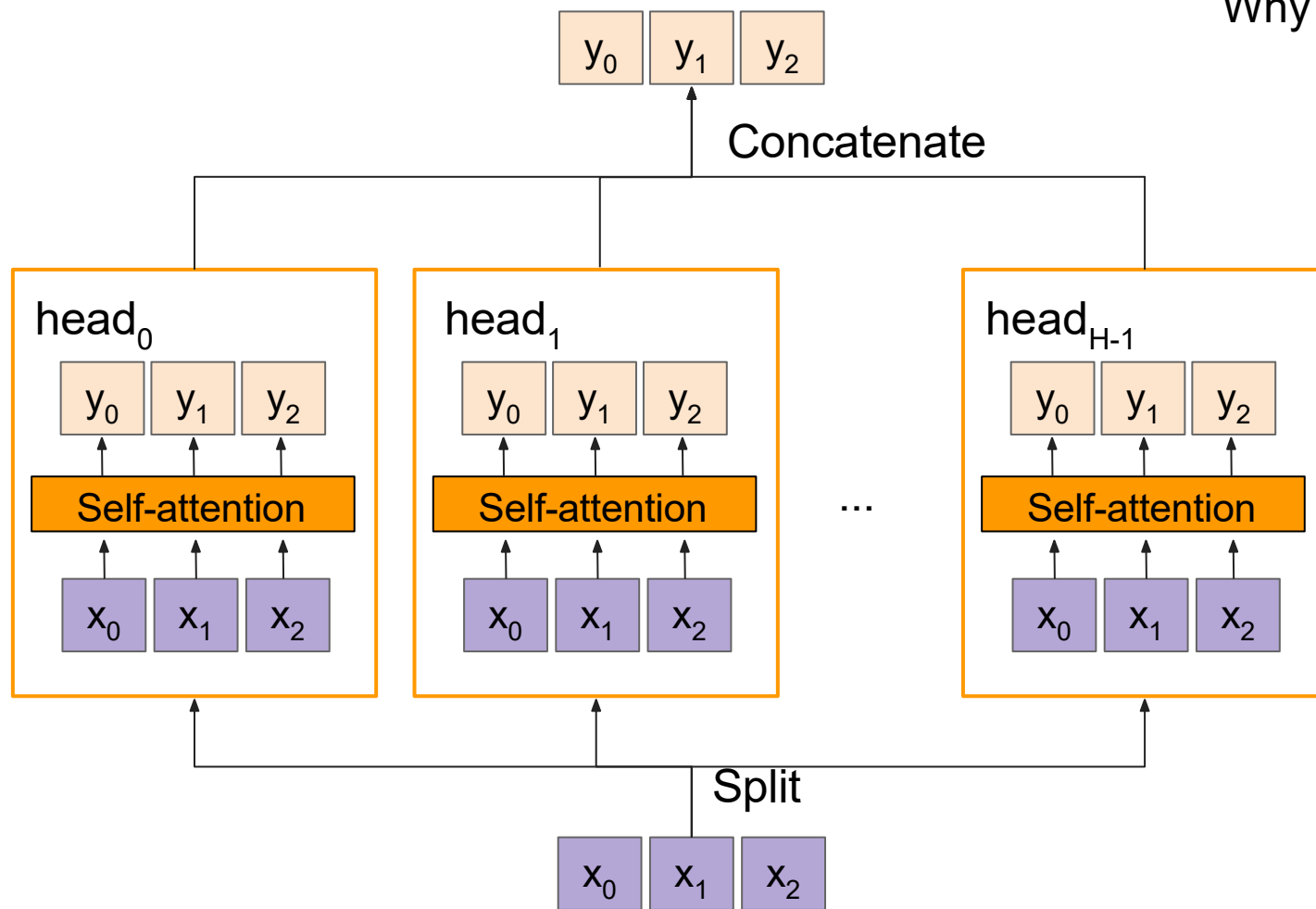
General attention versus self-attention



Multi-head self attention layer

- Multiple self-attention heads in parallel

Why do this?

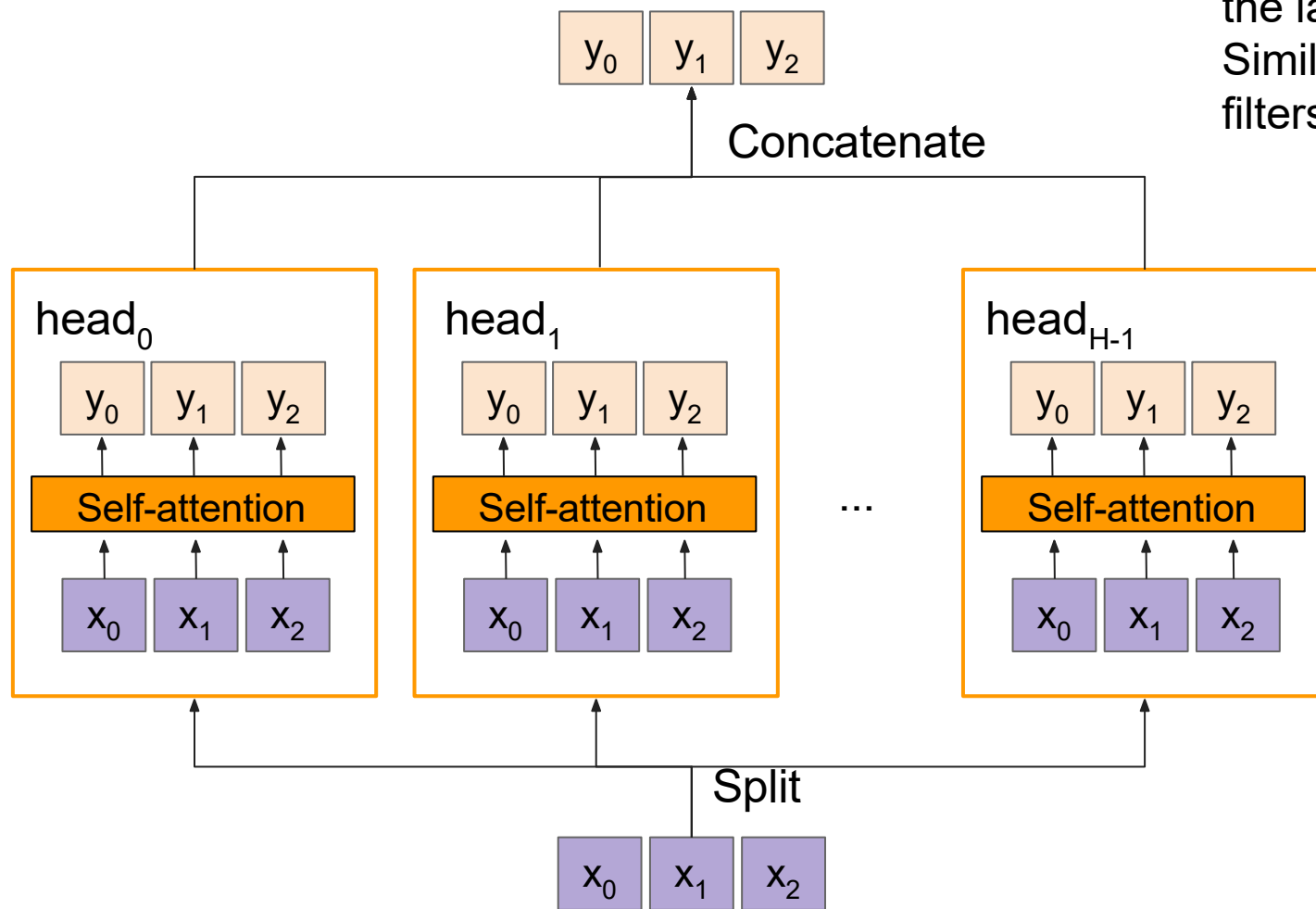


Multi-head self attention layer

- Multiple self-attention heads in parallel

We may want to have multiple sets of queries/keys/values calculated in the layer.

Similar idea to having multiple conv filters learned in a layer



Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- **Transformer Networks**
- BERT

Transformer Networks



Attention Is All You Need

Ashish Vaswani*

Google Brain

avaswani@google.com

Noam Shazeer*

Google Brain

noam@google.com

Niki Parmar*

Google Research

nikip@google.com

Jakob Uszkoreit*

Google Research

usz@google.com

Llion Jones*

Google Research

llion@google.com

Aidan N. Gomez* †

University of Toronto

aidan@cs.toronto.edu

Lukasz Kaiser*

Google Brain

lukaszkaizer@google.com

Illia Polosukhin* ‡

illia.polosukhin@gmail.com

“ImageNet Moment for Natural Language Processing”

Pretraining:

Download a lot of text from the internet

Train a giant Transformer model for language modeling

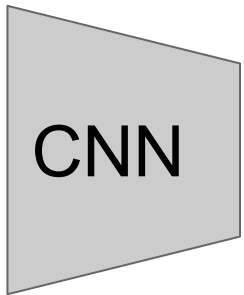
Finetuning:

Fine-tune the Transformer on your own NLP task

Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$



$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Extract spatial features from a pretrained CNN

Features:
 $H \times W \times D$

Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $\mathbf{c} = T_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$T_w(\cdot)$ is the transformer encoder

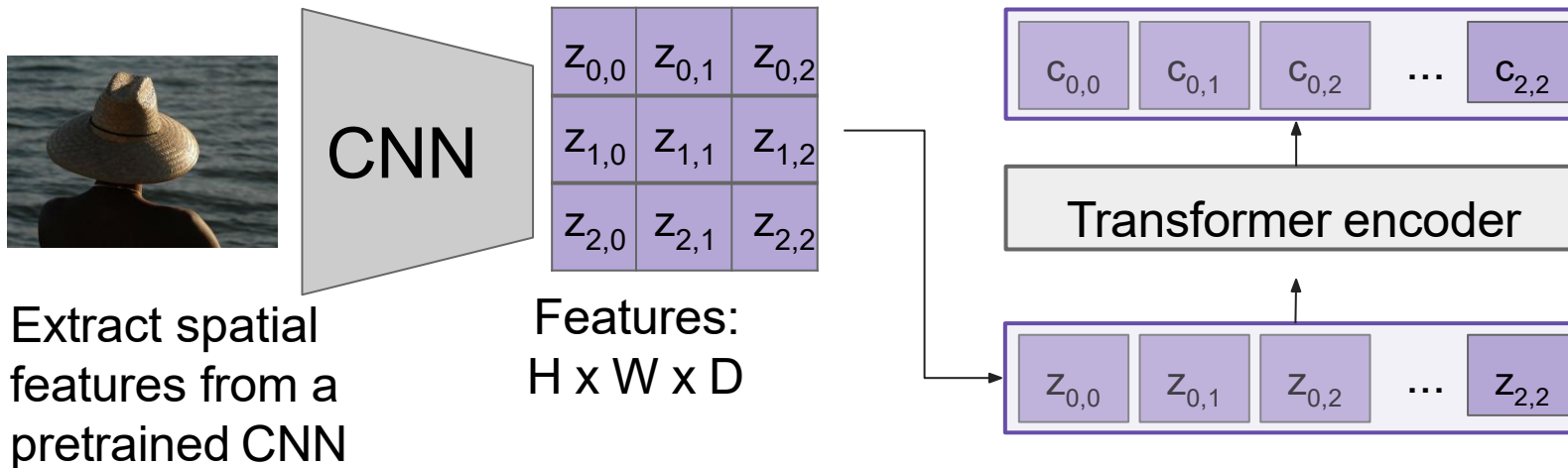


Image Captioning using Transformers

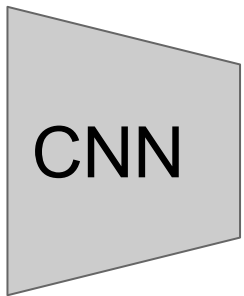
Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $\mathbf{c} = T_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$T_w(\cdot)$ is the transformer encoder



$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

Extract spatial
features from a
pretrained CNN

$\mathbf{K}$ and $\mathbf{V}$ values
from the input

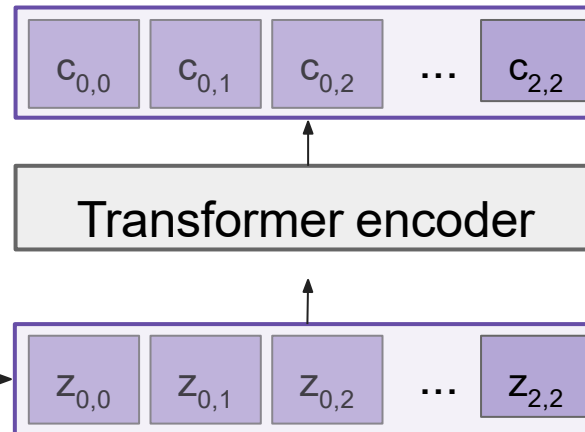


Image Captioning using Transformers

Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

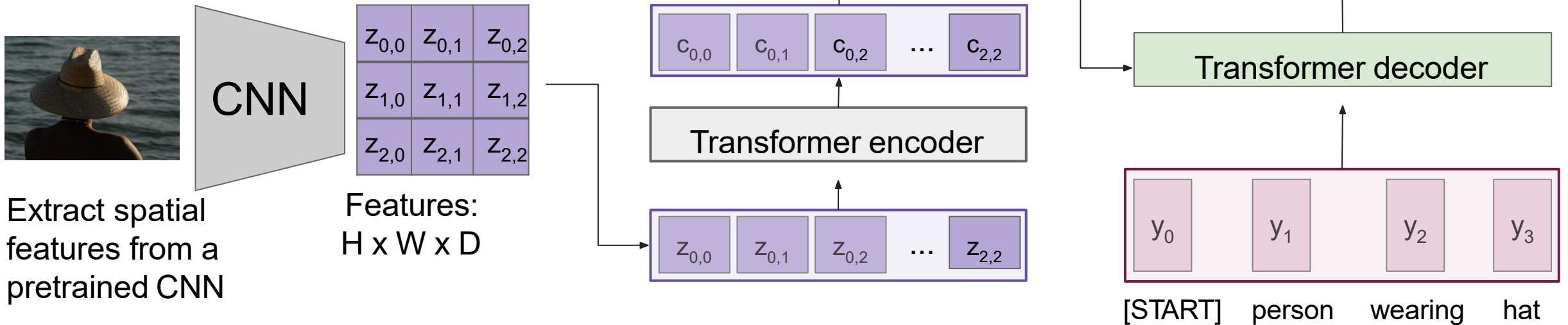
Decoder: $y_t = T_D(\mathbf{y}_{0:t-1}, \mathbf{c})$

where $T_D(\cdot)$ is the transformer decoder

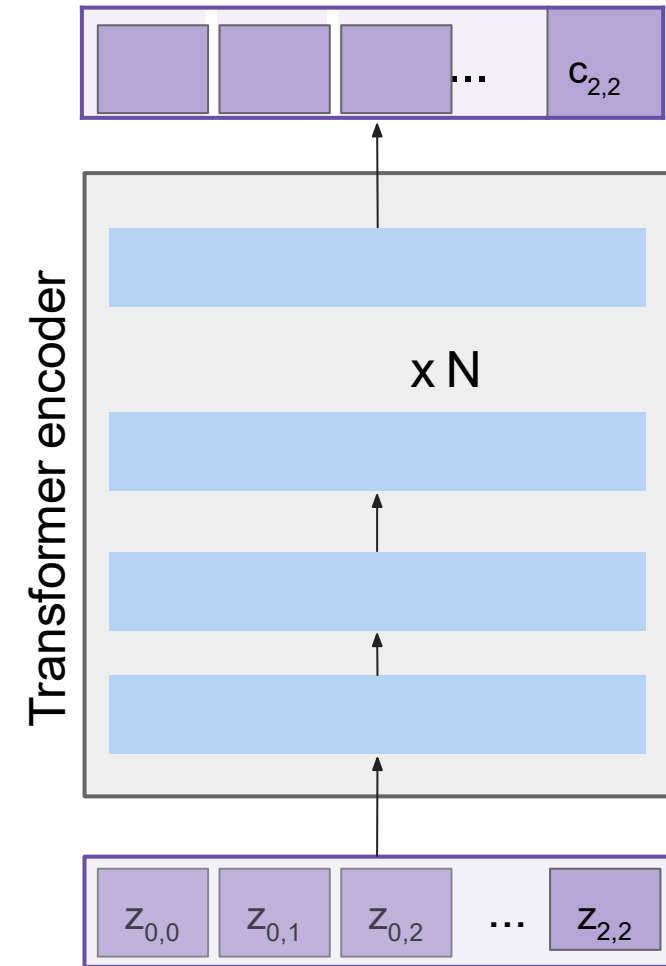
Encoder: $\mathbf{c} = T_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$T_w(\cdot)$ is the transformer encoder



The Transformer encoder block

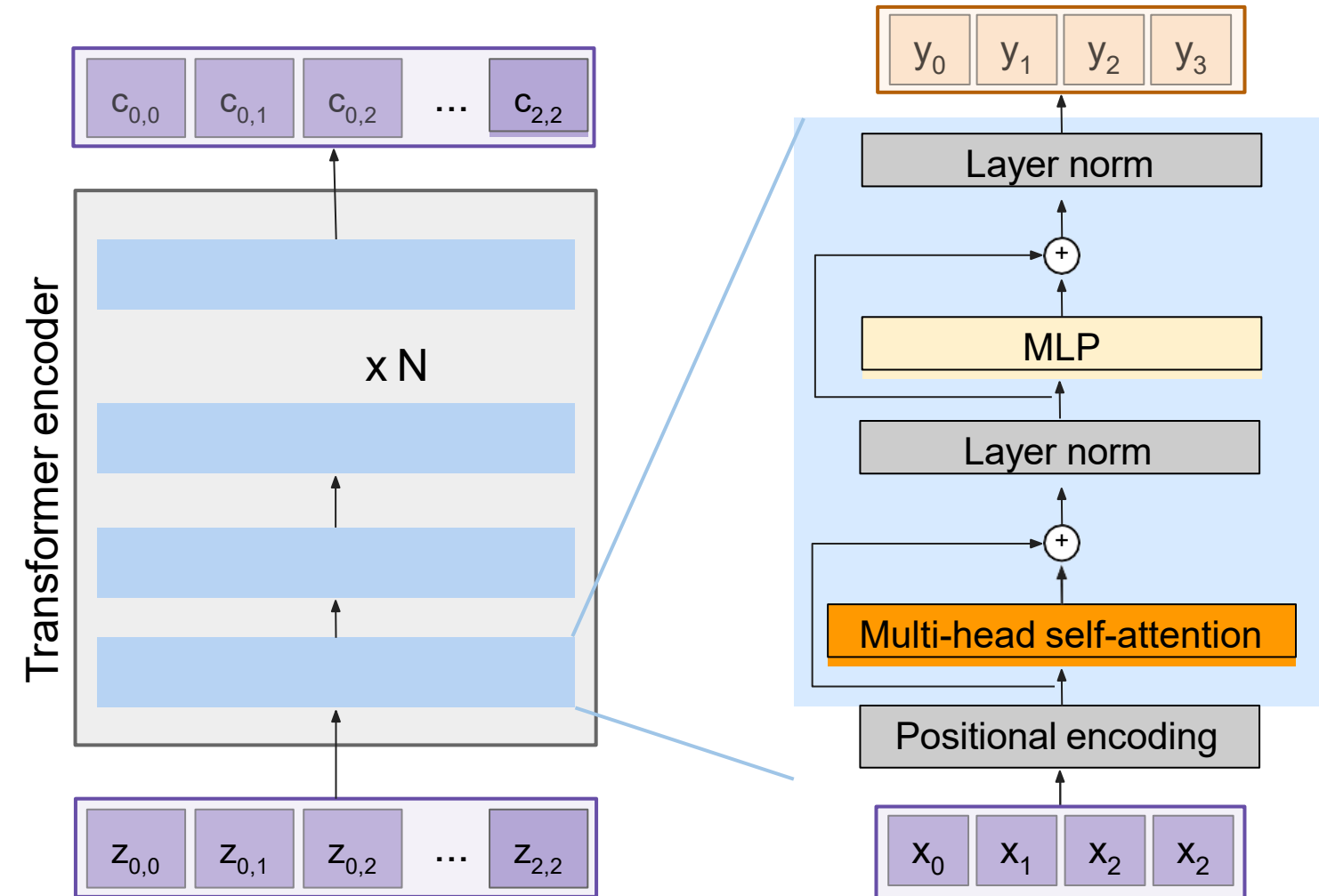


Made up of N encoder blocks.

In vaswani et al. $N = 6$, $D_q = 512$

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer encoder block



Transformer Encoder Block:

Inputs: Set of vectors $\mathbf{x}$

Outputs: Set of vectors $\mathbf{y}$

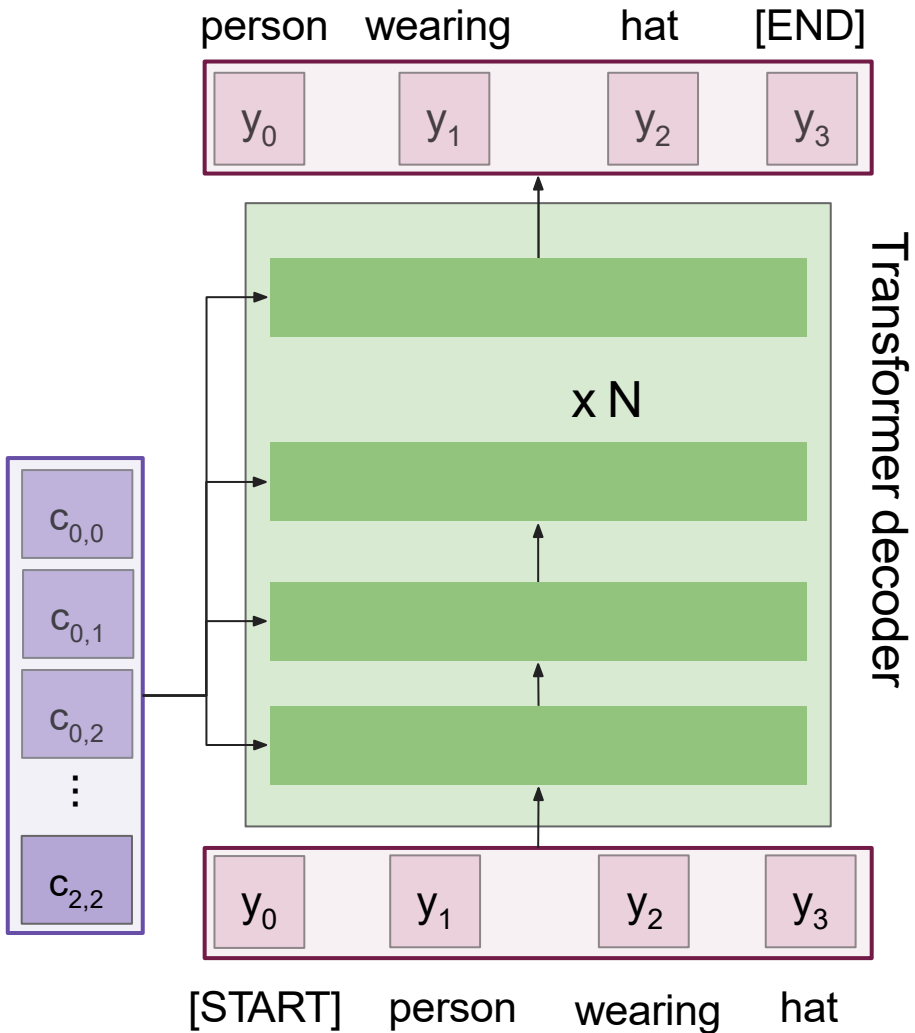
Self-attention is the only interaction between vectors.

Layer norm and MLP operate independently per vector.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer decoder block



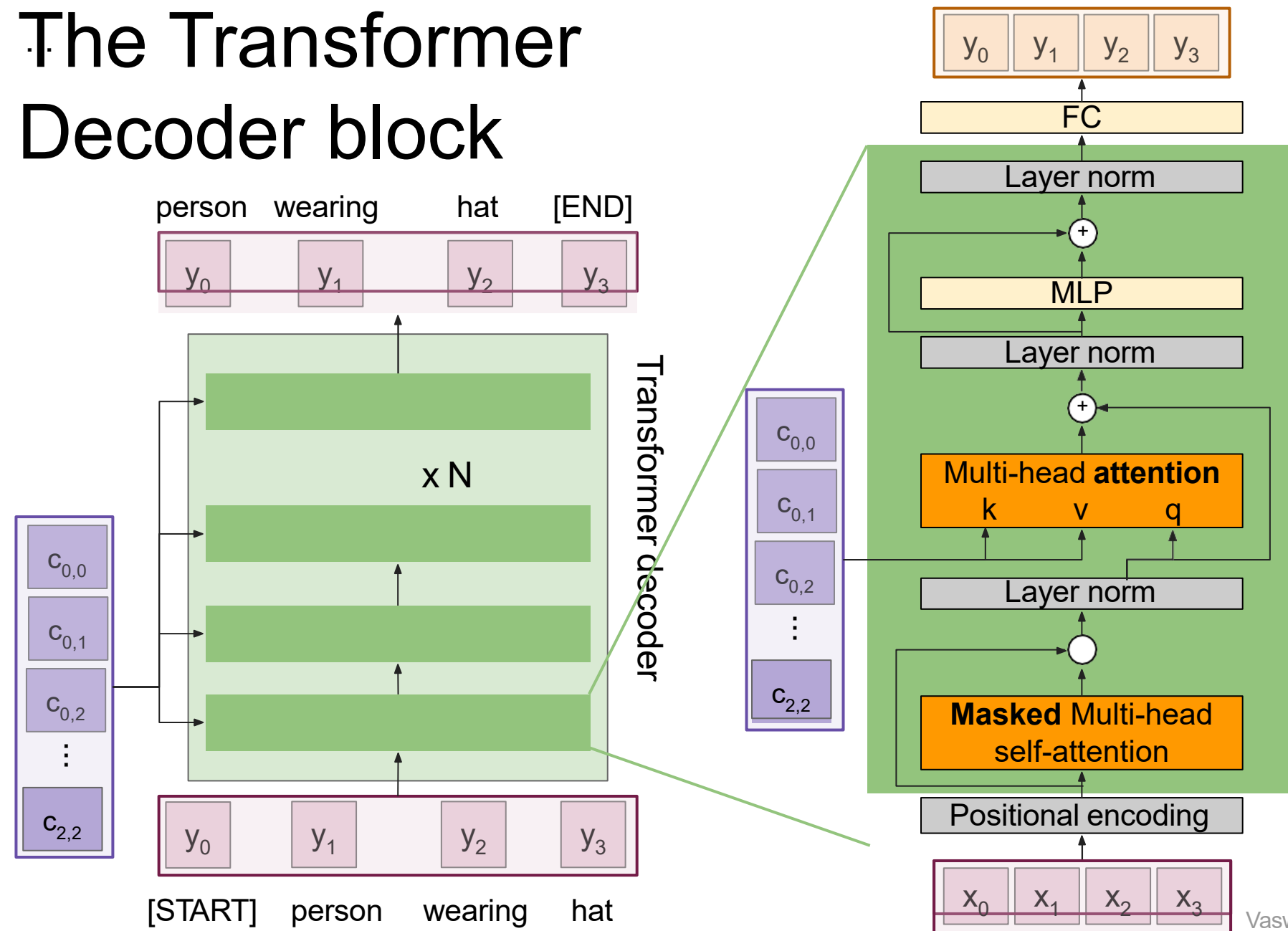
Made up of N decoder blocks.

In vaswani et al. $N = 6$, $D_q = 512$

Vaswani et al, "Attention is all you need", NeurIPS 2017

The Transformer

Decoder block



Transformer Decoder Block:

Inputs: Set of vectors $\mathbf{x}$ and Set of context vectors $\mathbf{c}$.

Outputs: Set of vectors $\mathbf{y}$.

Masked Self-attention only interacts with past inputs.

Multi-head attention block is NOT self-attention. It attends over encoder outputs.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, "Attention is all you need", NeurIPS 2017

Comparing RNNs to Transformers

RNNs

- (+) LSTMs work reasonably well for long sequences.
- (-) Expects an ordered sequences of inputs
- (-) Sequential computation: subsequent hidden states can only be computed after the previous ones are done.

Transformer:

- (+) Good at long sequences. Each attention calculation looks at all inputs.
- (+) Can operate over unordered sets or ordered sequences with positional encodings.
- (+) Parallel computation: All alignment and attention scores for all inputs can be done in parallel.
- (-) Requires a lot of memory: $N \times M$ alignment and attention scalars need to be calculated and stored for a single self-attention head. (but GPUs are getting bigger and better)

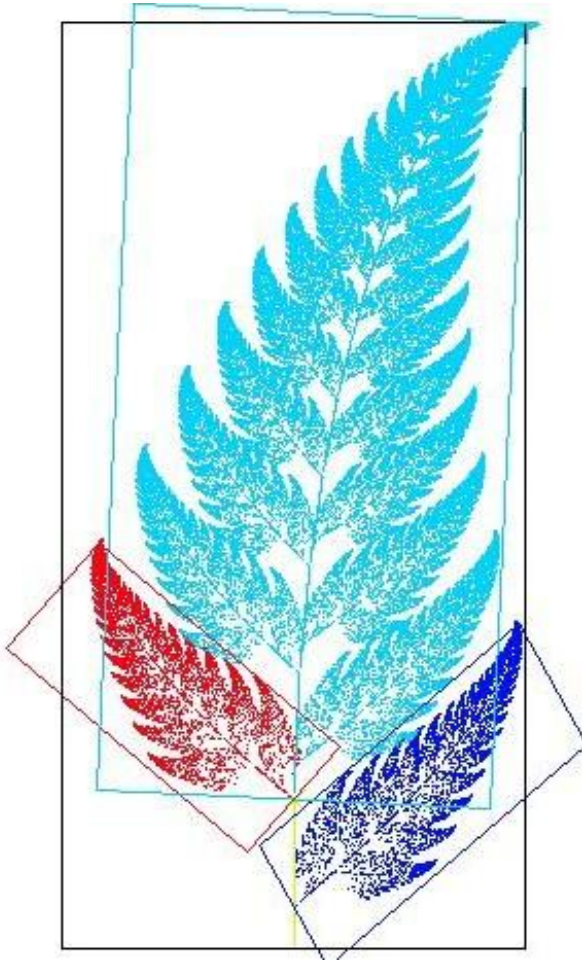
Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- **Transformer Networks**
 - Transformer Networks for Images
- BERT

Transformer Networks for Images



Self-similarity in images



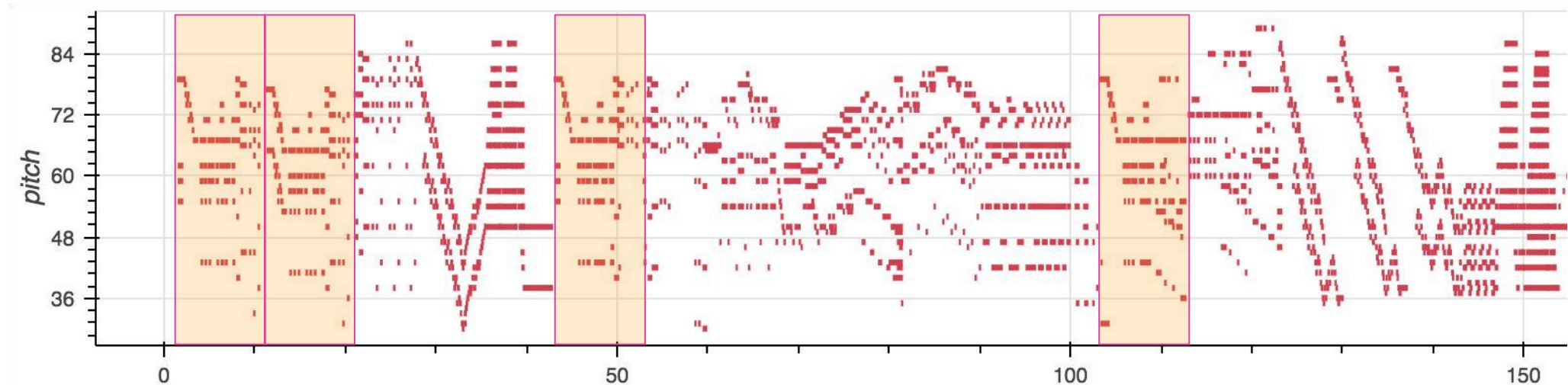
Self-Similarity in Images



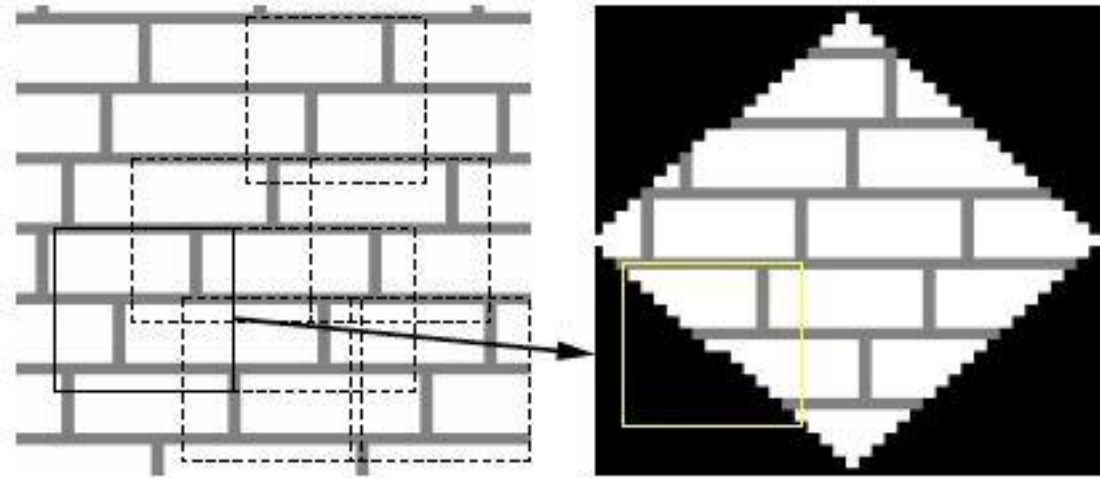
Starry Night (Van Gogh, June 1889)

Self-similarity in music

Motifs repeat, immediately and also at a distance



Texture Synthesis with Self-Similarity



Texture Synthesis by Non-parametric Sampling (Efros and Leung, 1999)

Non-local Means

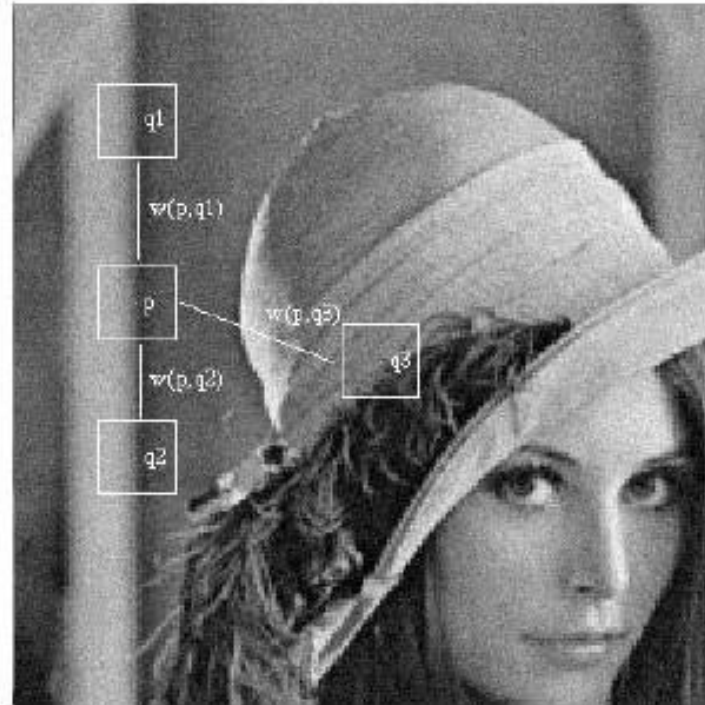
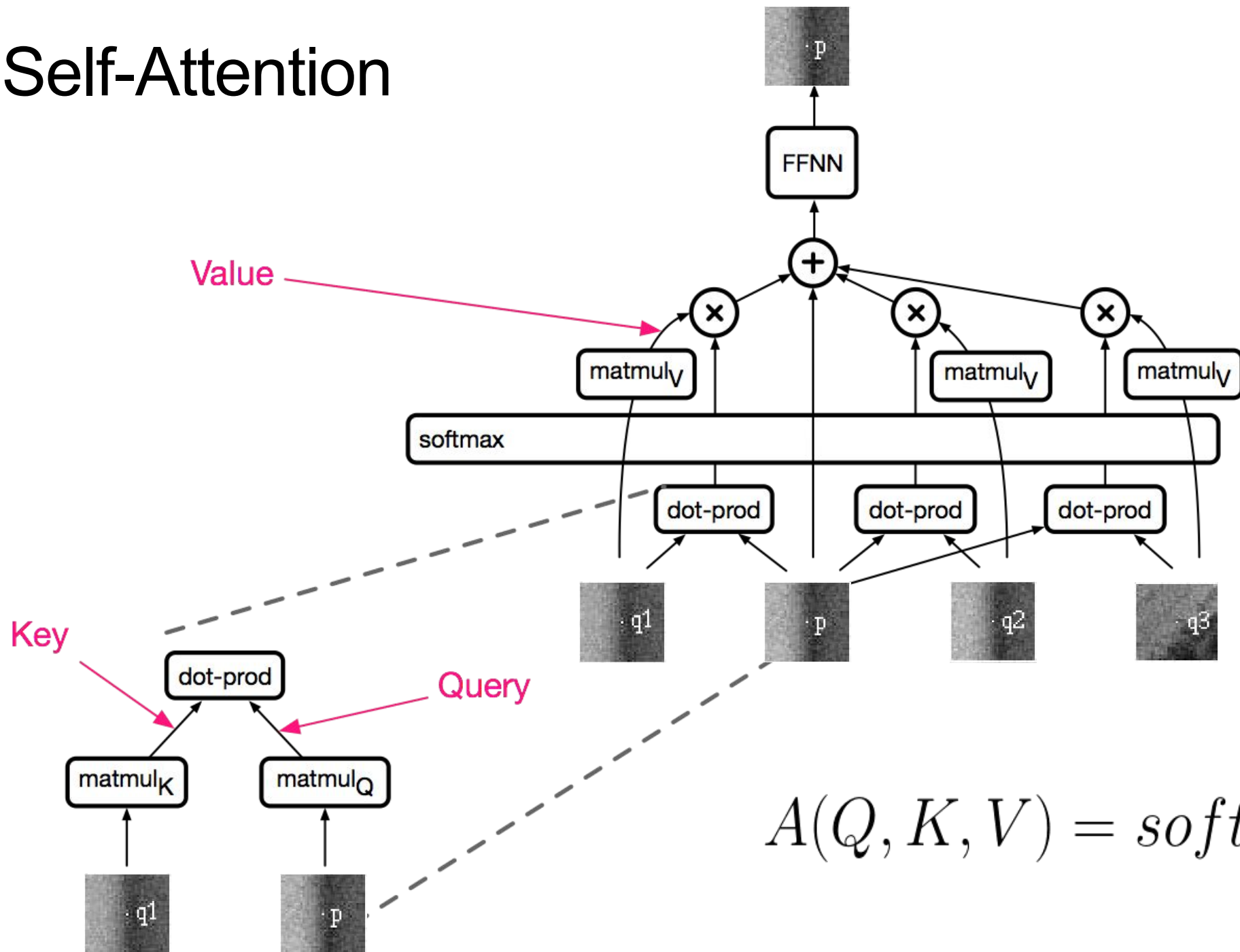


Figure 1. Scheme of NL-means strategy. Similar pixel neighborhoods give a large weight, $w(p,q_1)$ and $w(p,q_2)$, while much different neighborhoods give a small weight $w(p,q_3)$.

Self-Attention



$$A(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The Image Transformer

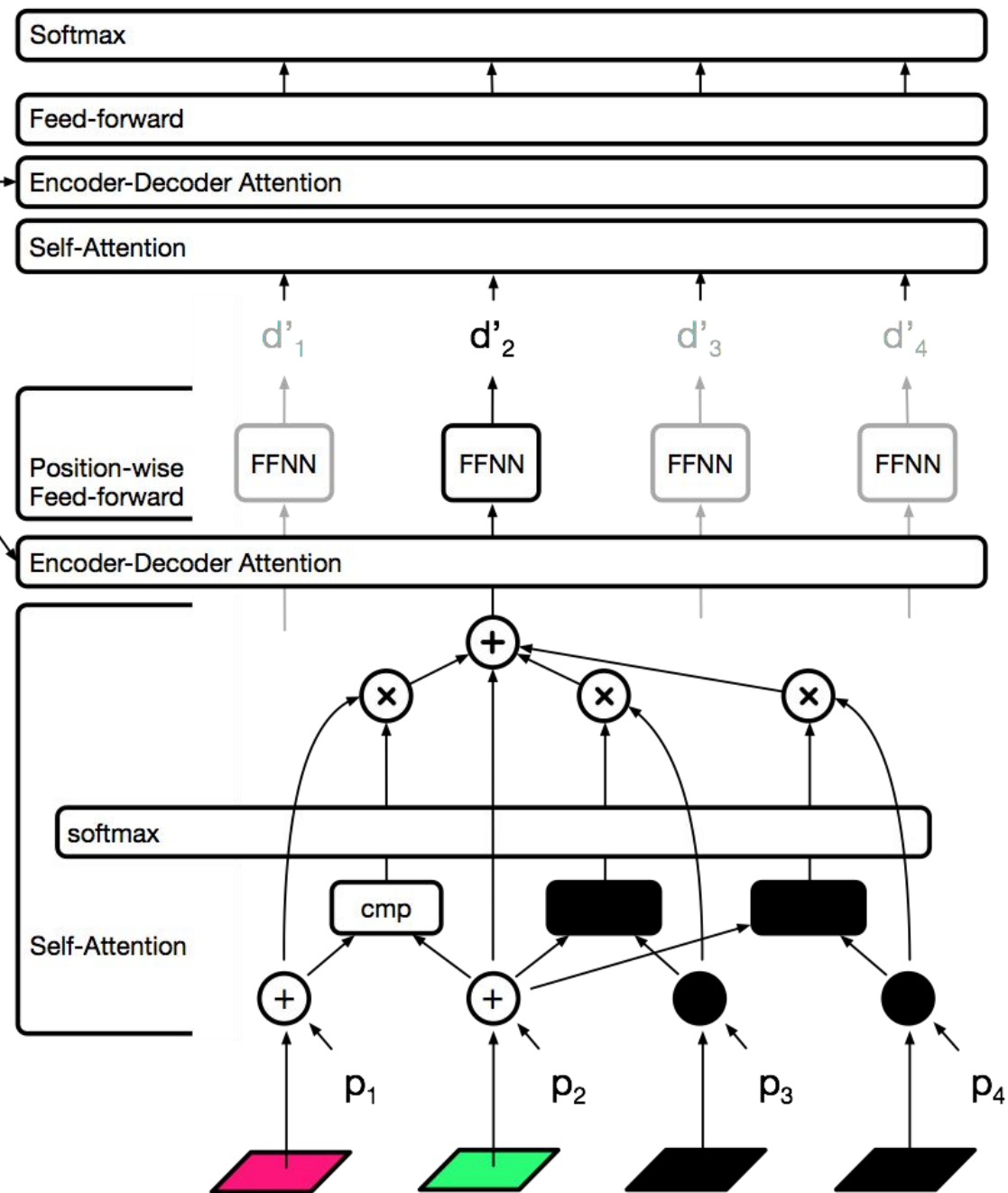
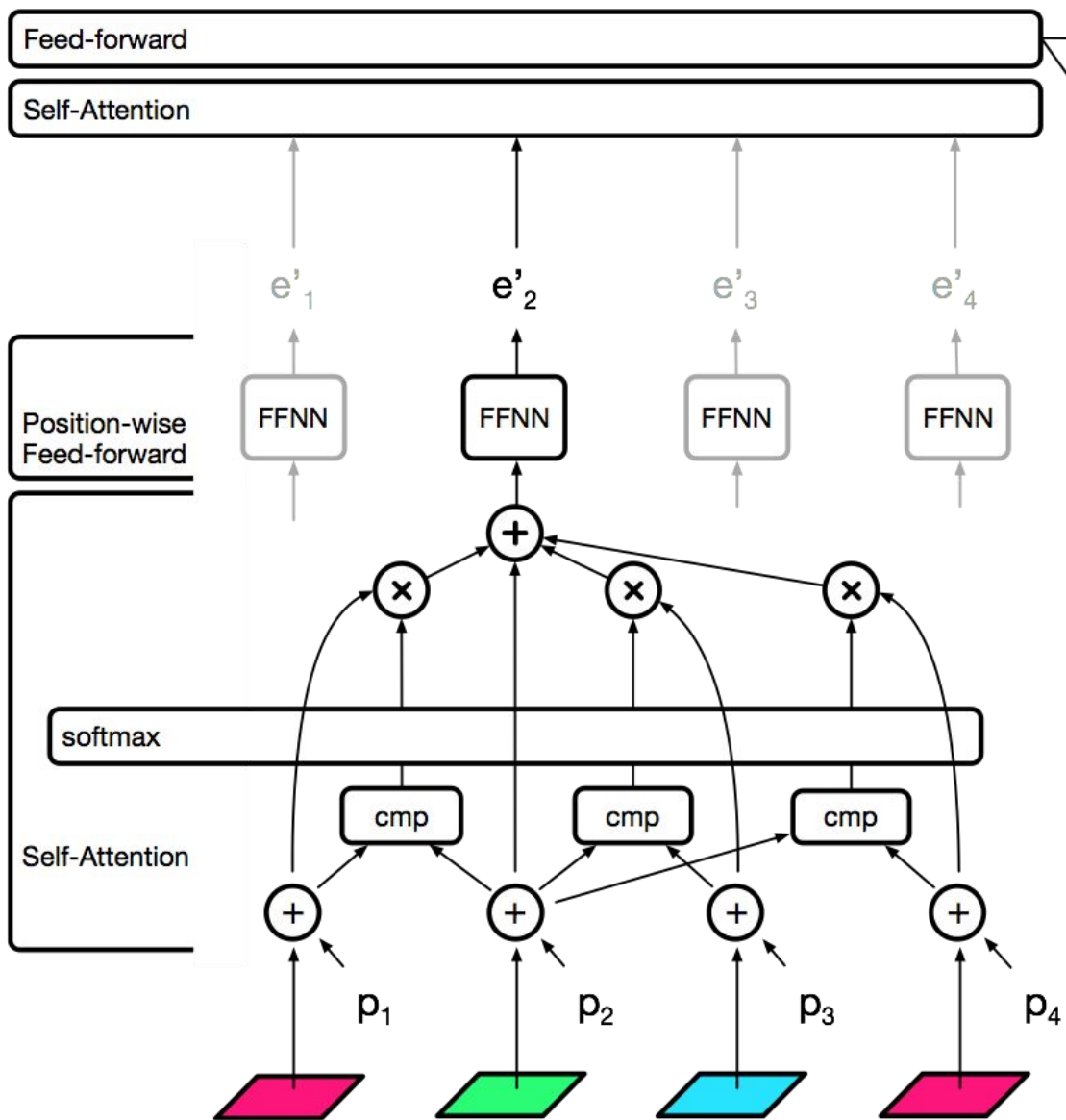
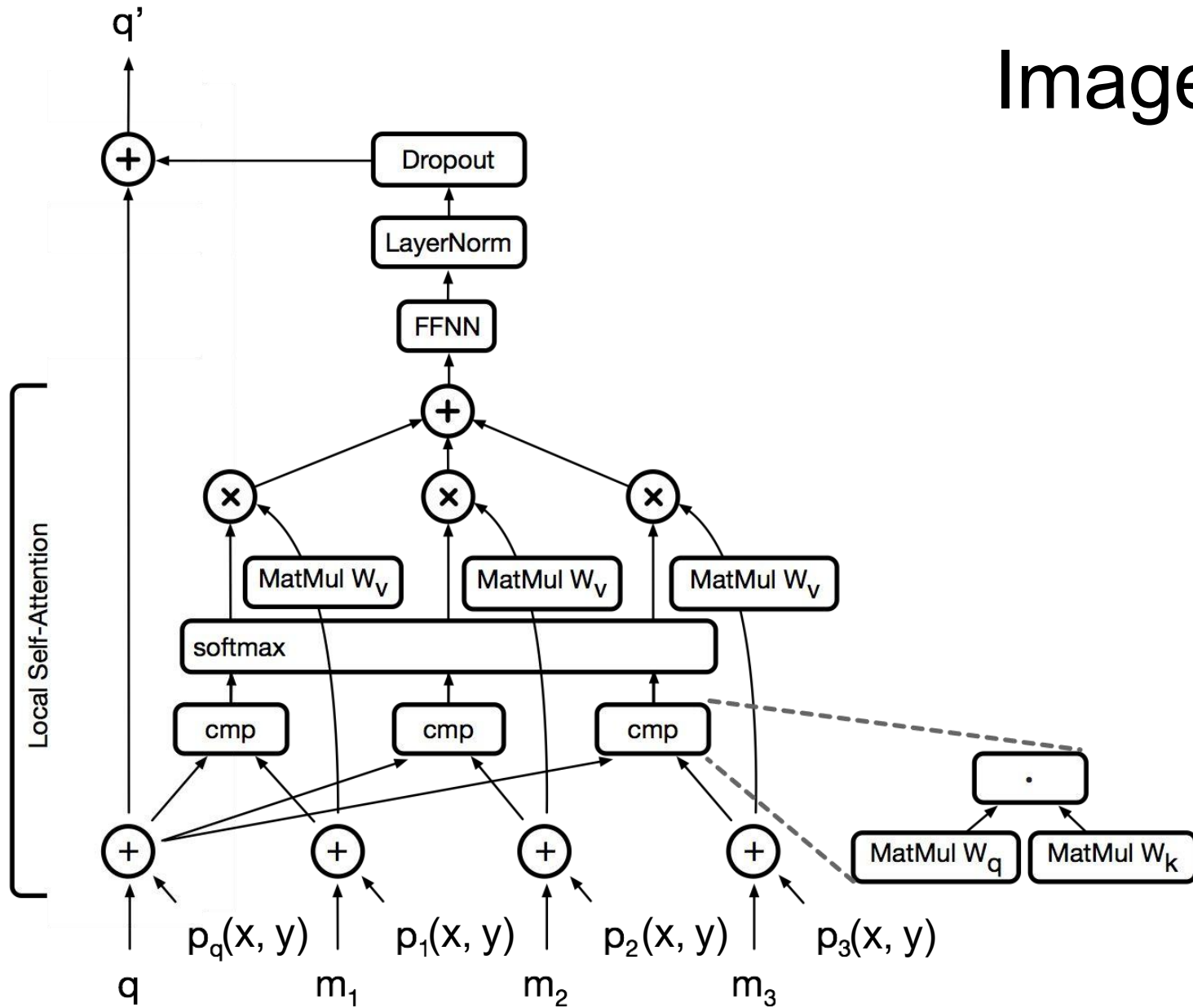


Image Transformer Layer



Self-attention with relative positional encoding

Self-attention:

$$z_i = \sum_j \alpha_{ij} (x_j W^V)$$

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_k}} \quad \text{and} \quad \alpha = \text{softmax}(e)$$

Self-attention with relative positional encoding:

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + \mathbf{a}_{ij}^K)^T}{\sqrt{d_k}}$$

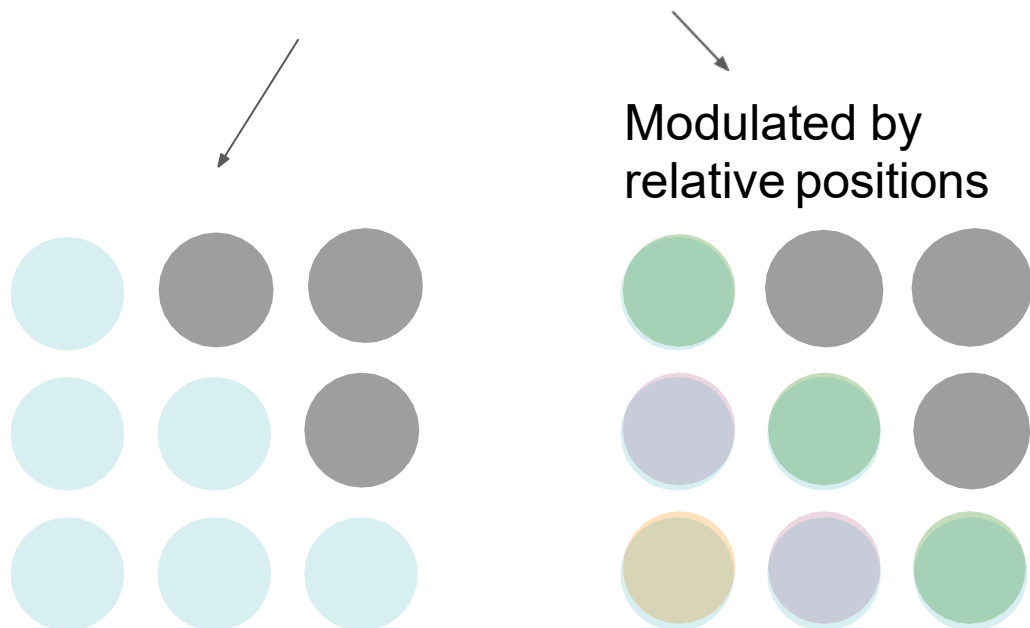
$$z_i = \sum_j \alpha_{ij} (x_j W^V + \mathbf{a}_{ij}^V)$$

$\mathbf{a}_{ij}^K$ and $\mathbf{a}_{ij}^V$ are **learnable embedding vectors**

function of the relative distance $(i - j)$

Self-attention with relative positional encoding

$$\text{softmax}(QK^\top + Qf(E_{rel}))$$



Self-attention with relative positional encoding:

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K + \mathbf{a}_{ij}^K)^T}{\sqrt{d_k}}$$

$$z_i = \sum_j \alpha_{ij} (x_j W^V + \mathbf{a}_{ij}^V)$$

$\mathbf{a}_{ij}^K$ and $\mathbf{a}_{ij}^V$ are **learnable embedding vectors**

function of the relative distance $(i - j)$

Image Captioning using transformers

- No recurrence at all

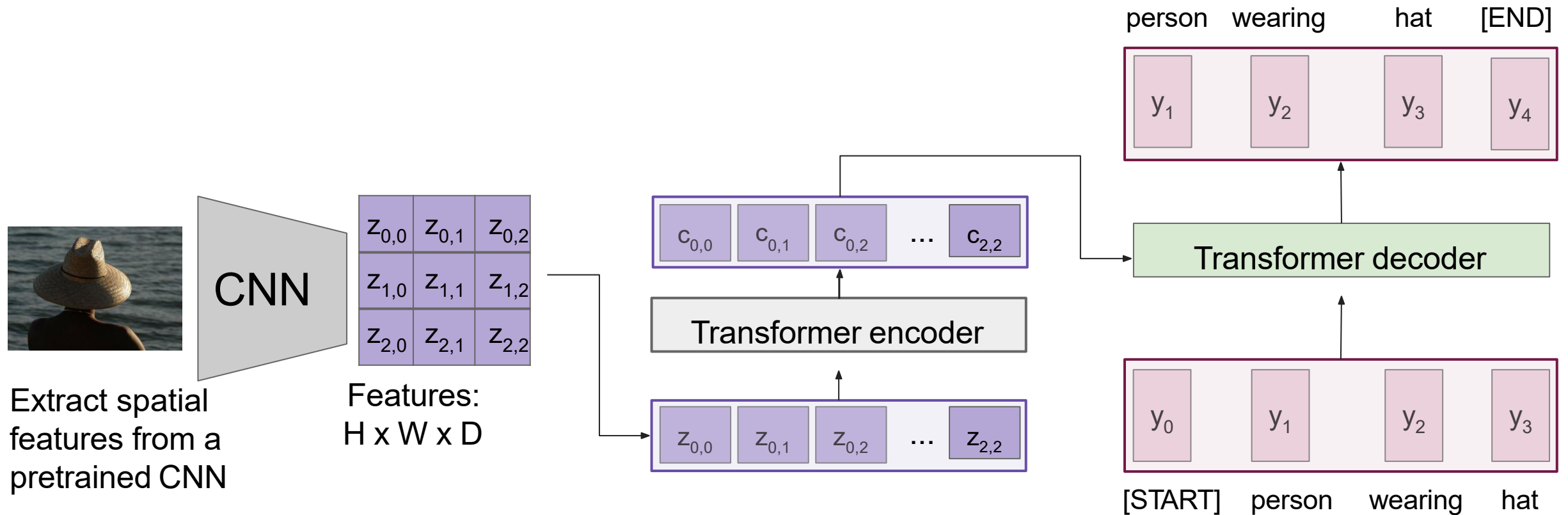


Image Captioning using transformers

- Perhaps we don't need convolutions at all?

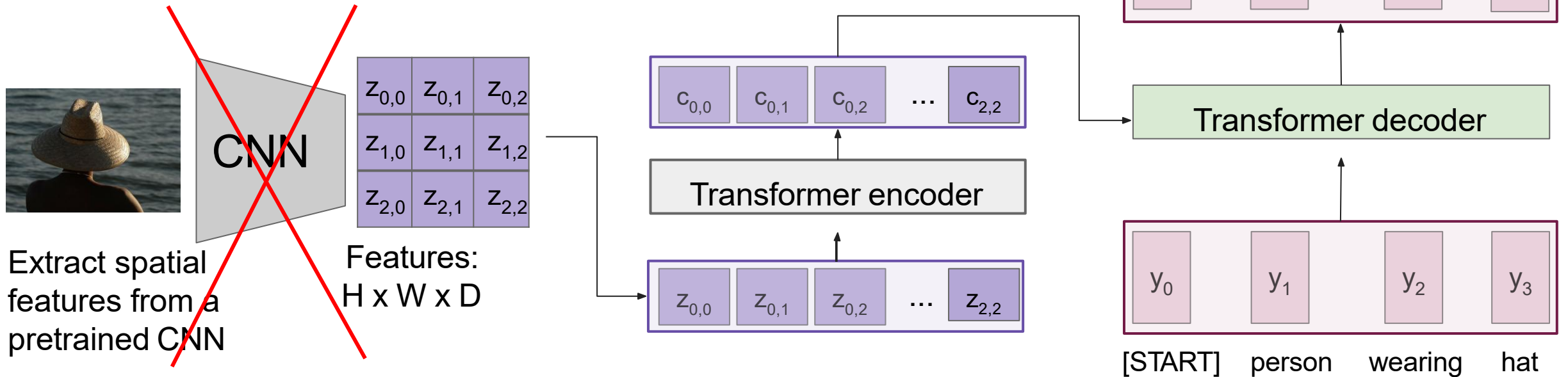
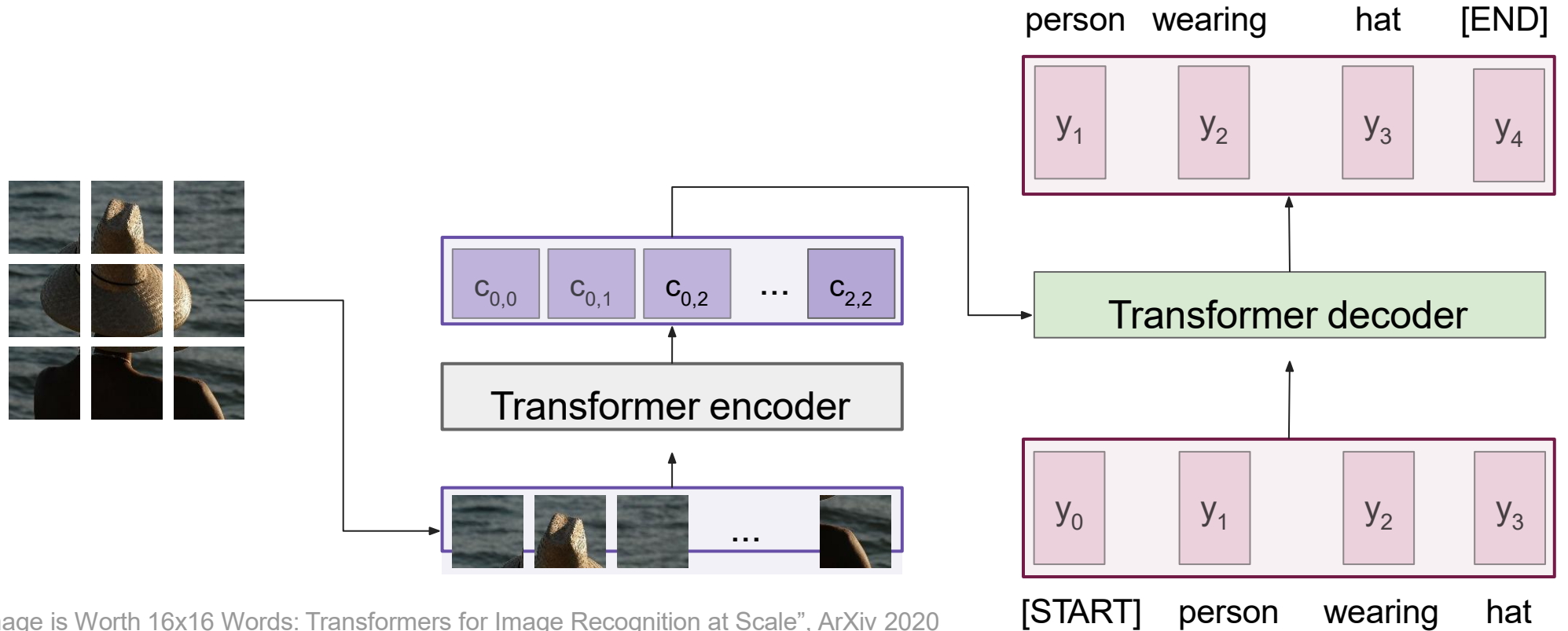


Image Captioning using **ONLY** transformers

- Transformers from pixels to language



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020
[Colab link](#) to an implementation of vision transformers

Vision Transformers vs. ResNets

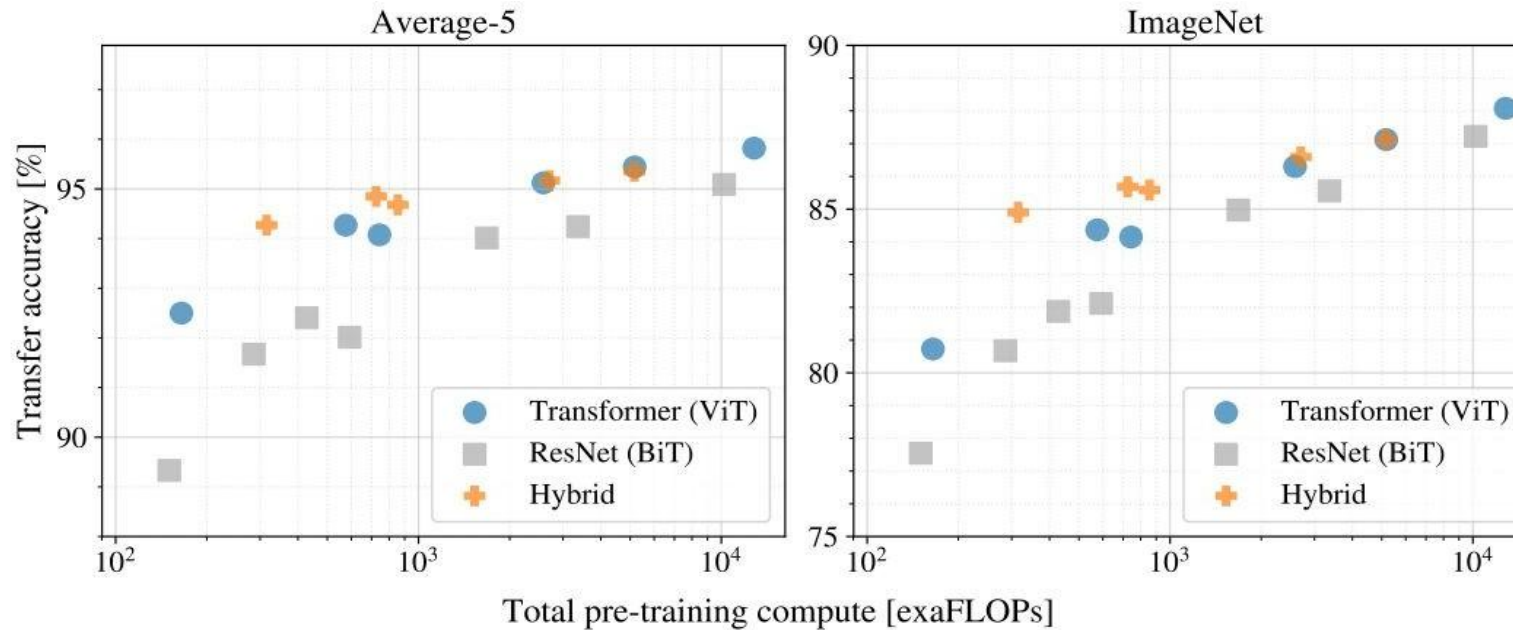
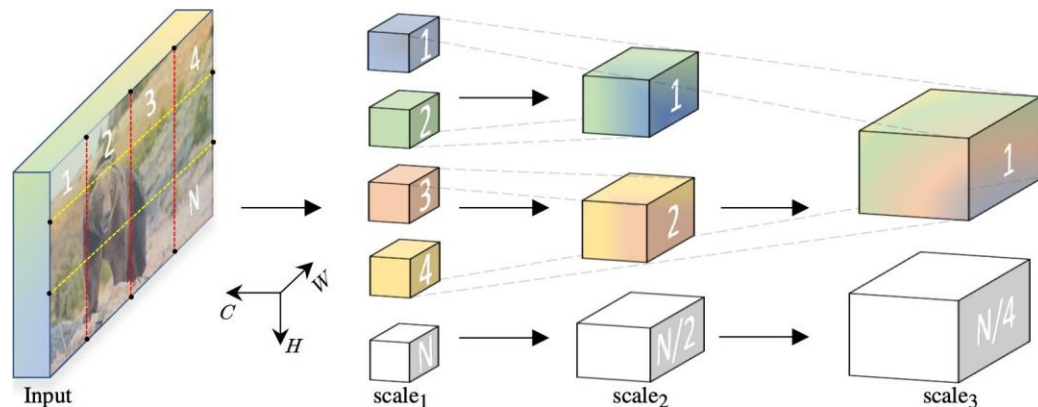


Figure 5: Performance versus cost for different architectures: Vision Transformers, ResNets, and hybrids. Vision Transformers generally outperform ResNets with the same computational budget. Hybrids improve upon pure Transformers for smaller model sizes, but the gap vanishes for larger models.

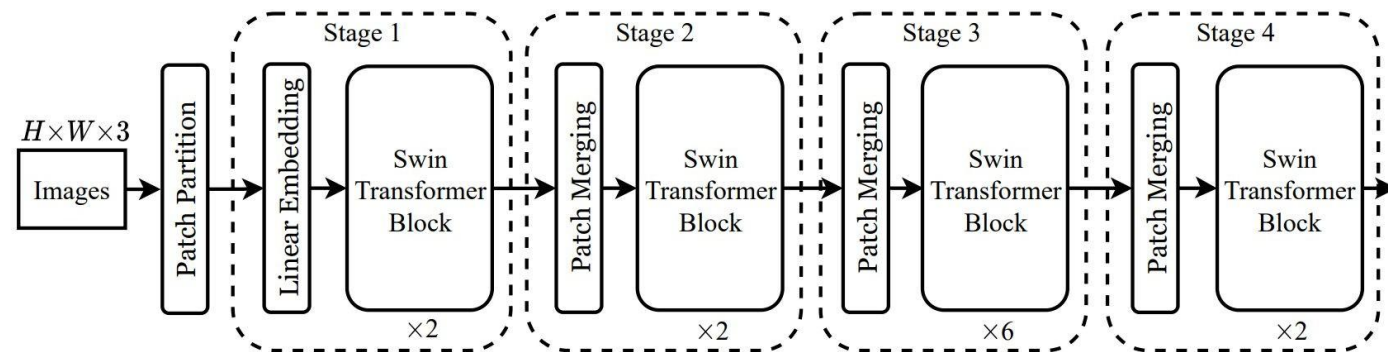
Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020

[Colab link](#) to an implementation of vision transformers

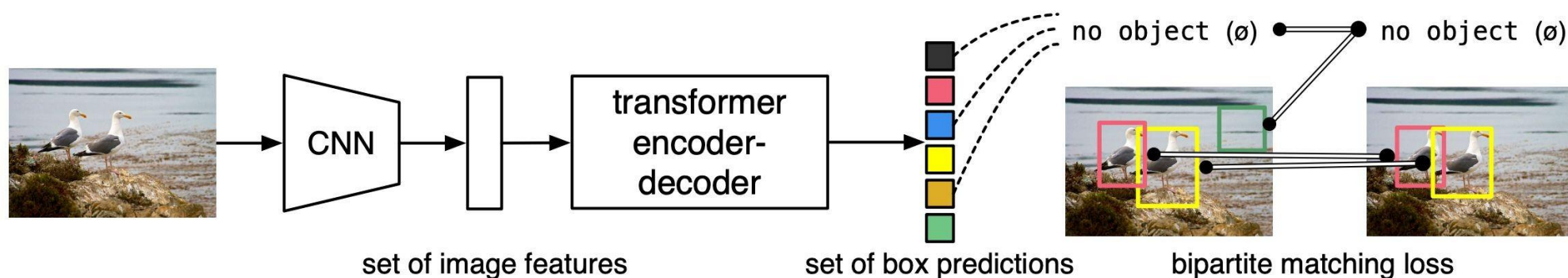
Vision Transformers



Fan et al, "Multiscale Vision Transformers", ICCV 2021



Liu et al, "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows", CVPR 2021



Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

Outline

- Towards Transformers: Attention and self-attention
- Positional encoding and masked self-attention
- Transformer Networks
- BERT

BERT



SAPIENZA
UNIVERSITÀ DI ROMA

Problem with Previous Methods

- **Problem:** Language models only use left context *or* right context, but language understanding is bidirectional.
- Why are LMs unidirectional?
- Reason 1: Directionality is needed to generate a well-formed probability distribution.
- Reason 2: Words can “see themselves” in a bidirectional encoder.

Masked LM

- **Solution:** Mask out $k\%$ of the input words, and then predict the masked words
 - We always use $k=15\%$

the man went to the [MASK] to buy a [MASK] of milk

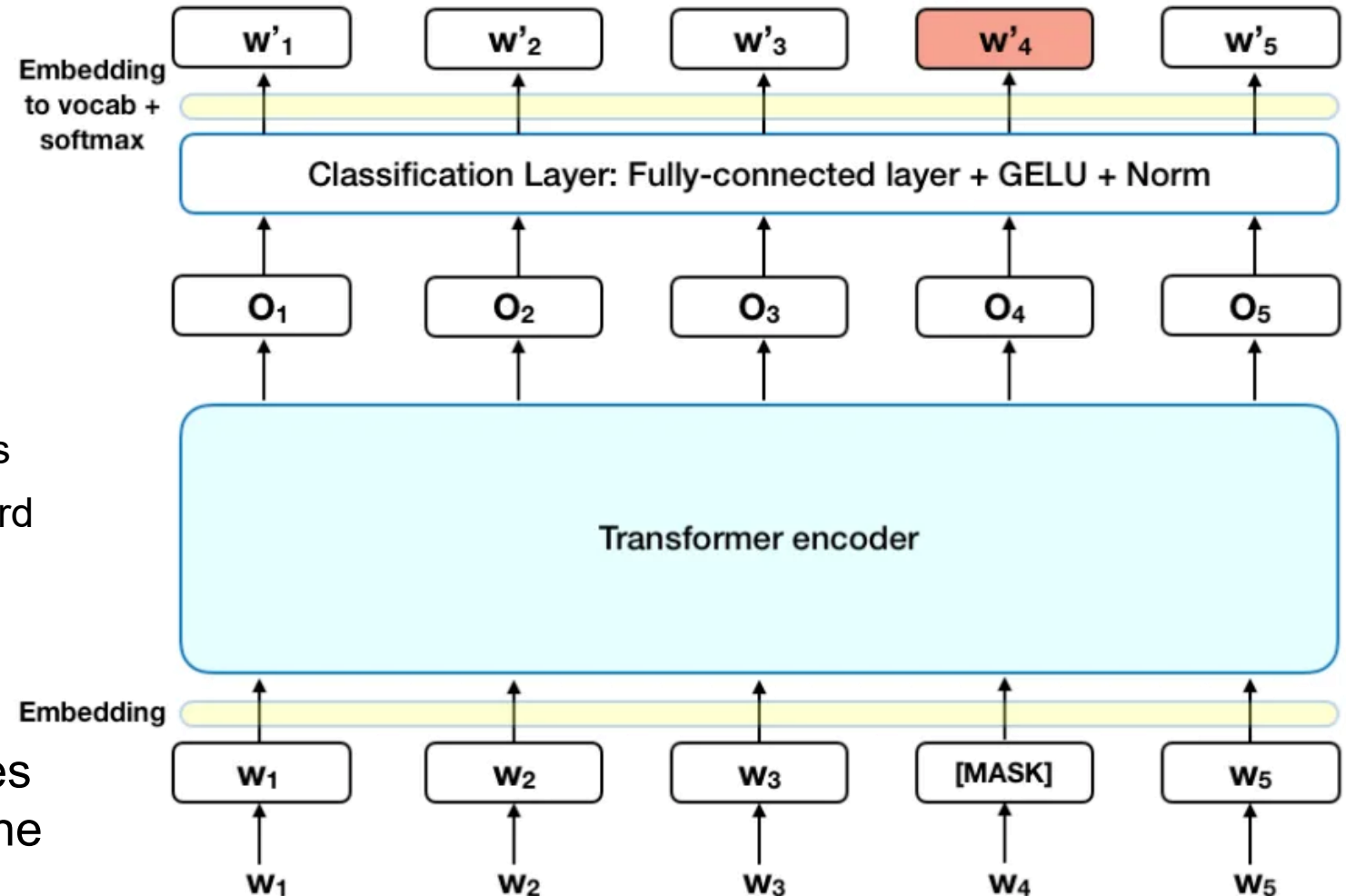
store gallon

↑ ↑

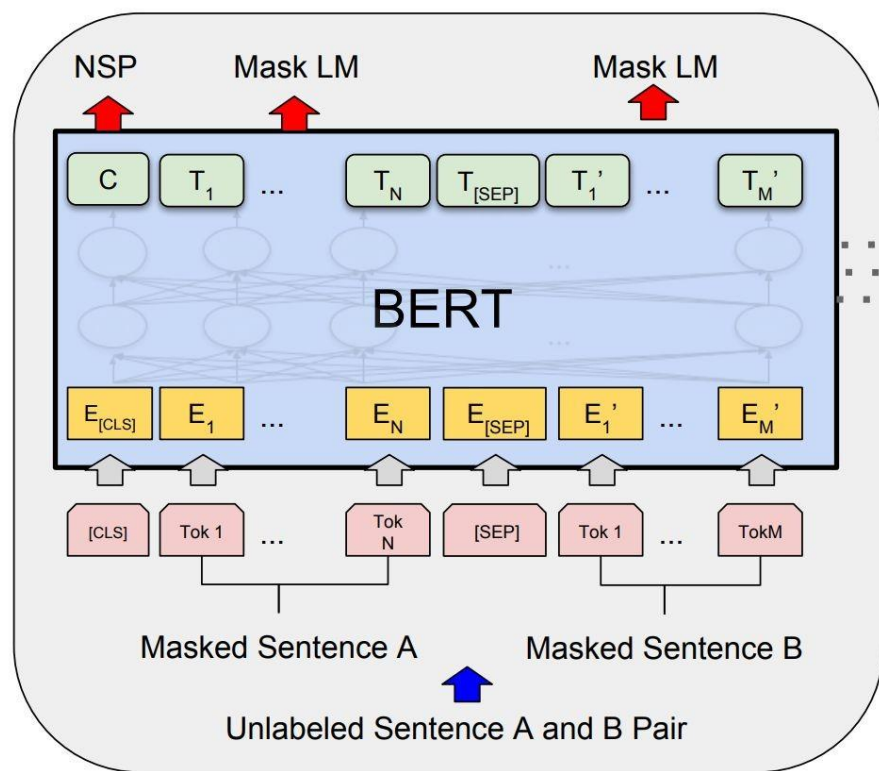
- Too little masking: Too expensive to train
- Too much masking: Not enough context

BERT

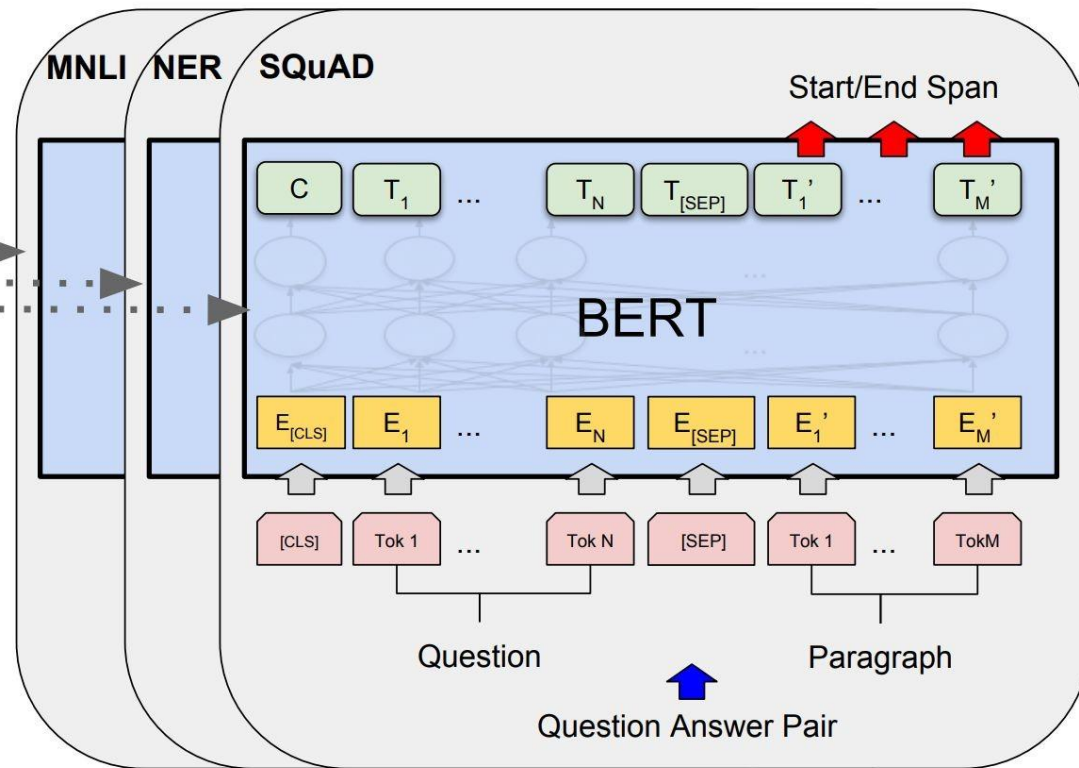
- Mask 15% of the words
- Use context of other words
- Predict the original value
- This requires
 - Classification layer
 - Transform into vocabulary terms
 - Compute probability of each word
- The loss function takes into consideration only the prediction of the masked values and ignores the prediction of the non-masked words



Fine-Tuning Procedure

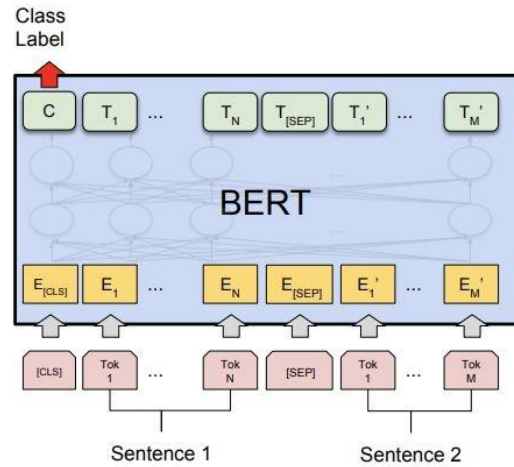


Pre-training

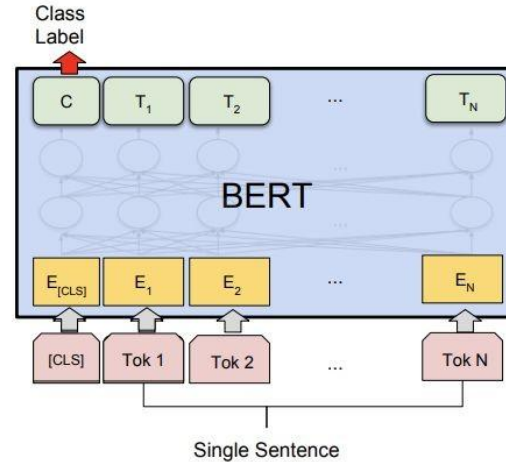


Fine-Tuning

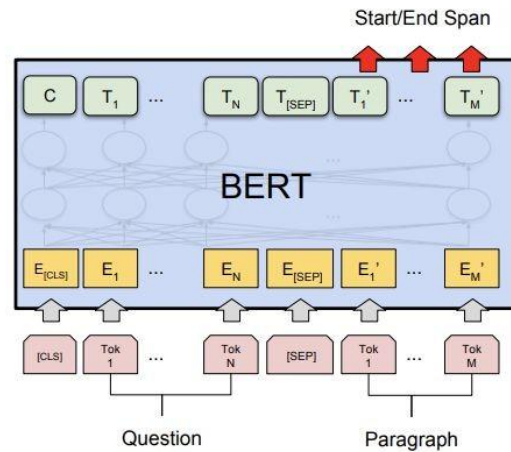
Fine-Tuning Procedure



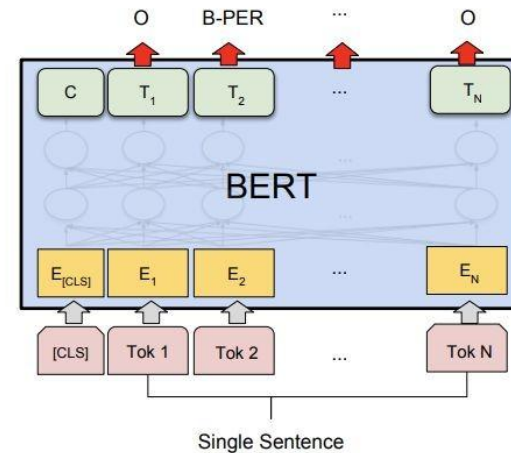
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



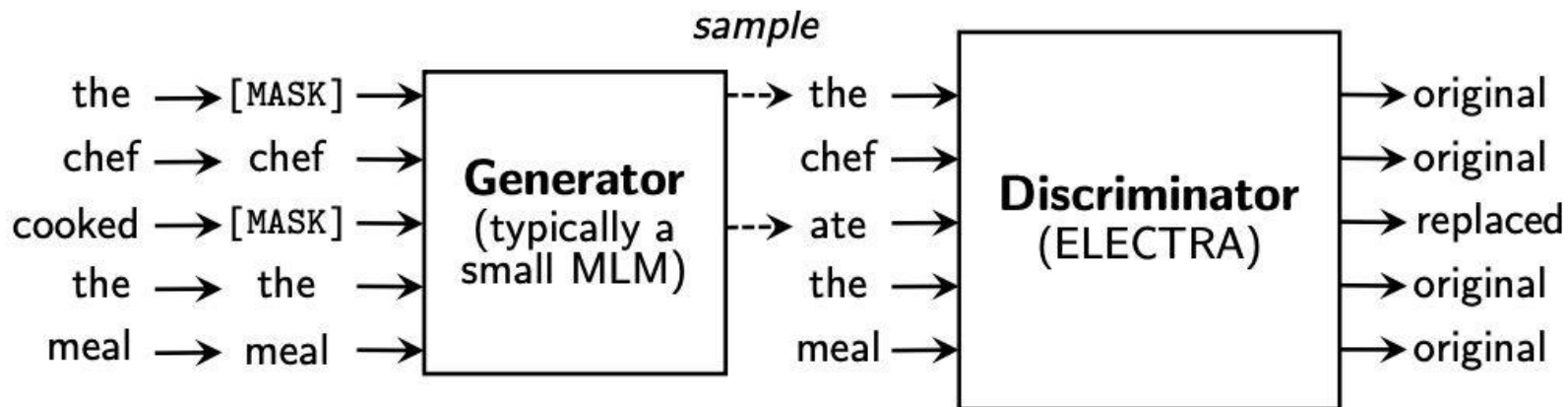
(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

ELECTRA

- *ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators* (Clark et al, 2020)
- Train model to discriminate locally plausible text from real text



ELECTRA

- SOTA results with less compute

Model	Train FLOPs	Params	SQuAD 1.1		SQuAD 2.0	
			EM	F1	EM	F1
BERT-Base	6.4e19 (0.09x)	110M	80.8	88.5	–	–
BERT	1.9e20 (0.27x)	335M	84.1	90.9	79.0	81.8
SpanBERT	7.1e20 (1x)	335M	88.8	94.6	85.7	88.7
XLNet-Base	6.6e19 (0.09x)	117M	81.3	–	78.5	–
XLNet	3.9e21 (5.4x)	360M	89.7	95.1	87.9	90.6
RoBERTa-100K	6.4e20 (0.90x)	356M	–	94.0	–	87.7
RoBERTa-500K	3.2e21 (4.5x)	356M	88.9	94.6	86.5	89.4
ALBERT	3.1e22 (44x)	235M	89.3	94.8	87.4	90.2
BERT (ours)	7.1e20 (1x)	335M	88.0	93.7	84.7	87.5
ELECTRA-Base	6.4e19 (0.09x)	110M	84.5	90.8	80.5	83.3
ELECTRA-400K	7.1e20 (1x)	335M	88.7	94.2	86.9	89.6
ELECTRA-1.75M	3.1e21 (4.4x)	335M	89.7	94.9	88.1	90.6

Applying Models to Production Services

- BERT and other pre-trained language models are extremely large and expensive
- How are companies applying them to low-latency production services?

GOOGLE TECH ARTIFICIAL INTELLIGENCE

Google is improving 10 percent of searches by understanding language context

Say hello to BERT

By Dieter Bohn | @backlon | Oct 25, 2019, 3:01am EDT

Bing says it has been applying BERT since April

The natural language processing capabilities are now applied to all Bing queries globally.

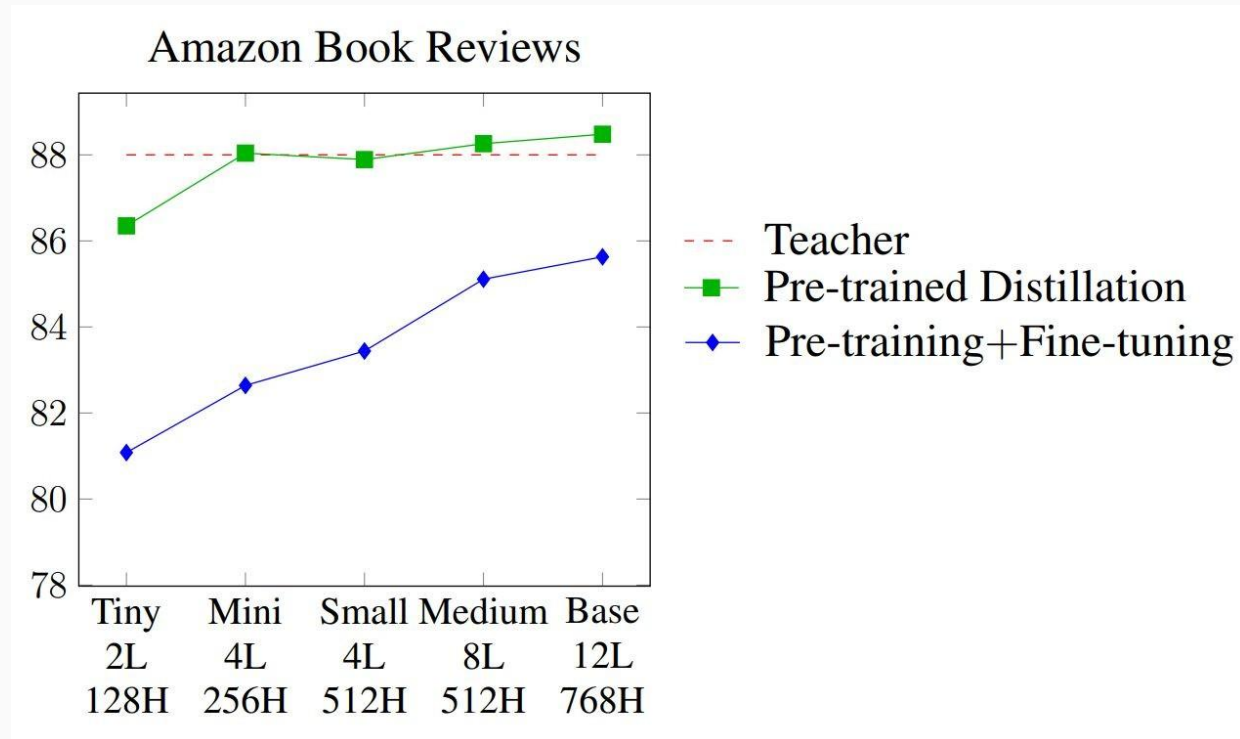
George Nguyen on November 19, 2019 at 1:38 pm

Distillation

- Answer: Distillation (a.k.a., model compression)
- Idea has been around for a long time:
 - *Model Compression* (Bucila et al, 2006)
 - *Distilling the Knowledge in a Neural Network* (Hinton et al, 2015)
 - *Adversarial Network Compression* (Belagiannis, Farshad, Galasso, 2018)
- Simple technique:
 - Train “Teacher”: Use SOTA pre-training + fine-tuning technique to train model with maximum accuracy
 - Label a large amount of unlabeled input examples with Teacher
 - Train “Student”: Much smaller model (e.g., 50x smaller) which is trained to mimic Teacher output
 - Student objective is typically Mean Square Error or Cross Entropy

Distillation

- Example distillation results
 - 50k labeled examples, 8M unlabeled examples



Well-Read Students Learn Better: On the Importance of Pre-training Compact Models
(Turc et al, 2020)

- Distillation works *much* better than pre-training + fine-tuning with smaller model

Conclusions on BERT

- Pre-trained bidirectional language models work incredibly well
- However, the models are extremely expensive
- Improvements (unfortunately) seem to mostly come from even more expensive models and more data
- The inference/serving problem is mostly “solved” through distillation

Thank you

Acknowledges: some slides and material from Fei-Fei Li, Justin Johnson, Serena Yeung, Jeff Donahue, Ashish Vaswani, Anna Huang, Jacob Devlin, Jay Alammarr



SAPIENZA
UNIVERSITÀ DI ROMA